可视计算与交互概论

Baoquan Chen Mengyu Chu

Organized by Elanor

2026 Fall

注：更新目录请 Ctrl+Shift+r

目录

LESSON ONE 0914早八

一、课程导入

二、显示设备的发展

2.1 早期显示设备：从数码管到CRT
2.2 CRT显示器：从矢量显示到二维光栅显示
2.3 彩色显示与颜色空间
  （1）RGB颜色空间
  （2）HSV颜色空间
  （3）色域与人眼可见范围
  （4）伽马校正与颜色位深
2.4 LCD液晶显示器
2.5 OLED有机发光显示器
2.6 色域标准
2.7 分辨率的发展
2.8 立体显示技术展望
  （1）双目视觉原理
  （2）偏振3D与头显技术
  （3）AR/VR/MR
  （4）裸眼3D与全息显示

三、色彩在可视化中的应用

四、图像的定义与表达

4.1 图像的数学定义：连续与离散
4.2 矢量图与光栅图
  （1）矢量图
  （2）光栅图（位图）
  （3）两者的对比

五、帧缓冲区与GPU基础

5.1 GPU的诞生与图形管线
5.2 帧缓冲区（Frame Buffer）
5.3 图像存储格式
  （1）调色板格式（GIF）
  （2）其他常见格式
5.4 Alpha通道与图像合成
5.5 深度缓冲（Z-Buffer）
5.6 双缓冲区

六、图像处理概述与课程预告

课程预告

LESSON 2 0915早十

图像处理：抖动、量化、滤波、边缘提取、补全与抠图

一、图像处理引入

二、抖动与量化

1. 背景
2. 直接二值化的问题
3. 抖动的思想

三、白噪声与蓝噪声

四、滤波与模糊

五、有序抖动与空间换颜色

六、Floyd-Steinberg 抖动

七、图像滤波与卷积 *image filtering*

八、边缘提取、Sobel 与拉普拉斯

九、频率的直观理解

十、图像补全与拉普拉斯编辑

十一、图像克隆、融合与泊松编辑

十二、泊松方程的离散求解

十三、抠图

LESSON 3 2026-09-21 第1-2节

一、课程导入与内容安排

二、光栅化绘制的背景与核心需求

2.1 显示硬件的演进：从矢量到光栅
2.2 帧缓冲区与光栅化流程
2.3 画线操作的普遍性与性能要求
2.4 扫描绘制的核心目标

三、直线绘制的基础算法

3.1 暴力算法及其缺陷
3.2 朴素迭代算法
3.3 朴素算法的性能分析

四、DDA数字微分分析仪算法

4.1 优化思路：乘法转加法迭代
4.2 算法步骤

4.3 仍然存在的问题

五、布雷森汉姆（Bresenham）直线算法

5.1 核心观察：像素步进的二选一
5.2 中点判定法与隐函数思想
5.3 构造整数隐函数
5.4 判别式的迭代更新
5.5 算法推导与迭代过程
5.6 算法伪代码
5.7 算法意义

六、直线算法的扩展

6.1 全空间直线的八分区处理
6.2 扩展到画圆
6.3 多边形扫描线算法
6.4 凹多边形与三角化

七、颜色插值与图像变形

7.1 颜色线性插值
7.2 双线性插值
7.3 图像变形（Image Warping）
7.4 交互式形变的实现
7.5 四边形插值的两种方式

八、曲线内容展望

8.1 曲线的表达需求
8.2 矢量图与曲线
8.3 工业领域的曲线应用：CAD/CAE/CAM
8.4 后续课程预告

LESSON 4 2026-09-22 第5-6节

课程导入

显式曲线

隐式曲线

参数化曲线

参数化曲线的定义与优势
多项式曲线及其不足
分段曲线的思路

插值与基函数基础

线性插值
基函数的概念

贝塞尔曲线

直观认识与构造过程
阶数与递归构造
代码实现思路
数学形式与伯恩斯坦基函数
数值稳定性与矩阵形式

端点与切线性质
连续性：C连续与G连续
贝塞尔曲面（三维扩展）
凸包性与局限性
前沿拓展：犹他茶壶与G2连续问题

其他参数化曲线

三次样条曲线
Catmull-Rom曲线与厄米曲线
    Catmull-Rom曲线
    厄米曲线
B样条曲线简介

课堂问答

问题1：C1连续和G1连续的核心区别是什么？
问题2：厄米曲线和贝塞尔曲线的对应关系是什么？
问题3：伯恩斯坦多项式可以表示任意多项式曲线吗？
问题4：考试中曲线部分的考察重点是什么？

LESSON 5.1（2026-09-28 第1-2节）

第1节 B样条曲线的引入（对应PPT第56页）

1.1 贝塞尔曲线的控制特性回顾
1.2 B样条曲线的定义式
1.3 核心基础概念：节点（Knot）

第2节 B样条基函数的递归定义（对应第57-58页）

2.1 递归起点：0阶B样条基函数
2.2 Cox-de Boor递推公式
    公式解释：
2.3 低阶基函数的形态
2.4 基函数的核心性质：单位分解

第3节 B样条基函数的通用规律（对应第59-60页）

3.1 局部支撑规律
3.2 光滑性规律
3.3 非负性与凸包性质
3.4 局部控制特性的本质

第4节 三次均匀B样条的显式表达（对应第61-62页）

4.1 参数归一化
4.2 三次均匀B样条的显式基函数
4.3 矩阵形式（便于编程计算）
4.4 凸包性质验证

第5节 三次B样条的分段逻辑与核心特性（对应第63-64页）

5.1 滑动窗口的分段机制
5.2 天然高阶连续性
5.3 与三次贝塞尔曲线的等价转换

第6节 三次B样条曲面片（对应第66页）

6.1 张量积构造思想

6.2 曲面的数学表达式
6.3 曲面的核心特性

第7节 样条的推广：节点向量的一般理论（对应第67-71页）

7.1 节点与节点向量的严格定义
7.2 重节点的定义
7.3 均匀节点与非均匀节点
7.4 非均匀节点的核心作用
（1）调整局部控制精度
（2）重节点控制曲线连续性
7.5 重节点降低连续性的原理推导
实例验证（三次B样条，$(p=3)$）

第8节 节点向量的类型与端点插值（对应第72-73页）

8.1 节点间距对基函数权重的影响
8.2 开节点向量与闭节点向量
8.3 闭节点实现端点插值的原理

第9节 NURBS曲线（对应第74-75页）

9.1 核心扩展：控制点权重
9.2 有理基函数与曲线表达式
9.3 权重的几何意义
9.4 与普通B样条的包含关系
9.5 齐次坐标视角下的NURBS推导
步骤1：控制点升维到齐次空间
步骤2：齐次空间中做普通B样条插值
步骤3：透视投影得到三维NURBS

LESSON 5.2 反走样

第1章 走样现象与本质

1.1 什么是走样
1.2 走样的直观成因

第2章 频域分析基础：傅里叶变换

2.1 一维傅里叶变换：从时域到频域
核心思想
数学定义
2.2 空间频率：从一维信号到二维图像
2.3 二维傅里叶分解：图像的基底展开
典型基底与视觉形态
图像的重建过程
2.4 二维频谱的性质

第3章 采样定理与走样的数学本质

3.1 离散采样的数学表达
3.2 采样的频域效应：频谱周期延拓（完整推导）
第一步：采样冲激串的频谱
第二步：卷积的物理意义
核心结论
3.3 奈奎斯特-香农采样定理
临界条件推导

定理正式表述
3.4 混叠的频率折叠效应
3.5 二维图像的采样与走样

第4章 抗锯齿的核心机制：低通滤波

4.1 抗锯齿的两种等价思路
4.2 面积采样：光栅化中的低通实现
4.3 常见的低通滤波核
4.4 基于距离函数的滤波实现

第5章 纹理重采样与Mipmap技术

5.1 图像映射中的走样问题
5.2 Mipmap：预滤波图像金字塔
核心原理
使用方式
5.3 Mipmap的存储开销计算
5.4 透视场景下的层级计算

第6章 几何光栅化抗锯齿技术

6.1 超采样抗锯齿（SSAA）
实现原理
采样点布局
6.2 多重采样抗锯齿（MSAA）
核心思想
6.3 SSAA与MSAA的对比
6.4 几何走样与着色走样的区分

全书知识框架总结

LESSON 6 2026-09-29 第5-6节

一、课程导入与安排

二、几何概述

2.1 几何的词源与研究对象
2.2 经典几何模型：犹他茶壶
2.3 图形学经典数据集：斯坦福四大模型

三、几何的表达分类

3.1 显式表达
3.2 隐式表达

四、点云与激光扫描技术

五、多边形网格基础

六、三角网格的存储方式

6.1 OBJ格式
6.2 Triangle Soup（三角汤）
6.3 两种存储方式的对比
6.4 邻接关系的预计算

七、半边数据结构（Half-Edge Data Structure）

7.1 数据结构定义
7.2 拓扑遍历操作
7.3 应用特点

八、面向渲染的专用网格结构

九、四边形网格（Quad Mesh）

十、多边形网格的优缺点

十一、网格细分（Mesh Subdivision）

11.1 Catmull-Clark细分
11.2 Loop细分

十二、网格参数化（Mesh Parameterization）

12.1 概念与应用
12.2 常见的参数化方法
12.3 参数化的分类
12.4 基于弹簧能量的参数化方法

十三、Lab2 介绍

LESSON ONE 0914早八

二、显示设备的发展

TL;DL:

对于 CRT 显示器而言，我们可以使用三个电子枪分别打在三种颜色的荧光点上，对应红绿蓝三种颜色的像素。液晶显示屏（liquid-crystal display，LCD）使用电压控制偏振光的偏转角度，从而控制白光透过偏振片的强弱，在经过红绿蓝三种颜色的滤光片之后混合出想要的颜色。有机发光二极管（organic light-emitting diode，OLED）屏幕则直接使用红绿蓝三种颜色的有机二极管发出对应颜色的光，并直接控制每种色光的强弱。

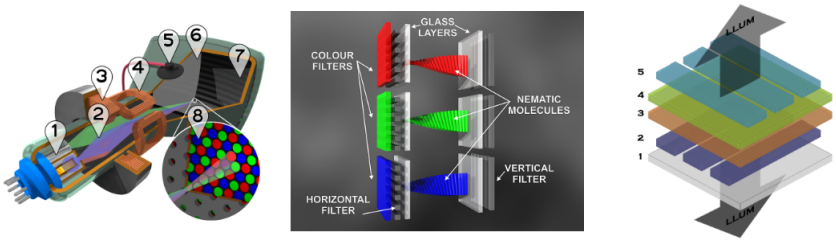


图 3.10 彩色 CRT[1]，LCD[2]，OLED[3] 屏幕的显色原理

2.1 早期显示设备：从数码管到CRT

其中比较早的是1998年的背投式设备，和早期电视类似，采用的是我们稍后会讲到的CRT（阴极射线管）显示技术。
我个人很喜欢2002年发布的台灯式iMac（iMac G4），这是一款非常经典的产品。它是第一款商业化的台式一体机中首次使用LCD屏幕的机型，让屏幕厚度大幅降低，在当时属于高端个人台式机。
等到后续课程讲到人机交互时，我们会更细致地讲解各类交互设备，届时还可以再回顾这款产品。它在当年是兼具设计感与品位的产品，而后来的显示器也沿着越来越薄的方向持续发展，直到今天的形态。

我们先简单梳理显示设备的发展脉络，大家可能对各类硬件已经有一定了解。
最早的简单显示设备是LED数码管，现在依然常见于红绿灯倒计时、嵌入式设备的数字显示等场景。它的特点是控制简单、亮度高、成本低廉，非常适合稀疏的数字与符号显示。但对于图形学而言，它的颗粒度和分辨率远不足以满足视觉显示的需求。

在此之后，显示技术逐步发展，先是出现了厚重的阴极射线管（CRT）显示器，后续又诞生了液晶显示器、等离子显示器等。随着技术进一步发展，为了满足更多前沿交互需求，各类新型显示设备也在持续研发中。以上是显示设备发展的整体概览。

2.2 CRT显示器：从矢量显示到二维光栅显示

接下来我们详细讲解CRT显示器。它曾广泛应用于电视和个人电脑，现在已经属于历史产物。同学们可能未必都用过，但在物理实验中接触过的示波器，就属于同类的阴极射线显示设备。

这类设备的核心是电子枪，它发射电子束，电子束的方向通过电极控制，最终打在前方涂有荧光层的屏幕上，使固定区域被点亮。因为电子带负电，这类射线被称为阴极射线，这类设备也因此得名。

早期CRT显示设备的电子束发射与位置控制频率不高，因此主要应用于矢量显示场景，比如示波器和简单显示单元。它的工作方式类似矢量作图：让电子束按照固定顺序移动，在屏幕上画出线条；快速遍历所有需要点亮的笔画，依靠人眼的视觉暂留和荧光粉的余晖，在视觉暂留时间内呈现完整的画面，满足稀疏内容的显示需求，同时功耗和亮度都相对可控。

原理图中可以看到，我们通过控制电场，让电子束做规则运动，在固定周期内系统性地刷新屏幕，最终呈现出矢量形式的内容。
在当时，硬件会接收算法下发的指令，比如“从某一坐标到另一坐标画一条线段”，硬件底层完成电场控制与划线操作，这是软硬件高度协同的体现。后续课程中我们会讲解直线生成的算法，也会穿插相关的交互实践。

今天我们的核心是显示技术，矢量显示只是其中的一个发展阶段。从矢量显示出发，技术逐步演进到二维光栅显示。
所谓二维显示，就是面向完整屏幕的显示。它的实现逻辑很直接：既然可以控制电子束划线，那就用扫描线对整个屏幕从左到右、从上到下完成完整扫描，所有需要点亮的位置都按照对应亮度点亮。
这是一帧扫描线执行过程的慢速演示。在实际应用中，整个扫描过程会在1/60秒内完成，依靠视觉暂留形成连续画面。早期CRT显示器的分辨率并不高，光栅扫描的性能主要受限于电子束的刷新频率，最初的显示刷新率大约只有24Hz。

从矢量显示到二维显示的演进，催生了一系列影响整个可视计算领域的核心概念：

- **像素**：屏幕上每次被点亮的最小单位区域，就是我们常说的像素。每个像素的亮暗状态是显示的基本单元。
- **分辨率**：由硬件能力决定，指在指定刷新率下屏幕能够刷新的像素总数，是屏幕显示能力的核心指标。
- **纵横比**：屏幕的宽高比例，是符合人眼视觉舒适度的设计参数。
- **帧与帧率**：完整扫描完一整幅画面称为一帧，每秒刷新的帧数就是帧率。

早期逐行扫描的刷新率较低时，人眼可能会感知到扫描的先后顺序。为了解决这个问题，人们发明了隔行扫描算法，在控制电子束移动距离的同时高效完成全屏刷新，同时保持屏幕亮度均衡。这些由硬件发展催生的概念，对后续的软件技术和整个行业都产生了深远影响。

2.3 彩色显示与颜色空间

在灰度二维显示技术成熟之后，人们在屏幕表面涂上红、绿、蓝三种不同的荧光粉涂层，实现了彩色荧光屏。
上节课我们已经讲过颜色的相关原理：对于人眼而言，只要最终接收到的红、绿、蓝三种光的强度相同，无论它来自连续光谱还是离散光谱，人感知到的颜色都是一致的。人类视觉只关心三种颜色贡献的积分总和，因此我们可以用红、绿、蓝三原色的组合，呈现出人类可感知的所有彩色。

颜色补充：视锥细胞负责感知颜色，视杆细胞负责感知强度。

（1）RGB颜色空间

上节课我们已经讲解了红绿蓝三原色，以及离散光谱与连续光谱对人眼的感知效果相近的原理。在此基础上，我们进一步扩展颜色空间的定义。
正如软硬件协同发展的规律，人们发现屏幕可以设置红、绿、蓝三个独立的荧光层，荧光层的发光强度由电子束的击打强度决定，由此可以精准控制每个像素的灰度值，进而构成完整的颜色空间。

这就是对硬件最友好、结构最简单的RGB三原色空间。当RGB三个通道的值均为1.0时，对应白色；纯红、纯绿、纯蓝分别位于RGB立方体的顶点；两色混合可以得到玫红、黄色等常见颜色；立方体内部则对应不同比例的混合色。

在RGB空间中，任意一个点都可以唯一确定红、绿、蓝三个通道的强度，硬件可以直接根据数值控制电子束的位置和强度，最终呈现对应颜色。因此早期的颜色空间设计完全服务于屏幕显示。但在实际使用中，人们发现这种对硬件友好的颜色空间，对人类用户而言可读性较差。如果没有绘画相关的经验，很难通过调整RGB三个通道的数值得到想要的颜色，数值的变化带来的视觉感受是跳跃的、不直观的。

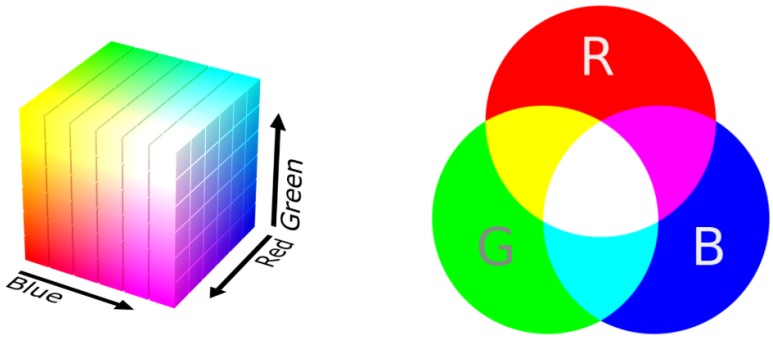


图 3.6 RGB色彩模型[1]

（2）HSV颜色空间

为了定义更符合人眼感知的颜色空间，人们提出了HSV颜色空间。
HSV同样是三通道颜色模型，但它按照人类的感知习惯，将颜色拆分为三个相互独立的语义通道：

- **色调（Hue）**：决定颜色的种类，比如红色、蓝色，只和色相本身相关。
- **饱和度（Saturation）**：决定颜色的浓郁程度，从高饱和的鲜艳色到低饱和的灰调。
- **明度（Value）**：决定颜色的明暗程度，从纯黑到纯白。

拆分后每个通道都有独立的语义，更便于用户控制，因此应用非常广泛。比如Office软件中的调色板，上方的滑块调节的就是色调，选中色调后下方的明暗区域调节的就是明度，这就是HSV模型的典型应用。

HSV的另一个优势是，它与RGB空间可以通过简单的线性变换实现一一映射。这意味着它既拥有人机交互友好的调节界面，又可以精准映射到硬件适配的RGB颜色空间，很好地解决了颜色空间的控制问题。

需要注意的是，HSV空间并不是正交的立体空间。在RGB空间中，坐标不同就对应不同的颜色；但在HSV空间中，同一种颜色可能对应多个坐标点。
举例来说，当饱和度趋近于0时，颜色呈现黑白灰的无彩色状态，此时无论色调是多少，最亮时都是白色，最暗时都是黑色，也就是空间的中心轴是完全重合的。因此HSV空间可视化后并不是立方体，而是一个圆锥状的光锥结构，全黑和全白汇聚为中心轴，不同色调沿圆周分布。这也是HSV更符合人眼感知，但空间结构并非正交的原因。

（3）色域与人眼可见范围

介绍完两种常见的颜色体系，大家通常会关心：人眼可见的光线范围有多大？屏幕、打印机能否完整呈现所有可见颜色？
答案是，设备的显色范围与人眼的可见范围有重叠，但并不完全一致。

人眼的可见光色域整体是一个马蹄形的区域，其中只有30%~60%的区域可以被显示器呈现；相机底片、打印机也只能覆盖其中的部分区域，它们都是可见光色域的子集。
人眼的感知范围远超现有硬件设备，这也是显示技术持续发展的方向。
一个有趣的现象是，早期相机底片的色域要大于显示器和打印机，这也是很多人偏爱胶片摄影的原因之一：底片的成像色域更广，且是连续的、没有明显像素感，视觉感受更舒适。这也是颜色与显示技术协同发展中的一个典型案例。

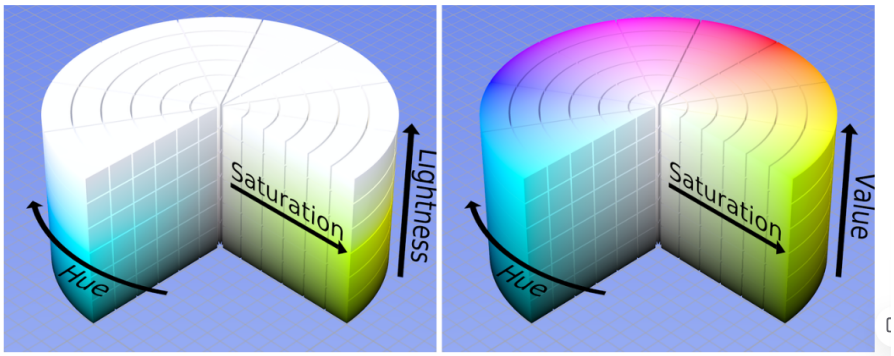


图 3.8 HSL 与 HSV 的区别[5]

（4）伽马校正与颜色位深

另外一个重要的概念是伽马校正。
人眼对亮度的敏感程度是非线性的：白天多开一盏台灯，人眼几乎感知不到差异；但夜晚门缝透出的微光却很容易被察觉。也就是说，人眼在暗部的亮度分辨率更高，在亮部的分辨率更低，均匀的亮度分布对人眼而言信息效率并不高。

因此我们会对图像的灰度值做非线性的伽马映射，将人眼敏感的暗部区域相对拉长，亮部区域相对压缩，让人眼可以更好地分辨图像中的细节。伽马校正是大多数图像软件都会自动完成的基础操作。

硬件领域一直在追求更高的**色深**。图像最终以离散形式存储和显示，每个颜色通道的比特数（比如8bit、32bit、64bit）决定了颜色可区分的层级，也定义了低通道、标准通道和高通道的颜色显示。

伽马校正详细补充：https://vcl-pku.github.io/vci-book/getting-started/color/color-display.html

伽马值决定了"你给的信号"和"实际亮度"之间差了多少倍。

（5）CMYK色彩空间

CMYK 色彩空间主要服务于印刷行业，称为**印刷四分色空间**。C、M、Y、K 分别对应四种打印色料：青色（cyan）、洋红色（magenta）、黄色（yellow）、黑色（key plate，或者 black）。与 RGB 颜色模型不同，CMYK 色彩模型的混色方式正好相反，C、M、Y 三种颜色的混合接近黑色而不是白色。这是因为对于印刷品而言，其显示的颜色对应其吸收颜色的补色。比如红色颜料是因为其本身会吸收主要的蓝色、绿色光，导致其反射自然光之后为红色。由于印刷品和显示屏显色原理上的根本不同，很难做到印刷出来的颜色与显示屏显示的颜色一致。

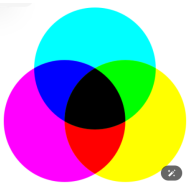


图 3.9 比较接近实际 CMY 叠色的示意图

CMYK 与 RGB 之间也存在转换关系，由下面的公式给出：

$$R = (1 - C)(1 - K)$$

$$G = (1 - M)(1 - K)$$

$$B = (1 - Y)(1 - K)$$

$$\begin{aligned} C &= 1 - \frac{R}{\max(R, G, B)} \\ M &= 1 - \frac{G}{\max(R, G, B)} \\ Y &= 1 - \frac{B}{\max(R, G, B)} \\ K &= 1 - \max(R, G, B) \end{aligned}$$

2.4 LCD液晶显示器

人们对显示硬件的追求始终是更便携、更清晰。液晶显示器（LCD）的出现是显示技术的一次重大飞跃，它大幅降低了设备厚度，同时显著提升了视觉分辨率。

LCD的基本结构从底层到上层依次为：背光层、两层偏振片、中间的液晶层，以及彩色滤光片。

- **背光层**：目前绝大多数采用LED作为光源，提供均匀的白光，类似在屏幕后方放置一盏台灯。LCD的主要能耗来自背光层，整体功耗相对较低。统一背光的设计省去了CRT中厚重的电子枪与偏转结构，是屏幕厚度大幅降低的核心原因。
- **偏振片与液晶层**：第一层偏振片控制入射光的偏振方向；液晶层通过改变液晶分子的排列，调节穿过光线的偏振方向。如果液晶层调整后的偏振方向与第二层偏振片完全一致，光线全部通过，对应亮点；如果方向完全垂直，光线完全无法通过，对应暗点。液晶层体积小、响应灵敏，可以高效控制每个像素的出射亮度。
- **彩色滤光片**：在偏振片之后加入红、绿、蓝三色滤光片，使不同像素透出对应颜色的光，最终进入人眼形成彩色画面。

LCD技术发展至今已经非常成熟，它的主要优势是功耗较低；劣势在于背光层常亮，显示纯黑画面时无法完全关闭背光，会存在漏光与余晖，因此屏幕对比度存在上限。

2.5 OLED有机发光显示器

LCD之后的下一代主流显示技术是OLED（有机发光二极管），它与微缩电路技术的发展紧密相关。

LCD的思路是统一发光、通过液晶控制偏振；OLED的思路则是在屏幕上排布大量独立的发光单元，每个单元独立控制颜色与亮度，省去了液晶、偏振控制等复杂结构。显示白色时，发光单元全力发光；显示黑色时，发光单元完全关闭，因此可以实现极高的对比度，视觉效果更纯净。

OLED的性能高度依赖硬件的微缩工艺：发光单元的尺寸决定了屏幕的分辨率。除此之外，它对硬件工艺还有很高要求：

1. 所有发光单元的质量需要高度一致，避免个别单元提前老化，出现坏点影响整体显示效果；
2. 需要保证相邻区域的发光单元使用均匀，减少老化不均的问题。

目前OLED多用于中高端便携设备，电视等大功率大屏仍以LCD为主。OLED的功耗整体高于LCD，且高亮度下发光单元的寿命衰减更快，因此OLED屏幕的峰值亮度通常低于LCD。但在显示质量上，OLED的观感普遍更优。

2.6 色域标准

接下来我们介绍与颜色显示相关的行业标准。
最早的相关标准由美国国家电视标准委员会制定，以高端高动态范围（HDR）显示设备为基准，定义了100%色域的标准，早期只有少数设备能够达到，苹果是较早采用该高标准的厂商。

面向更广泛的个人电脑与打印设备，微软、惠普等厂商制定了sRGB标准，色域覆盖约为基准的72%，对应CIE色度图中更深的蓝色三角形。Windows系统、主流消费级显示器与打印机均采用sRGB色域，是目前最通用的显示标准。

除了显示与打印需求，设计领域对色域有更高的要求。艺术家需要更广的色域进行创作，作品最终可能用于海报打印、3D打印等场景。为此Adobe提出了Adobe RGB标准，对应图中红色区域，色域范围更大。
目前主流的显示与打印硬件还无法完全覆盖Adobe RGB，它更多是设计领域的概念标准，同时也是硬件与打印技术未来的发展方向。

大家未来如果有打印海报等需求，对方通常会询问颜色设置。如果使用了Adobe RGB色域，需要选择转换为标准sRGB还是高动态范围色域，标准色域打印成本更低，高色域打印成本更高。打印时会在目标色域内选取最接近的颜色，还原设计效果。

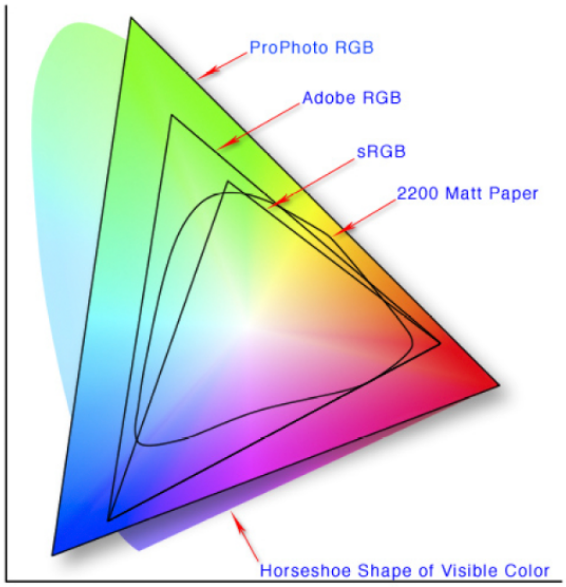


图 3.7 不同 RGB 色彩空间的色域，可视化 $R+G+B=1$ 的切面[4]

2.7 分辨率的发展

回到显示设备本身，一个非常直观的趋势是分辨率逐年提升。早年1080P已经属于高分辨率，后来逐步出现4K需求，现在游戏设备普遍追求4K，更高的还有8K。
分辨率的提升对显卡的存储、渲染与刷新能力都提出了更高要求，这部分我们在后续图像章节会详细讲解。超高分辨率不仅用于个人电脑，也广泛应用于大型展板、微缩模型展示等场景，提供更精细的视觉效果。

2.8 立体显示技术展望

以上是平面显示的核心内容，作为后续图像知识的基础铺垫。接下来我们展望立体显示技术，这部分在人机交互章节会有更详细的讲解，这里仅作为显示章节的补充。

（1）双目视觉原理

人类天生的立体视觉主要来源于双目视觉：两只眼睛存在视角差，看到的物体在遮挡关系、相对位置上存在细微差异。人类经过长期进化，已经可以根据这种差异判断深度：近处物体偏移小，远处物体偏移大，最终从两幅平面图像中恢复出深度信息。这是所有立体显示技术的生理基础。

（2）偏振3D与头显技术

最早的立体显示技术是偏振式3D眼镜：屏幕发出两组偏振方向不同的光，眼镜上的偏振片分别筛选左右眼对应的画面，大脑自动将两幅画面合成为3D视觉。这是后续所有立体显示的基础原理。

在此基础上，结合头部追踪设备可以实现更沉浸的体验：通过追踪头部位置定位双眼位置，分别计算左右眼应该看到的画面，再由显示设备向对应方向投射光线。这就是裸眼3D与人眼追踪的基本设计。
纯裸眼3D设备只能针对单个人进行投射；如果佩戴专用眼镜，则可以支持多人使用。无论是戴眼镜还是裸眼方案，核心都是结合人眼位置进行定位与画面计算。

（3）AR/VR/MR

我们可以看一下AR/VR的效果演示。图中的画面是合成效果：真实场景中人物站在绿幕中，我们还原了他在头显中看到的内容。VR头显向双眼分别投射对应的画面，营造立体感，结合手柄可以实现空间拾取等交互，是可视交互在虚拟现实方向的重要发展。

- **虚拟现实（VR）**：呈现完全虚拟的环境，用户看到的所有内容都由计算机生成。
- **增强现实（AR）**：用户可以直接看到真实环境，设备会在真实场景中叠加虚拟人物或信息，实现真实与虚拟的共存。AR技术在游戏等领域已经有成熟应用。

关于AR、VR以及混合现实（MR）的更多内容，我们在虚拟现实章节会详细介绍。

（4）裸眼3D与全息显示

除了近眼显示，裸眼3D显示器也是大家很感兴趣的方向。
裸眼3D的核心思路是：在LCD屏幕前增加一层视差屏障，控制光线的出射方向，让不同方向的光线分别进入左右眼。这样一个像素可以拆分为左右眼两个子像素，分别显示对应画面，无需佩戴眼镜即可实现3D效果。
这种方案的代价是屏幕有效分辨率会折半，同时需要底层高分辨率、高亮度的屏幕，以及精准的视差屏障和人眼追踪技术，目前仍在持续研发中。

除了基于屏幕的立体显示，还有更接近真实立体的显示技术：

- **旋转式立体显示**：发光设备高速旋转，依靠视觉暂留在空间中形成持续发光的立体影像，目前仅能在小范围内实现。
- **全息显示**：在立体空间中照射强光，通过全息底片的干涉与衍射效应，在空间中呈现真实的立体光点。全息显示的原理来自物理光学：拍摄时，一束激光照射物体，另一束同相位的参考光直接打到底片上，两束光干涉后，底片记录下物体光波的相位信息；再现时，只用参考光照射全息底片，衍射出的光波与原物体的反射光完全一致，人眼看到的效果就和真实物体相同。

早期全息显示使用物理底片，现在已经发展出数字全息摄影与显示技术，但受限于硬件，目前的显示区域仍然很小。

还有一些更前沿的多模态显示技术，比如触觉显示（通过特定分辨率的纹理让人感知触觉图案）、嗅觉显示（控制物质挥发成分产生不同嗅觉）、体感交互等，这些都属于显示方向的前沿研究，技术尚未完全成熟。

三、色彩在可视化中的应用

我们先补充色彩在可视计算领域的应用案例。
色彩是人眼感知信息的重要渠道，因此也是数据可视化的核心手段。举个例子：统计不同地区的饮料消费偏好，如果只用表格展示数据，很难直观体现地域分布的倾向性。因此可视化中最常用的方法是结合地图，用不同颜色对应不同数据，直观展示区域特征。

但简单的颜色填充也存在问题：一是美观度不足，二是数据重叠区域的信息表达不清晰。
一种常见的改进思路是，根据数据成分对整个区域进行颜色渐变绘制：比如可乐消费高的区域偏蓝，气泡水消费高的区域偏黄，两者混合的区域呈现橙色。这种方式比点状标注更美观，信息也更均匀。

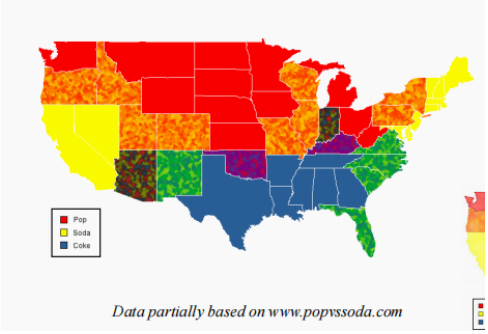
但这种方法依然存在认知误区：如果图例中只有三种颜色，混合出的中间色很容易被观众解读为第四种、第五种独立语义，造成信息误解。这说明不能简单地将颜色直接混合，用户会自然地将混合色识别为新的类别。

可视化领域借鉴了真实世界的颜料混合现象来解决这个问题：
水彩颜料如果提前在调色盘里混合均匀，涂抹后会呈现第三种颜色；但如果先后在画布上涂抹黄、红、蓝三种颜色，颜料无法完全溶解混合，最终画面会保留三种颜色的斑驳痕迹，同时存在局部混合。观众会自然地将其理解为三种颜色的叠加，而不是一种新的颜色。
油画也有类似的特性：颜料干燥后覆盖性强，未干时则可以互相融合。通过控制颜料的融合与稀释程度，可以控制画面呈现“融合”还是“独立叠加”的视觉效果。

基于这个思路，可视化设计衍生出了**噪声叠加混合**的方法：纯色区域使用对应颜色，混合区域则给不同颜色分别叠加随机噪声，让三种颜色以斑驳的形式叠加。最终画面中既有均匀混合的区域，也有保留原色的区域，更接近真实颜料的混合效果，也能更清晰地表达“多语义共存”的含义，而不会被误读为新的类别。

我们可以把这个技术应用到地图可视化中：比如分别用两种颜色可视化致死蚊虫分布区域和活火山活动区域。如果直接颜色混合，中间的黄色会带来语义歧义；如果采用噪声叠加的方式，斑驳的混合区域会被直观理解为两种现象的重叠区域，能更好地展示多维度数据的地理分布。饮料消费的可视化也可以用同样的方法扩展，清晰呈现不同偏好的交界与混合区域。

这是一种更符合人类视觉认知、能准确表达混合语义而非新增语义的可视化方法。



四、图像的定义与表达

4.1 图像的数学定义：连续与离散

图像可以定义为二维空间上的四元组：

- 前两个元素是像素的空间坐标(x, y)，对应二维位置；
- 第三个元素是波长λ，对应该位置光的波长与浓度，也就是颜色，可以用RGB、HSV等颜色空间表示；
- 第四个元素是时间t，对应曝光时长，会影响画面的亮度。

  注：这个四元组里的 t ***是图像的内在属性****，描述的是“这张图是用多长时间曝光出来的”。亮度 = 光的强度 × 曝光时间*

这个定义具有普适性：它既适用于电脑中的JPEG、PNG等数字图像，也适用于胶片底片——底片上每个位置的曝光程度、色彩浓度，都可以用这个四元组描述。

这里有一个非常重要的认知：虽然我们日常接触的数字图像是固定分辨率的离散表达，但图像从数学本质上是连续的。胶片底片的曝光是连续能量作用于化学物质的过程，每个点的曝光程度都是连续变化的，因此图像本质上是一个连续的四元函数。
只是在数字计算机中，我们无法保留连续的本质，最终会以离散的方式存储和显示。

这个“连续本质、离散呈现”的趋势贯穿整个计算机图形学领域：真实世界中的物体，无论是图像、立方体、球体还是流体，本质上都是连续的，都可以用连续函数拟合；但最终都要以离散的方式显示在屏幕上。
计算机图形学的核心内核之一，就是建立连续世界到离散空间的映射，通过离散空间的计算解决连续空间的问题——比如渲染解决光照能量分布的问题，仿真解决连续物理场在离散空间的呈现问题，本质上都是在处理两个世界的衔接与转换。

4.2 矢量图与光栅图

将连续图像离散化主要有两种思路：

（1）矢量图

第一种是将图像离散为几何原语，比如点、线段、曲线等。
矢量图的构成是离散的几何元素，但在显示时可以无限缩放，不会失真。它的优势是可以适配任意分辨率，非常适合图标、设计曲线、工程图纸等场景。从硬件角度看，矢量图和早期的矢量显示器、示波器天然匹配，适合精确的曲线展示。
它的不足是表达复杂画面的能力有限，对复杂图像的编辑效率较低。

矢量图广泛存在于我们的生活中，各类系统图标基本都采用矢量格式，目前最通用的格式是SVG（Scalable Vector Graphics）。
SVG文件内部是语义化的元素定义，比如通过`polygon`标签定义多边形的坐标、填充色和边框，就可以绘制出一个红色填充、黑色描边的四边形；多个四边形组合就可以构成立方体的二维投影。
对于规则图形，SVG可以用极低的信息量实现高质量表达。大家可以在在线编辑器中尝试SVG的编写。

（2）光栅图（位图）

第二种也是我们最熟悉的方式，就是光栅化。
“光栅化”这个名字就来自光栅化显示器：电子束逐行扫描屏幕，逐个点亮像素，图像也就对应地存储为像素的二维数组，每个像素记录对应的颜色。这是现在最主流的图像表达方式。

光栅图的优势是颜色控制简单，适配显示、摄影等绝大多数场景，随着硬件的普及成为了行业主流，这也再次体现了软硬件协同发展的规律。
它的不足是信息量与分辨率直接绑定，缩放时会出现锯齿、模糊等问题；后续我们会讲解反走样技术，就是为了在有限分辨率下尽可能优化显示效果。此外光栅图需要存储每个像素的颜色信息，存储密度远高于矢量图。

矢量图诞生于矢量显示器时代，光栅图则诞生于光栅扫描显示器。如今LCD和OLED成为主流，狭义上的“图像”通常就是指光栅化的像素二维数组。

五、帧缓冲区与GPU基础

5.1 GPU的诞生与图形管线

软硬件协同的规律不仅体现在显示硬件定义了数字内容的表达，也体现在计算单元定义了数字内容的计算方式。

当图像以像素二维矩阵的形式存在后，大量图像处理操作都是对每个像素执行相同的运算，比如统一调整亮度、伽马校正等。这类操作高度并行，而CPU的单线程架构并不适合这种简单重复的并行任务，因此GPU（图形处理器）应运而生。

GPU是专门为简单并行运算设计的硬件。它内部包含专门的存储单元，早期完全用于存储下一帧要显示的像素数据；同时集成了大量并行计算线程，专门处理像素级的统一操作。
为了高效完成图形计算，GPU设计了专门的图形管线，可以高速完成下一帧画面的计算与调整，再配合屏幕刷新率，将数据从帧缓冲区刷新到屏幕上，完成显示输出。
后续渲染章节我们会详细讲解GPU如何从简单的显示辅助单元，逐步发展出完整的图形学功能，这也是可视计算课程的核心脉络之一。

5.2 帧缓冲区（Frame Buffer）

图像通常存储在帧缓冲区中。下一时刻屏幕要显示的内容，会以二维图像的形式保存在帧缓冲区里，每个像素对应颜色与亮度信息。
显卡上的这块专属存储区域就是帧缓冲区，它会按照屏幕的刷新率（比如60Hz）将数据刷新到屏幕上，这是硬件底层的核心工作。

我们常说的显示器“位深”，指的就是帧缓冲区的比特长度。

- 仅显示黑白内容的简单设备，1bit就足够；
- 基础彩色设备至少需要8bit；（RGB每个通道8bit）
- 主流个人电脑通常是24位(8*3)或32位(8*4,带α)；

从图像表达的角度，帧缓冲区的每个像素可以包含多个通道，对应不同的信息，这是硬件与图像标准协同设计的结果。

最常见的是24位真彩色：将24位拆分为三个8位通道，分别对应RGB三原色，每个通道的取值范围是0~255（2^8），总共可以表示约1677万种颜色。这个规格在显示效果和成本之间取得了很好的平衡，因此被称为“真彩色”。

5.3 图像存储格式

直接存储全色彩的像素矩阵会占用大量空间，因此衍生出多种图像压缩与存储格式。

（1）调色板格式（GIF）

如果一张图片使用的颜色数量远少于1677万， 可以将所有用到的颜色存入一个颜色表（Color Map），每个像素只存储颜色表的索引，而不是完整的RGB值，从而实现压缩。
GIF格式就是典型的调色板模式，通常使用256色调色板，每个像素仅需8bit。GIF支持多帧动画，连续帧的颜色表通常高度连续，因此可以高效压缩。
调色板的大小可以配置，位数越高颜色还原越好，但压缩率也会相应降低。纯黑白图像甚至可以只用1bit存储，空间效率极高。

（2）其他常见格式

- **BMP**：无压缩的位图格式，直接存储像素原始数据，体积最大。
- **JPEG**：通用有损压缩标准，压缩比高，广泛用于照片存储，不同质量等级对应不同的精度损失。
- **TIFF**：支持多层图像，常用于摄影、印刷等专业场景，不同图层可以使用不同的位深。
- **PPM**：简单的图像格式，同样支持多层表达。
- **EPS**：常用于字体、印刷领域，文件体积较大，打印兼容性好。
- **PNG**：无损压缩格式，体积比BMP小，压缩率低于JPEG，支持透明通道，是目前互联网的主流格式之一。

JPEG和BMP不支持透明通道，默认以黑色作为背景；PNG支持透明通道，是设计与网页开发的常用格式。

5.4 Alpha通道与图像合成

帧缓冲区（*帧缓冲区是显卡上一块专门的内存，用来存放「准备输出到屏幕上那一张完整画面的全部像素数据」*）的位深通常会高于24位，额外的位数用于存储更多信息，其中最常见的就是Alpha（透明度）通道。

帧缓冲区的绘制是分层进行的，后绘制的内容会覆盖之前的内容，Alpha通道用于控制新图层的透明度，决定背景可以透出多少。最终画面混合完成后，未被不透明像素覆盖的区域会填充背景色，再输出到屏幕。

Alpha通道的取值范围是0~1：

- 1.0表示完全不透明；
- 0.0表示完全透明；
- 中间值对应半透明效果，非常适合表达阴影、光晕、光标、带透明效果的Logo等。

最基础的线性混合公式为：

$$\text{最终颜色} = \alpha \times \text{前景色} + (1-\alpha) \times \text{背景色}$$

Alpha通道广泛应用于影视制作：前景在绿幕中拍摄，后期将绿色区域的Alpha值设为0，就可以叠加任意背景，这就是绿幕抠图的核心原理。CG虚拟角色与实景的合成，也依赖多层Alpha通道的叠加计算。

5.5 深度缓冲（Z-Buffer）

除了Alpha通道，帧缓冲区中还会存储深度信息，也就是Z-Buffer（深度缓冲区）。
3D场景最终要呈现在2D屏幕上，每个像素对应的空间点到相机的距离，就是深度值。
绘制多个物体时，通过比较深度值可以判断哪个物体在前方、哪个在后方，决定该像素最终显示哪个物体的颜色，以及透明混合的顺序。这些逻辑现在都已经固化在GPU硬件中。

深度信息通常不用8bit存储，至少使用16bit，很多时候采用指数形式存储。原因是人眼对近处的深度变化非常敏感，遮挡关系是很强的视觉线索，需要高精度；而远处的深度精度要求较低。指数存储可以在近处提供更高的精度，远处精度较低，符合人眼的感知特性，同时也会占用更多的位深。

感觉很多时候就是结合人的心理来做的啊...也有道理，毕竟“显示”本身就是以人作为最后的消费对象来发展的

5.6 双缓冲区

随着帧率提升，硬件厂商普遍采用双缓冲区机制。
假设显示器正在显示帧缓存 A 中的内容，那么软件更新只会更新帧缓存 B 中的内容。假设显示器下次刷新时 B 中的内容还未更新完全，那么显示器会继续读取 A 中的内容显示到屏幕上。直到 B 中的内容更新完毕，显示器就会切换到 B 中的内容进行显示，而软件则会切换到更新 A 中的内容，如此往复。显示器在这个过程中的模式称为**垂直同步（V-sync）** 模式。

六、图像处理概述与课程预告

当图像以离散二维矩阵的形式存在时，连续到离散的映射会产生各种问题，因此需要图像处理技术来弥补。

图像处理可以按照处理范围分为三类：

1. **逐点处理（Point Operation）**：对每个像素独立进行运算，像素之间互不影响。比如图像拷贝、反色、伽马校正、亮度调整、曲线调色等。这类操作最简单，也最适合GPU并行计算。
2. **区域处理（Filtering，滤波）**：对像素的邻域进行运算，提取图像的连续特征，比如去噪、边缘检测、模糊等。滤波和数字信号处理、信号传输理论联系紧密，本质是在离散表达上运算离散算子，提取连续意义上的信息。
3. **全局处理**：对整幅图像进行操作，比如图像变形、压缩、语义分割、背景抠除，以及更高层的图像理解、视频处理等。

今天由于时间限制，我们不会深入讲解区域处理。后续我们会按照从局域到全局的顺序，依次讲解：

- 抖动技术：用更低的颜色位深模拟更丰富的颜色观感；
- 图像滤波；
- 图像补全、背景匹配；
- 直线生成、三角形光栅化、反走样、曲线绘制等内容。

这些都是课程基础部分的核心重点，也是第一次考试的主要考察内容。

LESSON 2 0915早十

图像处理：抖动、量化、滤波、边缘提取、补全与抠图

一、图像处理引入

今天讲图像处理部分，承接之前的图像表达。图像处理主要针对离散的、光栅化表示的图像。我们走一个从局部到全局的过程：一开始是局部计算，之后逐渐涉及全局约束、需要统一求解的运算。这是比较重要的算法内容，也是考试中比较重要的一部分。

二、抖动与量化

1. 背景

图像处理最前面的问题是抖动和量化。这里的量化，指的是把图像压缩成位数更低的表达。这是一个有实际应用需求的问题。

最初，人们把图像作为一种表达分离出来之后，可以存一个比较大的文件，表达比较多比特的图片，比如一张 8 比特灰度图 *（注：每个像素用一个8位二进制数来表示亮度，只有明暗信息，没有颜色信息.能表示2^8=256个不同的灰度等级）*。但图像最终还要在各种显示设备上显示。有时设备只有 1 bit，只能显示黑白两色；有时输入是 24 位真彩色 RGB 图像，但显示设备只能显示 8 比特灰度。因此，我们希望有一些压缩方法，使图像在这些设备上仍有较好的展示效果。这就是抖动和量化的背景。

2. 直接二值化的问题

如果把一张 8 比特灰度图显示成黑白图像，最简单的方法是阈值截断：灰度大于等于 0.5，或大于等于 128，就置为白色，即 255 或 1.0；小于阈值就置为 0。这样会得到一张黑白图。

对每个像素来说，这种做法的误差已经最小。但人们发现，这种视觉成像有很多不令人满意之处，于是提出了抖动。

误差不仅有大小的区别，还有频率的区别。将原图和二值化结果做差，得到二维误差图。对误差图做傅里叶分解，就转换到频率空间。我们关心不同频率 k 上误差的能量或振幅。以频率为横轴、能量为纵轴可视化，会发现：直接二值化时，每个像素误差最小，但误差全部处于最低频率，几乎没有周期性变化。

注：1.误差图：对每个像素，误差就是：

$$e(x,y) = I_{原图}(x,y) - I_{二值化}(x,y)$$

例如：原像素是 200，二值化后是 255，误差是 $200-255=-55$；

2.频率代表什么：表示信号变化的快慢。在图像里，低频意味着图像在很大范围内缓慢变化，比如大块平滑过渡或纯色；高频如边缘、纹理、噪点。

3.为什么误差都在最低频？直接二值化后，误差图往往是这样：

- *一大片浅灰色区域，所有像素都变成白色，误差都是差不多的负值；*
- *一大片深灰色区域，所有像素都变成黑色，误差都是差不多的正值。*

所以误差在空间上变化很慢，几乎没有频繁跳动。

对这样的误差图做傅里叶变换，能量就集中在低频。
高频部分几乎为零。

这就是"误差全部处于最低频率，几乎没有周期性变化"。

3. 抖动的思想

抖动做了一个有趣的事情：先给图像随机加入人为噪声，再做二值化。直觉上，加噪声会丢失信息，再做二值化又进一步丢失信息。但从视觉上看，很多人会觉得这张图传递了更多原始图像的信息。

同样可以计算误差并做频率分解。直接二值化的逐像素 L2 loss 可能更小，但抖动后的误差分布在低频和高频都有。人类视觉对低频误差非常敏感：大片区域系统性偏低或偏高，会被认为两张图差异很大，比如对比度不同、细节特征不同。但高频误差，例如局部噪点，人眼由于进化原因反而不敏感。

因此，虽然每个像素本身的选择不一定最优，但均匀噪声在局部区域中减少了低频偏差。所以，

抖动可以定义为：

> 把结构化的、容易被人眼察觉的低频量化误差，转换成频谱更加分散的噪声。

三、白噪声与蓝噪声

随机噪声称为白噪声。对噪声做空间到频率的傅里叶变换，会发现白噪声完全随机，因此频率分布比较满、比较连续。

另一种噪声只包含高频信息，称为蓝噪声。给原图加蓝噪声后，低频区域更接近原图，误差被累积到高频。人眼对高频不敏感，所以图像主体、黑 T 恤、眼镜等低频特征仍然可识别。

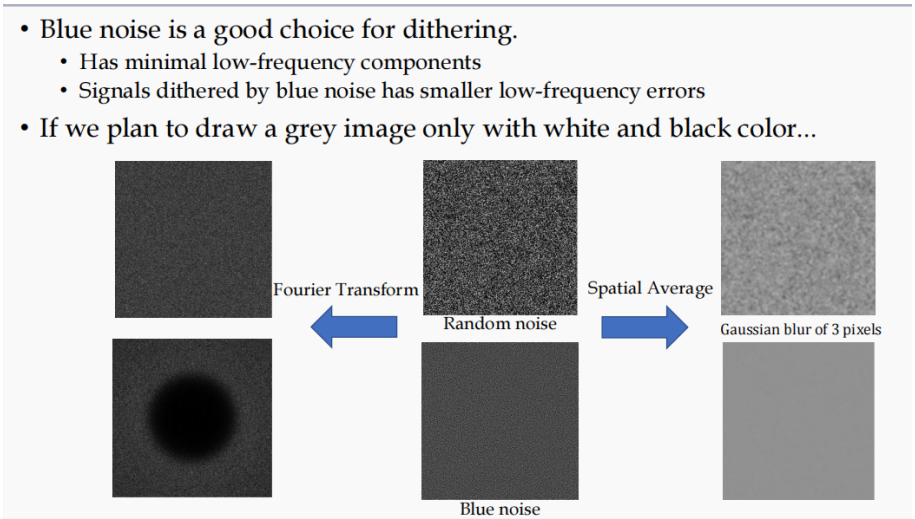
注：蓝噪声常常表现为，点分布比较均匀、不会出现大片空白也不会很多点挤在一起。

对比误差能量，蓝噪声的总能量甚至可能比白噪声更高，但误差都分布在高频，低频误差被抑制，因此视觉感受更好。

四、滤波与模糊

噪声可以作为图像信息源，再应用图像滤波。常见滤波包括模糊处理。模糊又叫低通滤波：它作用在高频信号上，把高频信号模糊掉，只让低频信号通过。高斯模糊或其他模糊常被称为低通滤波。

模糊是低通滤波，因为模糊会把每个像素和周围像素平均一下，快速跳变的部分被抹平了，只剩下缓慢变化的部分。所以说蓝噪声很适合做dithering。



如果做一个 3 像素高斯滤波（*高斯滤波是一种**用高斯函数作为权重的模糊滤波**，可以理解为：对每个像素，用周围像素的加权平均来替换它。离中心越近的像素权重越大，离得越远的权重越小。权重按高斯分布衰减。*），对于均匀噪声，仍会留下更低频的噪声底色；对于更高频的噪声，会得到接近均匀的灰度图，因为高频细节都被滤掉。这里也体现了可视计算的优势：想分析什么，就把它可视化，可视化结果通常很有说服力。

五、有序抖动与空间换颜色

加噪声是一种滤波思路，但早期受限于 GPU 上的噪声不容易获得、并行性不好等问题。若要把抖动做成硬件中的可执行程序，噪声可能不友好。解决手段包括预存一张噪声图片，但可能受图像大小影响。总之，加噪声有不足。

另一个研究方向是用更大的空间换取颜色通道上的不足。例如用 3×3 区域表达本来一个像素要表达的信息，从而获得更多灰度阶。全 0 表示完全黑色，全 1 表示完全点亮，这样 3×3 可以传递比 0/1 更多的信息。

这里最核心的是设计有序抖动（*有序抖动：按照一个固定的顺序，依次点亮不同数量的像素，以表示中间灰度。*）。0 和 9 比较明确，但中间要先点亮哪个像素、按什么顺序点亮，需要用空间组织。成熟结果中，一个重要性质是：先前点亮过的像素，后续必须持续保持点亮。同时希望相邻之间连续。因此会设计出 pattern：一开始点亮某个像素，之后逐次点亮邻居，直到 9 个全部点亮。

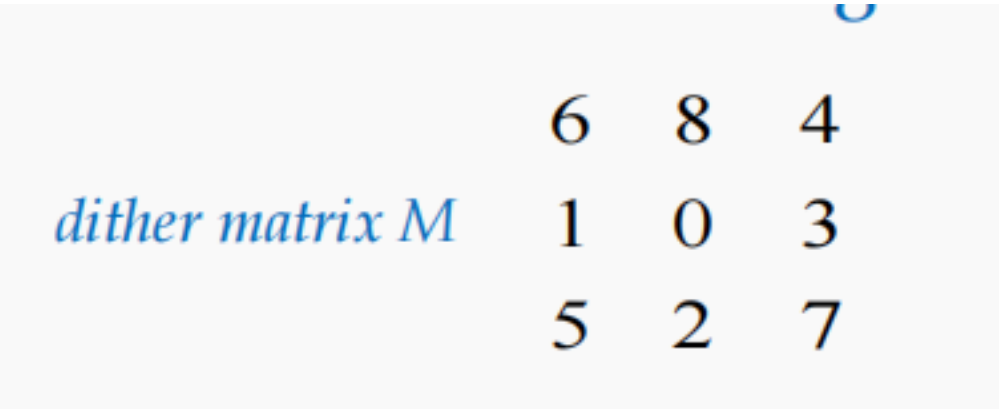
为什么一定要连续？如果随机点亮，从信息传递上似乎没有不同。但抖动不仅用于当前图像，也可用于随时间变化的图像。若输入是视频，可以对时间做同样抖动处理。在黑白设备上做动图展示时，抖动可以让中心较亮、周围以灰度逐渐过渡，虽然是二值图像，但能看到明暗渐变。

如果按顺序设计 pattern，当某个像素值从 3 变到 4、5，再回到 4、3，连续变化时，整个 pattern 也保持连续变化。在动图中，人不会注意到信息抖动，而会关注图像渐变本身。这是有序变化的一个理由。

在静态空间中，人也对不规律突变更敏感。若 1 显示一个点，2 突然显示对角两个点，相邻区域从 1 变到 2 时，不仅亮度变化，还有结构变化。拼在一起后，容易注意到本来不应有的结构错觉，形成图形学中的 artifact。连续设计则让人眼认为信号有结构性持续变化，更多识别渐变过程，而非结构化改变。

使用 3×3 空间换颜色的方法，很多时候是因为硬件有并行需求。查 pattern 表格可行，但算法上希望更优化。于是可以设计抖动矩阵，标注依次点亮像素的顺序。例如最首先点亮中间，然后左边，然后底下。只要表示矩阵，就记录了 pattern 变化轨迹，不必存储所有 pattern。

(pattern表格是指：为每个灰度级预先存一张黑白图案，用的时候直接查表。抖动矩阵是指：不存所有图案，只存一个"顺序矩阵"，用的时候通过比较动态生成图案。)



处理图像时，把待处理像素归一化到 0~9。若当前像素值为 4，则所有小于 4 的都应已点亮，大于等于 4 的尚未点亮。这样可以并行、快速地决定元素是否点亮。

抖动常存储抖动矩阵。但这增加了分辨率。如果不想让硬件组装更多屏幕，能否用同样分辨率、有序抖动、适合并行且不需要提前搞噪声？答案可以。

方法是将图像切成 3×3 格子。把每个 3×3 图像块与抖动矩阵比较。两边都归一化，例如抖动矩阵从 0~8 归一化到区间。若当前像素块的值大于抖动矩阵对应值，就置为点亮；小于则熄灭。这样把连续空间转换为固定 pattern，同时保持连续性。

这样不增加显示分辨率，也能做有序抖动。在图像上做有序抖动，即使不人为添加噪声，抖动本身有一定非规则性，人眼能忽视高频规则信息，效果较好。

对比白噪声抖动、蓝噪声抖动、规则有序抖动，有序抖动感受上介于前两者之间。蓝噪声早期若没有预存，生成较复杂。从信号处理角度，白噪声是全频谱噪声，蓝噪声是高频噪声、低频没有。规则有序 pattern 做频谱分析，会发现它存在于某几个特殊频率上，相对较高频，因此效果介于二者之间。

六、Floyd-Steinberg 抖动

图像抖动还有一种基于误差扩散的抖动算法 (dithering with Error diffusion)，叫做弗洛伊德-斯坦伯格抖动算法 (Floyd-Steinberg Dithering)，它的思想是将单个像素的量化误差传递到其他像素，从而保证总误差接近于 0。

弗洛伊德-斯坦伯格抖动算法的流程是：从左到右、从上到下依次处理每一个像素点，将当前像素点的亮度以0.5为阈值进行二值化，将二值化前的亮度减去二值化后的亮度作为误差值，然后将误差值分成占比分别为$7/16, 3/16, 5/16, 1/16$的四个部分，并分别加到右方、左下方、下方、右下方四个像素的亮度值，已处理好的像素点不受影响。如图7.24所示，黑点所在的方格为当前处理的像素点，灰色方格表示已处理的像素点，白色方格表示未处理的像素点。

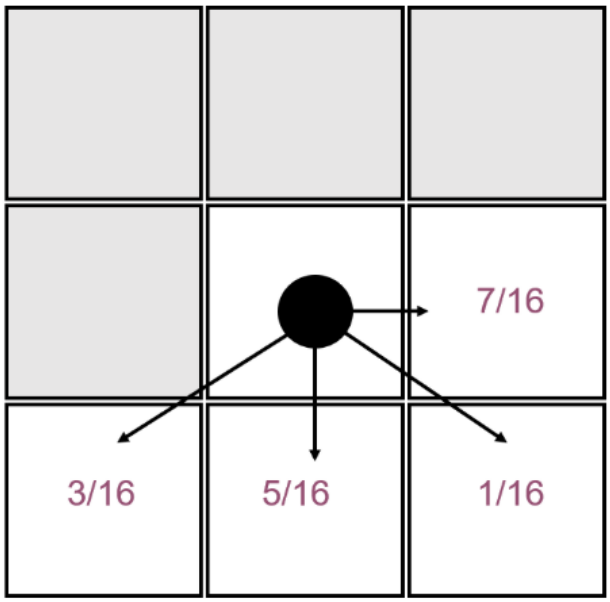


图 7.24 弗洛伊德-斯坦伯格抖动算法的误差扩散过程 #

七、图像滤波与卷积 *image filtering*

讲完抖动，它是在局部、和硬件联系紧密的优化。接着讲更常见的图像处理。在 CV 领域，特征提取很重要，与之相关的是 filter。

最简单的 filter 是模糊。模糊往往和频率相关：滤掉某些高频信息，只通过低频信息。模糊核决定哪些高频被滤掉、剩下哪些低频。图像分辨率不高时，例如 64×64，分别做 3×3 或 5×5 模糊，高频信息逐渐减少，剩下信息越来越偏低频。

模糊是常见的区域性图像处理手段。最简单的模糊是均值滤波。把 3×3 区域中以某像素为中心的邻居全部加权、相加、求平均，用平均值覆盖本来的像素颜色值，就完成局部均值模糊。

也可以写成矩阵计算形式。模糊核写成 3×3 矩阵，每个数代表对应像素点应乘以的权重。可以扩充到 $N\times N$，只需适度归一化，使图像不会过亮或过暗。

从数学上看，filter 定义了一种卷积操作。连续空间中的卷积:

$$(FG)(t)=\int F(\tau)G(t-\tau)d\tau.$$

注：连续空间中的卷积公式

$$(F*G)(t)=\int F(\tau)G(t-\tau)\ d\tau$$

- *F：输入信号，比如一段声音或一张连续图像。*
- *G：卷积核，比如高斯核。*
- *t：当前时刻或当前位置。*

- τ：*积分变量，遍历整个信号。*
- $G(t-\tau)$：*把核 G 翻转并平移 t。*

*这个公式的意思是：**输出在 t 处的值，等于输入 F 和翻转平移后的核 G 在整个区间上的乘积积分。***

图像是离散的，二维离散卷积：

$$(FG)(x,y)=\sum_{m,n}F(m,n)G(x-m,y-n).$$

连续定义可用无穷大空间和无穷大卷积核。实际使用中，常用有限维度卷积核，认为离当前像素太远的信息不重要，因此远处卷积核 $G=0$，只有 3×3 或 5×5 区域有值。手工设计卷积核时，常设计对称或反对称核，认为其具备一定空间性质。

这里我纠结了很久的一个点在于，卷积核为什么要先翻转。最后得到的结论是，数学定义有关。翻转后的东西叫“相关”(correlation)

卷积核 $G(t-\tau)$ 可看作把无穷维函数 $G(\tau)$ 镜像成 $G(-\tau)$，再向右平移 t，与图像对齐。二维可视化时，输入为 F，有限 3×3 核先原地镜像，然后随处理数据移动。卷积时对应项相乘，最终生成输出。输出每个点的值，等于卷积核与它对齐后对应位置数值两两相乘并叠加。

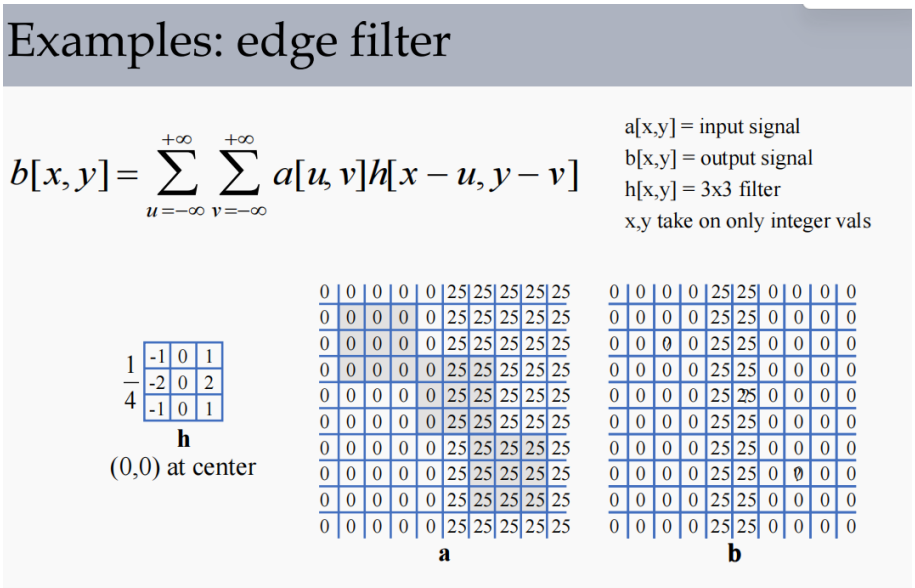
卷积运算可交换：用 A 卷积 H 和用 H 卷积图像 A，最终效果对称一致。

八、边缘提取、Sobel 与拉普拉斯

卷积可以提取信息，尤其高频信息。先看人为设计的卷积核。

假设输入图像左边全黑，右边全亮，例如 25。应用一个对称但取相反值的核，对每个像素，相当于右边减左边，并按同样比例相减。这其实是有限差分：总是用右边的数减左边的数。如果周围值完全一样，右边减左边没有东西；但在 0 和 25 的交界处，左右邻居出现跳变，右边减左边会准确捕捉跳变，并存储有限差分。到 25 这边后，因为完全一致，继续保持 0。

逐像素做卷积操作，非常适合并行。结果中，所有跟周围邻居发生突然跳变的信息点被保留，其他信息没有。这就是边缘提取：把颜色突变、两个不同区域之间的边提取出来。



水平方向有限差分展示竖直边；垂直方向有限差分展示水平边。采用水平方向和竖直方向的有限差分，可以分析图像中竖直和水平分割线。竖直和水平方向都重要，因此建立常用边缘提取卷积方式，逐渐固化形成各种算子，例如 Sobel 算子。

Sobel 算子思路：先用水平方向提取水平方向的边变化，再用竖直方向导数计算竖直方向的边信息。得到两张卷积图后，分别平方、求和、开根号：

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}.$$

1. Sobel 的两个卷积核

Sobel 算子有两个 3×3 核：

水平方向梯度核 G_x，用来检测**竖直边缘**：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

垂直方向梯度核 G_y，用来检测**水平边缘**：

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

在图像处理中，通常直接拿这两个核在图像上滑动，做“对应位置相乘再求和”。严格说这叫**相关**，但因为边缘检测只关心梯度大小，符号不重要，所以大家常直接叫它卷积。

---

2. 准备一个简单的图像块

我们取一个 3×3 的图像块，左边两列暗，右边一列亮，形成一个竖直边缘：

$$I = \begin{bmatrix} 10 & 10 & 50 \\ 10 & 10 & 50 \\ 10 & 10 & 50 \end{bmatrix}$$

我们计算**中心像素** $(1, 1)$ 处的 Sobel 响应。中心像素的值是 10。

5. 计算梯度模长

Sobel 算子的最终输出是梯度模长：

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}$$

代入：

$$|\nabla I| = \sqrt{160^2 + 0^2} = 160$$

结果很大，说明中心像素处存在很强的边缘。

6. 结果解读

- $G_x = 160$：水平方向变化剧烈，说明有**竖直边缘**。
- $G_y = 0$：垂直方向没有变化，说明没有水平边缘。
- 梯度模长 160：边缘强度。

这正好符合我们的图像：左边暗、右边亮，中间有一条竖直的分界线。

每个像素自身完成操作，最终得到边缘提取函数，保证横向和纵向边信息都被提取。

图像本身是连续信号源，只是以离散方式表达。很多运算符合这个思想：在离散表达上运行离散运算，例如离散卷积，但存在来自连续空间的含义。有限差分对应微分运算：求图像在水平方向的变化率，以及竖直方向的变化率。微分从连续空间离散到图像空间，就得到有限差分计算核。它会带来离散误差，但在离散空间中已经是对连续函数的最好近似。图像空间中将连续函数对应的离散处理方式称为图像离散算子。

Sobel 算子计算的是梯度模长：

$$|\nabla I| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}.$$

图像作为连续信号，在空间中变化。有的地方变化快，有的地方变化慢。梯度模长能告诉哪些地方变化快，哪些地方没有额外信息、比较不重要。因此 Sobel 卷积能提供边缘提取效果。

拉普拉斯算子在图形学中非常重要，后面还会多次接触。数学上，拉普拉斯算子计算二阶微分：

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}.$$

2. 二维拉普拉斯算子

图像是二维的，有 x 方向和 y 方向。
拉普拉斯算子的连续定义是：

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

也就是：x 方向二阶微分 + y 方向二阶微分。

分别离散化：

- x 方向：

$$f(x+1,y) + f(x-1,y) - 2f(x,y)$$

- y 方向：

$$f(x,y+1) + f(x,y-1) - 2f(x,y)$$

两者相加：

$$\begin{aligned}\nabla^2 f \approx & f(x+1,y) + f(x-1,y) - 2f(x,y) \\ & + f(x,y+1) + f(x,y-1) - 2f(x,y)\end{aligned}$$

合并同类项：

$$\nabla^2 f \approx f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$$

这就是离散拉普拉斯算子。

离散化时，先用有限差分表达一阶微分，再用一阶微分结果的有限差分表示二阶微分。拉普拉斯是二阶微分的和，因此把对角线相加，得到拉普拉斯。写成卷积矩阵：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

它非常对称，中心为 -4，周围为 $+1$。用拉普拉斯算子作为二阶微分，体现的是当前数据与周围数据的差异。若都一致，则没有信息；若明显比周围点大或小，则是局部极值点。这也符合二阶微分理解。拉普拉斯算子也可用于边缘提取。

九、频率的直观理解

有同学问频率该怎么理解。对二维图像做傅里叶分解，可以理解为：手上有一张 256×256 的图片，用一堆棋盘格叠加起来拟合它。棋盘格有不同频率：1×1 纯底色，2×2 黑白，4×4，6×6，甚至 256×256。到 256×256 时更接近高频、更不规律。每个棋盘格乘以强度，并可能移相位。所有棋盘格叠加，最终累计获得能近似原图的图片。因此，图像这个连续信号在空间中的频率变化，被拆解到各个频率维度上。

十、图像补全与拉普拉斯编辑

图像补全是相对全局的任务。数学上，假设有一张图片，中间被挖掉一块，或有一些不想要的图案被抠掉。我们希望填补这块内容。需求包括：边界从容过渡，边界信息和已知边界条件完全一致；内部填补时，若只是简单抠掉，一般希望不填补任何信息，像平静水面，中间什么信息都没有。

已经学过边缘提取算子和微分算子。如果某区域有微分，人眼会看到差别和内容。因此补全的核心思想是：填完后，该区域在微分上没有任何信息。理想状况是边界契合，整个区域连续变化。用微分算子跑边缘，希望微分意义上没有任何信息，即非常连续的变化，这样图像被完整补好，且没有提供不应提供的信息。

图像补全来源于实际生活。现在手机拍照片，选取路人，让手机补全，通常用 AI 算法，效果往往优于传统算法。但传统算法完全从数学出发，有助于理解这件事。

补全涉及的概念：第一，忠诚于周围邻居，也就是边界必须保持一致，没有更改。其他图像处理中，不想修改的部分也应尽可能保持本来信息。第二，遵循剃刀原则：如果只是补全，没有特殊要补的东西，那么所有东西都不应被增加，增加的信息量尽可能少。

1. 符号含义

$$\min_F \iint_\Omega \|\nabla F\|^2 \, dA, \quad F|_{\partial\Omega} = F^*|_{\partial\Omega}.$$

- F：待求的补全图像。它在区域 Ω 内每个像素的值是未知的。
- Ω：待补全的区域，也就是被挖掉的那块洞。
- $\partial\Omega$：区域 Ω 的边界。
- F^*：已知的原图。边界上的值是已知的。
- ∇F：F 的梯度，表示图像在 x 方向和 y 方向的变化率：

$$\nabla F = \left(\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y} \right)$$

- $\|\nabla F\|^2$：梯度模长的平方：

$$\|\nabla F\|^2 = \left(\frac{\partial F}{\partial x} \right)^2 + \left(\frac{\partial F}{\partial y} \right)^2$$

  它衡量图像在该点变化有多剧烈。
- $\iint_\Omega \|\nabla F\|^2 \, dA$：把整个区域 Ω 内所有点的梯度平方加起来，得到总能量。
- $\min_F$：寻找一个函数 F，使这个总能量最小。
- $F|_{\partial\Omega} = F^*|_{\partial\Omega}$：边界条件。在区域边界上，补全结果必须和原图边界完全一样，不能有跳变。

极端情况下，边界全是常数 1，则区域用纯色 1 填充，完成最小化。但边界往往是任意曲线，上面的值任意。因此需要计算手段，从数学上解决能量最小化问题。这个问题后来常称为拉普拉斯图像编辑。

若直接思考怎么做，因为它是能量最小化问题，可以先随机生成所有点的像素值，获得一张图像。在图像上计算梯度，计算边界与这些边的误差。把所有能量作为总能量，做梯度下降优化。每个像素调整自身值，使它与周围梯度的信息逐渐减少，与边界上点的信息逐渐接近。优化很多步后，可以得到图像补全结果。这是基于梯度下降优化的算法，可能不高效，但能完成能量最小化任务。图像补全发展较久，数学上可以较好求解，因此后面有更高效算法。

十一、图像克隆、融合与泊松编辑

图像补全更进一步是图像克隆、融合或图像融入。需求可能是：已有背景，另一张图有更感兴趣的前景，想把前景挖出来放到背景中。如果直接把两张图通过不透明度简单叠加，会出现不融合效果。

图像融合想做的事情与补全类似。希望图像所有边界信息与背景图本来边界信息保持一致，而内部所有信息与前景信息保持一致。内部保持一致又涉及微分、梯度。计算每个像素梯度，希望梯度尽可能与前景图像梯度一致。然后做能量最小化：

$$\min_F \iint_\Omega \|\nabla F - \nabla G\|^2 \mathrm{dA}, \quad F|_{\partial\Omega}=F|_{\partial\Omega}.$$

从这个例子看，图像补全，即 Laplacian editing，是 Poisson editing 的特例：要补的图片恰好是均值图片，梯度完全等于 0。

能量最小化问题怎么求解？这部分内容不考，只需记住结论：

https://vcl-pku.github.io/vci-book/getting-started/images/poisson.html#id4

函数的最小化问题，也就是泛函优化问题，需要借助欧拉-拉格朗日方程[1]。这里我们直接给出结论，公式 (7.12) 中优化目标取极值当且仅当 f 满足如下狄利克雷边界条件的泊松方程：

$$\nabla^2 f = \nabla^2 g, \quad \text{s.t.} \quad f|_{\partial\Omega} = f^*|_{\partial\Omega} \tag{7.13}$$

其中 $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ 为拉普拉斯算子。这也是泊松图像编辑名字的由来。我们需要求解的是颜色场 f，在离散场景下，我们需要求的是 Ω 中每个像素点上 $f(x,y)$ 的值。我们可以使用上一节公式 (7.10) 中介绍的离散 Laplacian 算子代入公式 (7.13) 中，右手项可以直接计算出结果，左手项展开为：

$$\nabla^2 f(x,y) = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y) \tag{7.14}$$

对于大多数 Ω 内的像素点，公式 (7.14) 中的五个 f 值都是待求的未知数。但是对于在边界上的像素点 (和 $\partial\Omega$ 相邻的像素点)，其四个相邻的像素点不全在 Ω 中，此时我们就需要借助公式 (7.13) 中的边界条件，把公式 (7.14) 中相应的邻居项换成 f^* 的取值。举个例子，如图 7.13 所示，计算 $(2,2)$ 处的拉普拉斯时，其左、上邻居为 $\partial\Omega$ 内的像素点，所以这两项需要替换成 f^* 的值，即：

$$\nabla^2 f(2,2) = f(3,2) + f^*(1,2) + f(2,3) + f^*(2,1) - 4f(2,2) \tag{7.15}$$

$f^*(1,1)$	$f^*(1,2)$	$f^*(1,3)$	$f^*(1,4)$
$f^*(2,1)$	$f(2,2)$	$f(2,3)$	$f^*(2,4)$
$f^*(3,1)$	$f(3,2)$	$f^*(3,3)$	$f^*(3,4)$
$f^*(4,1)$	$f^*(4,2)$	$f^*(4,3)$	$f^*(4,4)$

图 7.13 待填补的像素示例。其中白色为 Ω，蓝色为 $\partial\Omega$，橙色为剩余的图片已知部分。

于是这个像素的 $\nabla^2 f$ 中只包含了三个未知数。对于每个 Ω 中的像素 (x,y)，我们都可以写出一个方程

十二、抠图

https://vcl-pku.github.io/vci-book/getting-started/images/segmentation.html

问题描述

令 C 表示输入的图像，F 表示前景图，B 表示背景图，这些图都是 RGB 图像，α 表示透明度（或前景图的占比），则它们之间的关系如下式：

$$\mathbf{C}=\alpha\mathbf{F}+(1-\alpha)\mathbf{B}$$

对于某一个像素点 i，就是要解如下方程：

$$\mathbf{C}_i^R=\alpha_i\mathbf{F}_i^R+(1-\alpha_i)\mathbf{B}_i^R$$

$$\mathbf{C}_i^G=\alpha_i\mathbf{F}_i^G+(1-\alpha_i)\mathbf{B}_i^G$$

$$\mathbf{C}_i^B=\alpha_i\mathbf{F}_i^B+(1-\alpha_i)\mathbf{B}_i^B$$

其中 $\alpha_i, F_i^R, F_i^G, F_i^B, B_i^R, B_i^G, B_i^B$ 为未知量，C_i^R, C_i^G, C_i^B 为已知量。这是一个欠定的线性方程，有无穷多组解。

对于欠定方程，我们有两种求解的思路，一个是减少未知量个数，一个是增加方程的数量。这两种方法分别对应于**蓝（绿）幕抠图（blue (green) screen matting）** 和**三角抠图（triangulation matting）**，它们都是在特定拍摄环境下使用的算法，简易但是有较大的局限性。

LESSON 3 2026-09-21 第1-2节

一、课程导入与内容安排

从今天开始，我们进入画图的部分。这里先说明两个方向的区别：

- **图像处理**：一般是针对一张已有的图像，从中提取连续语义上的内容。比如从离散的像素中识别出边、角等特征，本质是从离散信号恢复连续的语义，是计算机视觉入门的基础。
- **图形学**：核心思想是你在脑中已经定义好了一条曲线、直线或形状，希望在离散的屏幕空间中将它呈现出来，本质是把连续的几何对象在离散空间中进行表示。我们今天讲的画图（Drawing），也叫2D图形学，就是图形学的入门内容。

二、光栅化绘制的背景与核心需求

2.1 显示硬件的演进：从矢量到光栅

画图最基础的任务就是画一条直线。这个问题看起来简单，但我们会花较长的篇幅讲解，也是大家习题中需要实现的内容。

之前讲显示原理时提到过，最早的显示器是阴极电子束成像的矢量显示器。对于矢量显示器来说，硬件指令只需要指定从某一个点划到另一个点，硬件会控制电子束的运动，激活屏幕上对应区域的荧光粉，自然形成一条直线。在那个年代，画线是由硬件自动完成的。

后来出现了光栅化显示器，显示控制方式变成逐点扫描每一个像素位置。光栅显示器的分辨率是固定的，因为它必须以固定的频率刷新屏幕才能保证帧率（FPS）。由此，硬件层面第一次出现了需求：如何把两个点之间连续的直线，用离散的像素点组合呈现出来。

2.2 帧缓冲区与光栅化流程

随着硬件发展，显示器有了专属的存储区域——**帧缓冲区（Frame Buffer）**。有了帧缓冲区之后，每一次刷新屏幕，本质上都是在帧缓冲区上完成各种绘图操作。后来图像存储也沿用了类似的思路，位图文件存储的就是固定分辨率的图像数据。

这就是光栅化从硬件到软件再到存储的完整流程。

2.3 画线操作的普遍性与性能要求

画线操作在今天的显示器中依然占有极大的应用比例。大家可以想象，无论是1920×1080还是4K分辨率的显示器，每个像素的绘制都依赖画线操作。

Windows最著名的创造之一就是多窗口UI交互。屏幕上同时运行着PowerPoint、Word、画图等不同的应用程序，每个程序都要在自己的窗口区域内完成绘制。对于帧缓冲区来说，这是一个逐层覆盖的过程：先绘制桌面的所有像素，再绘制PowerPoint窗口，最后绘制最上层的Word窗口和其中的线条。

可以说，画图操作成千上万次地出现在屏幕显示的全过程中。这就要求画线算法必须在非常高效的环境中完成。效率是图形学领域的普遍要求，因此我们这节课的任务不只是讲解如何画直线，更重要的是讲解如何高效地完成直线绘制。

2.4 扫描绘制的核心目标

画图也常被称为**扫描（Scan）**，因为绘制过程和屏幕扫描线的硬件需求高度对应。扫描绘制就是专门针对屏幕扫描做优化的画图方式。

对于画线算法，我们有两个核心需求：

1. **速度越快越好**：这是最核心的性能要求。
2. **尽可能准确**：因为离散化必然存在误差，我们的目标是在当前分辨率下，尽可能准确地还原连续的直线信号。

三、直线绘制的基础算法

3.1 暴力算法及其缺陷

最朴素的思路是暴力求解：遍历屏幕上的每一个像素，检查它是否在直线上。比如1920×1080的屏幕，就检查所有两百万个像素。

这种方法一定能得到结果，但资源消耗和时间开销都非常高。我们不希望每次刷新屏幕都遍历所有像素，因此画线算法的优化方向很明确：只修改需要改变的像素，只计算必要的像素位置，执行最直接的运算。

3.2 朴素迭代算法

我们从实际的画线场景出发，先做一些基础设定。图形学中一般需求是画线段，我们可以给自变量一个范围。为了方便，我们先把坐标归一化到0~1区间，0对应左下角像素，1对应右上角像素，实际实现时再根据屏幕分辨率映射到具体的像素坐标。

直线的方程是连续形式的 $Y = \mathrm{MX} + B$。最直观的像素化操作是：沿着自变量X的方向逐个遍历，对每个X计算对应的Y值，将Y四舍五入到最近的像素位置，然后点亮该像素。

这里就产生了第一个问题：应该顺着X轴遍历，还是顺着Y轴遍历?

- 如果直线方程写成 $Y = \mathrm{MX} + B$，顺着X遍历会更简单。
- 也可以把方程变形为 $X = f(Y)$，顺着Y遍历。

两种方式绘制的像素数量是不同的。因此我们有第一个优化原则：**选择增长更快的方向作为迭代的主循环方向。**

- 当斜率绝对值小于45°（$|M| < 1$）时，X方向增长更快，沿X轴遍历，保证不会遗漏任何像素。
- 当斜率绝对值大于45°（$|M| > 1$）时，Y方向增长更快，沿Y轴遍历。

图形学的接口都是按这个逻辑设计的，画线的输入规范就是起点坐标 (X_0, Y_0) 和终点坐标 (X_1, Y_1)。当然，连续坐标经过四舍五入映射到整数像素坐标，会损失一部分精度，但在高分辨率下这个误差可以接受，而固定的API格式能让所有应用更直接高效地调用。

3.3 朴素算法的性能分析

朴素迭代算法已经比暴力算法优化了很多——它只触碰真正需要绘制的像素。但如果精益求精，就会发现它依然很慢，问题出在计算环节：

- 每次迭代都需要做一次浮点乘法 $M \times X$，再加上浮点加法。
- 最后还要做一次四舍五入运算。

其中浮点乘法是开销最高的操作。后续的优化方向，就是逐步把这些昂贵的运算替换掉，甚至最终完全消除浮点运算。画线算法是图形学中被优化到极致的经典案例。

四、DDA数字微分分析仪算法

4.1 优化思路：乘法转加法迭代

图形学中非常经典的一种优化手段：如果循环内部存在和迭代变量相关的乘法，就可以利用乘法是加法的累计这一性质，把乘法转换成加法迭代。这也是我们第一步优化的核心思想——把浮点乘法优化掉。

这种优化通常需要额外存储一些历史信息。我们的思路是：既然已经算出了当前X对应的Y值，那么下一个X+1对应的Y值，不需要重新用乘法计算，只需要在之前的结果上累加一个增量即可。

我们来推导一下:
已知当前点的Y值为 $Y_x = M \cdot X + B$
那么下一个点的Y值为:

$$\begin{aligned} Y_{x+1} &= M \cdot (X + 1) + B \\ &= M \cdot X + B + M \\ &= Y_x + M \end{aligned}$$

这样就消除了循环内的乘法运算，每次迭代只需要做一次浮点加法。

4.2 算法步骤

DDA（Differential Digital Analyzer，数字微分分析仪） 算法的步骤如下：

1. 输入起点 (X_0, Y_0) 和终点 (X_1, Y_1)，计算斜率 $M = \frac{Y_1 - Y_0}{X_1 - X_0}$。
2. 以X为主循环方向（$|M| < 1$时），从X0开始遍历到X1。
3. 初始Y值为Y0，点亮对应像素。
4. 每次迭代中，Y累加斜率M，四舍五入后点亮对应像素。

DDA算法已经避免了循环内的乘法， 整体性能提升很多。 这个算法的名字来源于它的核心思想：通过微分步进的方式，把乘法转换为加法。

4.3 仍然存在的问题

DDA算法依然存在浮点运算， 每一步都需要做浮点加法和四舍五入。 在早期CPU算力稀缺的年代，浮点运算单元非常少，整数运算单元更多，因此人们希望进一步优化，完全消除浮点运算。

五、布雷森汉姆（Bresenham）直线算法

5.1 核心观察：像素步进的二选一

我们先观察直线光栅化的结果： 当斜率在0到1之间时， 每向右走一个像素（X+1）， Y方向只有两种选择——要么保持不变，要么向上走一格（Y+1）。

也就是说，每次迭代X必然加1，Y要么加0要么加1。这本质上是一个累积的判断过程，和四舍五入的逻辑一致。这意味着我们有可能只用整数运算就完成整个画线过程。

接下来我们一步步推导，如何从DDA算法进化到全整数的布雷森汉姆算法。

5.2 中点判定法与隐函数思想

我们的核心问题转化为：每次X加1后，直线上的真实点，离 $(X+1, Y)$ 更近，还是离 $(X+1, Y+1)$ 更近？离哪个像素更近，就点亮哪个像素。

这个距离判断可以转化为**中点判定**：取两个候选像素的中点 $(X+1, Y+0.5)$，判断这个中点在直线的上方还是下方。

- 如果中点在直线下方，说明直线更靠近上方的像素 $(X+1, Y+1)$，应该点亮它。
- 如果中点在直线上方，说明直线更靠近下方的像素 $(X+1, Y)$，应该点亮它。

这个判定可以用**隐函数**来实现。直线可以写成隐式方程 $F(x, y) = 0$ 的形式。对于直线上的点，函数值等于0；对于直线上方的点，函数值符号为负；对于直线下方的点，函数值符号为正。我们只需要判断中点代入隐函数后的正负，就能决定点亮哪个像素。

> 隐式表达：不写成 $Y = F(X)$ 的显式形式，而是写成 $F(X, Y) = 0$ 的形式。*隐函数的表达能力强于显函数*，比如圆等几何图形更容易用隐函数表示。

5.3 构造整数隐函数

我们希望隐函数的计算尽可能避免浮点运算，因此选择用向量点乘的方式构造隐函数。

已知直线的起点 $P_0(X_0, Y_0)$ 是整数坐标，直线的切向量为$\vec{T} = (D\,X, D\,Y)$，其中 $\mathrm{DX} = X_1 - X_0$，$\mathrm{DY} = Y_1 - Y_0$，都是整数。

直线的法向量可以通过切向量旋转90°得到，即$\vec{N} = (D\,Y, -D\,X)$。这个法向量不需要归一化，因为我们只关心正负号，缩放法向量不会改变正负判定的结果，并且它的分量全是整数。

对于任意一点P，我们构造判别函数：

$$F(P) = F(P) = \overrightarrow{PP_0} \cdot \vec{N}$$

这个函数的正负对应点P在直线的哪一侧：

- $F(P) = 0$：点在直线上
- $F(P) > 0$：点在直线下方
- $F(P) < 0$：点在直线上方

> 课间修正：之前PPT上的正负对应关系有误，这里更正：当点乘结果大于0时，P在直线下方，此时应该点亮上方的像素；当点乘结果小于0时，P在直线上方，应该点亮下方的像素。

5.4 判别式的迭代更新

和DDA的思路一样，我们不希望每次都重新计算点乘，而是用迭代的方式更新判别式。

假设我们已经算出了当前点P的判别值F(P)，现在要计算下一个待判定的中点P+Δ的判别值。根据点乘的性质：

$$F\,(P + \Delta) = F(P) + \vec{\Delta} \cdot \vec{N}$$

也就是说，每次移动后，判别值只需要加上一个固定的增量。这个增量可以提前预计算，循环内只需要做加法。

现在我们的判定点是中点 $(X + 1, Y + 0.5)$，其中的0.5会引入浮点运算。我们可以利用法向量可以任意缩放的性质，把法向量乘以2，这样中点的0.5就会被抵消，整个计算就全部变成整数运算。

我们使用$2\vec{N} = (2\mathrm{DY}, -2\mathrm{DX})$作为新的法向量，此时判别式的增量全部为整数。

5.5 算法推导与迭代过程

我们从起点开始一步步推导迭代过程：

1. 起点 (X_0, Y_0) 在直线上，因此初始判别值 $F_0 = 0$。
2. 第一个待判定的中点是 $(X_0 + 1, Y_0 + 0.5)$，代入得：

   $$F_{\text{test}} = F_0 + 2\mathrm{DY} \cdot 1 + (-2\mathrm{DX}) \cdot 0.5 = 2\mathrm{DY} - \mathrm{DX}$$

   这就是初始的测试判别值，是一个整数。
3. 分支判断：
   - 如果 $F_{\text{test}} < 0$：中点在直线上方，直线更靠下，点亮右侧像素 $(X + 1, Y)$。
     下一个测试点向右平移一格，判别值更新为：$F_{\text{test}} \mathrel{+}= 2\mathrm{DY}$
   - 如果 $F_{\text{test}} \geq 0$：中点在直线下方，直线更靠上，点亮右上像素 $(X + 1, Y + 1)$。
     下一个测试点向右上平移一格，判别值更新为：$F_{\text{test}} \mathrel{+}= 2\mathrm{DY} - 2\mathrm{DX}$

4. 循环执行，直到X到达终点。

5.6 算法伪代码

布雷森汉姆直线算法（$0 \leq M \leq 1$ 情况）的伪代码如下：

```
输入：起点(X0, Y0)，终点(X1, Y1)
DX = X1 - X0
DY = Y1 - Y0
F = 2 * DY - DX  // 初始判别值
Y = Y0

for X from X0 to X1:
    点亮像素(X, Y)
    if F >= 0:
        Y = Y + 1
        F = F + 2 * DY - 2 * DX
    else:
        F = F + 2 * DY
```

整个循环内只有整数加法和比较操作，没有任何浮点运算，也没有乘法，性能达到了极致。

5.7 算法意义

布雷森汉姆算法是图形学非常经典的优化案例。它利用隐式方程和迭代思想，把一个看似必须用浮点运算完成的连续直线绘制任务，完全转化为整数运算。这个算法的思路对后续很多图形学算法都有深远影响。

5.5 算法推导与迭代过程（详细讲解）

我们先明确**大前提**：以下推导针对**斜率 0 ≤ m ≤ 1** 的直线。此时 X 方向是主步进方向（每一步 X 一定 + 1），Y 方向是次步进方向（每一步 Y 要么 + 0，要么 + 1）。

一、核心思想：中点判定法

我们的目标是：每向右走一格（X+1），判断接下来该点亮**右侧像素** $(X+1, Y)$，还是**右上像素** $(X+1, Y+1)$。

判断的标准是：**直线上的真实点，离哪个像素中心更近。**
为了简化距离计算，我们引入**中点判定规则**：
取两个候选像素的中点 $M\,(X+1, Y+0.5)$，观察这个中点在直线的哪一侧：

- 如果中点在**直线上方** → 说明直线位置更靠下 → 离下方的右侧像素更近，点亮 $(X+1, Y)$
- 如果中点在**直线下方** → 说明直线位置更靠上 → 离上方的右上像素更近，点亮 $(X+1, Y+1)$

六、直线算法的扩展

6.1 全空间直线的八分区处理

上面的推导只针对斜率在0到1之间的直线。对于其他斜率的直线，我们可以利用对称性处理：

- 斜率大于1时，交换X和Y，沿Y轴方向迭代。

- 斜率为负时，对Y坐标取反，处理完后再映射回来。

通过把平面分成八个卦限，每个卦限都用对称的方式套用上述算法，就可以解决全空间的直线绘制问题，核心逻辑完全一致，只是初始状态和坐标方向略有不同。

6.2 扩展到画圆

布雷森汉姆的思路不仅可以画直线，还可以扩展到画曲线，比如圆。

圆同样可以写成隐式方程 $F(x,y)=x^2+y^2-R^2=0$，同样可以用判别式的正负判断点在圆内还是圆外。和直线类似，画圆时每一步也只有有限的几个候选像素，通过判别式迭代就可以完成绘制。

需要注意的是，圆的斜率从0到360°连续变化，因此我们通常利用八对称性，只绘制$1/8$个圆弧，剩下的部分通过对称变换得到。可以开8条线程并行绘制这8段圆弧，进一步提升效率。

完整的布雷森汉姆画圆算法超出了本节课的主要范围，但它是非常有趣的思维扩展，感兴趣的同学可以在维基百科上查阅相关内容。

6.3 多边形扫描线算法

除了画线，图形学中另一个重要任务是绘制多边形。扫描线算法就是面向多边形填充的优化算法。

扫描线算法的核心思路是：用水平扫描线从上到下遍历多边形，每一条扫描线只需要确定左右两个边界点，然后直接填充两点之间的所有像素。

左右边界点的更新，本质上就是两个布雷森汉姆画线过程。我们不需要每次都重新计算边界位置，只需要用迭代的方式更新XL和XR，全程都是整数运算，效率非常高。

6.4 凹多边形与三角化

扫描线算法默认处理凸多边形。如果是凹多边形，扫描线会和多条边相交，此时需要用**奇偶判定法则**：扫描线与边的交点按顺序排列，第奇数个交点到第偶数个交点之间的区域填充，偶数到奇数之间的区域不填充。

但在现代图形学中，已经很少单独处理凸多边形或凹多边形了。更通用的做法是把所有多边形都做**三角化**处理——任何多边形都可以拆分成若干个三角形。

三角化的优势在于：

1. 三角形一定是凸多边形，算法稳定，不需要额外的凹凸判断。
2. 三角形是现代图形学的基本图元，无论是算法层面还是硬件层面都被深度优化。

现在的显卡都自带硬件三角形绘制加速，你只需要传入三个顶点坐标，硬件就会自动完成整个三角形的光栅化填充，效率远高于软件算法。等到后面讲解渲染、GPU和帧缓冲区的时候，我们还会详细讲硬件并行实现的原理。

> 课间问答
> 问：现在浮点运算单元很多，还有并行计算能力，为什么还要优化画线算法？
> 答：算力永远是稀缺的。并行能力通常用在更粗粒度的任务上，比如同时画100条直线，而不是一条直线上的100个点。每条直线自身依然会用最精简的算法，把并行资源留给更适合并行的操作。即使在今天，单条直线的极致优化依然有意义。

七、颜色插值与图像变形

7.1 颜色线性插值

上面讲的都是几何坐标的光栅化，实际绘制时我们还需要处理颜色。屏幕是RGB颜色空间，绝大多数图形都不是纯黑的，而是带有颜色的，甚至同一个三角形的三个顶点可以有不同的颜色。

要实现顶点颜色的平滑过渡，只需要做颜色插值。最基础的是**线性插值**：已知两个端点的颜色值，中间任意一点的颜色按距离比例混合。

对于RGB三个颜色通道，每个通道独立做线性插值即可，这是非常简单的操作。

7.2 双线性插值

如果是四边形区域，四个顶点都有颜色值，要计算区域内任意一点的颜色，常用的方法是**双线性插值**。

- 第一步：先对上下两条边分别做线性插值，得到两条边上对应点的颜色。
- 第二步：对这两个点再做一次纵向的线性插值，得到最终点的颜色。

简单来说，就是通过点插值出直线，再通过直线插值出平面，这就是双线性插值的过程。

7.3 图像变形（Image Warping）

我们在界面中经常会遇到图像变换的场景，比如拖动窗口、缩放、旋转，还有一些非线性的窗口动效。这些操作本质上都是**图像变形**。

图像变形和插值的原理是相通的，只不过插值的对象从颜色空间变成了坐标空间：

- 图像本身有自己的**UV坐标空间**，通常归一化到$(0,0)$到$(1,1)$之间，这是图像的原始规范空间。
- 我们要把图像绘制到屏幕的XY坐标空间，四个顶点对应到屏幕上的四个位置。
- 对于屏幕上的任意一个像素，我们通过坐标插值反算出它对应的UV坐标，再从原图中取出对应位置的颜色，绘制到屏幕上。

整个过程的核心是坐标空间的插值，颜色只是坐标索引的结果。这也是为什么我们把图像变形放在图形学的画图部分讲解，而不是图像处理部分——它本质上是几何坐标操作，而非像素值操作。

7.4 交互式形变的实现

图像变形在图像处理软件中有非常广泛的应用，比如Photoshop中的变形工具。

为了方便用户交互，通常的做法是在图像上覆盖一层网格，用户可以拖动网格上的控制点来改变图像形状。为了让操作更自然，还发明了笔刷式的交互：笔刷中心的位移权重最高，周围按半径逐渐衰减，用户拖动笔刷就能让一大片区域跟着变形，实现非线性的形变效果。

7.5 四边形插值的两种方式

这里补充一个重要的观察：四边形的线性插值并不是唯一的，有两种典型的实现方式，结果不同：

1. **双线性插值**：先沿水平方向插值，再沿垂直方向插值。当四边形是矩形时，过渡非常均匀；但当四边形被拉伸变形时，对角线方向会出现扭曲。
2. **三角剖分插值**：先把四边形沿对角线拆成两个三角形，在每个三角形内部做线性插值。

两种方式没有绝对的对错，适用场景不同：

- 如果是三维物体的透视投影变形，双线性插值的结果可能更符合真实透视。
- 如果是规则的网格变形，三角剖分的结果可能更符合预期。

在图形学中，最终的渲染结果和输入的表达形式高度相关。如果输入是三角形网格，就会走三角形插值的管线；如果输入是四边形网格，就会走双线性插值的管线。具体选择哪种，要根据实际应用需求决定。

八、曲线内容展望

8.1 曲线的表达需求

讲完直线，我们自然会扩展到曲线。曲线在图形学、艺术、设计和工业领域都有非常广泛的应用，有强烈的表达需求。

艺术领域，梵高的画作中就用大量旋转、湍流的曲线来表现云朵、植物等元素，传递出独特的生命力。西班牙建筑师安东尼·高迪有一句名言："直线属于人类，曲线属于上帝。"他的作品，比如米拉之家、圣家族大教堂，大量使用贴近自然的曲线构造，极力避免火柴盒式的现代建筑。

无论是还原真实世界，还是艺术表达，我们都需要描绘曲线。图形学已经很好地解决了曲线的表示和光栅化问题。

8.2 矢量图与曲线

图像分为矢量图和光栅图两大类。在矢量图格式中，曲线是最核心的表达内容，最典型的就是SVG格式。Logo设计、字体设计最终通常都会用SVG格式存储，因为矢量图可以任意缩放而不失真。

曲线最终呈现在屏幕上，都需要经过光栅化。和直线一样，我们可以用隐函数结合布雷森汉姆的思路把曲线光栅化。但曲线的非线性更强，低分辨率下信息损失更多，因此用矢量格式保存曲线的原始定义，在需要时再光栅化，是更合理的方式。

8.3 工业领域的曲线应用：CAD/CAE/CAM

曲线不仅用于艺术和设计，还广泛应用在工业领域，也就是CAx（计算机辅助技术）领域，主要包含三个方向：

1. **CAD（计算机辅助设计）**：核心需求是设计出优雅、流畅的曲线曲面。比如汽车的流线型外形，既要美观也要符合空气动力学。CAD中的几何表达都是连续的曲线曲面。
2. **CAE（计算机辅助工程）**：核心需求是物理仿真，比如结构力学、温度扩散、流体流动等。这些问题都需要求解复杂的偏微分方程（比如流体的纳维- 斯托克斯方程、热扩散方程）。连续的几何体无法直接求解，必须离散化成网格（Mesh），比如三角形网格。离散化的精度、网格的疏密分布，直接决定了仿真的误差和效率，这是CAE领域非常重要的研究方向。
3. **CAM（计算机辅助制造）**：核心需求是把设计好的零件制造出来。无论是切割、模具还是3D打印，刀具的运动轨迹往往是线性的，因此CAM是一个混合了连续与离散的领域——既要忠实还原CAD的连续曲面设计，又要用线性的刀具路径去逼近它。

感兴趣的同学可以去GrabCAD网站看看，上面有非常多的3D CAD设计模型，广泛应用在SolidWorks、AutoCAD等经典设计软件中。

8.4 后续课程预告

2D图形学的曲线有很多种表达，比如圆、隐式方程、折线，其中我们重点讲解的是贝塞尔曲线。关于曲线的详细内容，我们留到明天下午的课继续讲解。

今天的课就到这里，谢谢大家。

> 课后说明：平时成绩占5分，评分主要看大家作业的实现效果。除非有大问题，一般不会扣分。

LESSON 4 2026-09-22 第5-6节

课程导入

上节课我们讲解了直线扫描算法，以及2D图像图形学的基础，同时初步介绍了曲线的方向，包括曲线的广泛应用背景，以及在CAD、CAE、CAM中的具体需求与区别。

今天我们从曲线开始，讲解与图形学更相关的2D曲线基础内容。上节课我们已经提到，曲线分为很多类别，包括显式曲线、隐式曲线、分形，以及我们会重点讲解的贝塞尔曲线。接下来我们逐一讲解这些曲线的内容。

显式曲线

我们先从显式曲线讲起。最简单的显式曲线，就是把很多直线段连接在一起，构成一条非直线的线，这就已经是一条曲线了。显式表示通常比较容易编辑和操作。

从定义上来说，显式曲线一般写成 $y = f(x)$ 的形式。它既可以是分段的形式，每一段都是 $y = m\,x + b$ 的直线表达式，也可以是更复杂的非线性函数，比如 $y = x^2$。

在图形学中，尤其是在表达几何的相关方向，显式和隐式是非常常用的概念，它们界定了方法本质上的特性。提到显式，很多人会想到离散、不连续，或者点云这类存储格式，但这些并不是显式最本质的区别，只是它常表现出的优缺点。

更本质地说，显式是直接定义为 $y = f(x)$ 形式的表达方式，可以用于曲线表达，也可以用于空间几何体表达。这种表达最大的特点是，可以直接代入公式计算，得到想要的y值。比如想知道点云在空间中的位置，就可以直接找到点云坐标进行插值，也就是可以直接显式地计算。

显式的优势很明显：需要取值的时候，代入公式就能快速求出结果。但它的不足也非常突出：要把复杂形状表示成显式形式往往并不容易。比如圆的显式表达，相对来说就会复杂一些。

隐式曲线

与之相对的隐式曲线，应用会更普遍一些，因为它的表达能力更强。我们可以把它抽象为 $f(x, y) = 0$ 的函数形式，包含的范围很广，比如常见的解析方程。以圆为例，用隐式形式可以非常自然地表达出来。

很多人提到隐式时，会认为它是连续的，不是离散形式。这确实是它的一个特性——隐式表达通常描述的是一个连续的对象。但这个连续的对象也可以存储在离散空间中，比如可以用规则网格，在每个网格上存储数值，通过这些数值定义一个空间。只要空间本身的定义是一个函数，确定每个点的值需要求解 $f(x) = 0$ 的方程，那么这个空间就可以是连续的隐式方程。

讲到几何建模时，我们会更多地讲解几何表达，其中**符号距离函数（Signed Distance Function）**就是后面要讲到的一种重要的隐式表达方式。

除了直接写出解析解、通过距离场反向求解的形式外，还有经典的隐式表达例子，比如分形。分形相对复杂，不是本课程的重点，但它比较有趣。分形是通过无穷迭代，每一层按规则重复结构，最终达到极限时组成的确定形状。只不过在有限的计算时间内，很难完成这样的绘制。分形可以是连续的，但很难做到光滑，它属于隐式能够表达的内容，可以表达非常复杂的曲线。

参数化曲线

参数化曲线的定义与优势

显式和隐式各有优缺点。显式的优势是可以高效计算某点的值，但表达能力有限；隐式的表达能力很强，但每次取值都必须求解方程，很多表达特性不直观。

因此很自然地，人们需要一种介于二者之间的表达方式，由此产生了**参数化（parametric）**表达。

需要说明的是，隐式和显式的定义在领域内相对模糊，很难明确划分二者的界限，因此参数化到底属于隐式还是显式，也没有明确的定论。我们先写出参数化的形式。

参数化曲线通常以参数t作为自变量，x和y共同随t变化，写成：

$$\begin{cases} x=f(t) \\ y=g(t) \end{cases}$$

我们可以用经典的例子来理解。之前提到圆用显式表达会比较复杂，但用参数化的方法，就很容易写成关于参数t的x、y表达式。

这样一来，参数化继承了所有显式的优势：可以随机采样，在任意t值处采样，就能高效计算出x、y坐标。这也是目前更多人倾向于认为参数化表达属于显式范畴的原因。

圆很容易用参数化表达，这个优势还可以进一步扩展。圆的解析方程大家很熟悉，但如果要表达螺旋线，找它的可解析方程就会相对抽象。但如果用参数化表达，这件事就会变得容易：在角度旋转的同时，圆的半径逐步增长，只要把半径变成与t相关的表达式，就能扩展成螺旋线的表达。

由此我们可以感受到参数化曲线的优势：首先，它的表达能力远超一般的显式表达；同时，它的计算也非常高效。

为了感受参数化表达的优势，非常推荐大家尝试各类工具软件，比如Desmos，它支持通过简单的交互编写参数化方程。你只要敲入参数，它就可以把这些参数变成用户可控的变量，动态感受参数变化对曲线最终结果的影响。这类软件也体现了参数化曲线易于控制、易于编辑的优势。

我们可以举更多例子，比如各类著名的曲线，以及后面会重点讲解的贝塞尔曲线——也是大家实验课需要实现的内容。总的来说，参数化曲线就是根据某个参数t，对应变化的一组表达形式。

参数化曲线还有另外一个优势：它很容易扩展到三维，甚至可以扩展到更高维度。比如螺旋线扩展到三维，就可以得到空间圆柱螺旋线，大家也可以在Desmos上尝试。这类曲线设计在建筑等领域非常有用。

我们来总结一下参数化表达的优势：

1. 容易扩展到三维；
2. 可视化非常容易，只要确定参数范围（比如0到1），在范围内均匀取点，就可以完成可视化绘制；

3. 便于求解微分。曲线和直线最大的不同是非线性变化，我们通常非常关心曲线的微分变化。对于显式表达 $y = f(x)$， 切线的$\mathrm{dy}/\ \mathrm{dx}$比较好计算； 对于参数化表达， 可以借助参数t，快速得到随t变化的切线微分信息，计算同样高效。

多项式曲线及其不足

参数化方程和隐式方程有一点相似：表达能力强的同时，表达形式通常不唯一。以圆为例，常见的参数化方程是一种形式，但参数可以自行选择，也可以选特殊的量作为参数。任何满足显式方程的解，都可以作为参数化表达，但其中有些形式更友好，有些会带来理解上的困难。

接下来我们介绍几种常见的参数化曲线形式。 首先是最容易理解、 但并不那么通用的**多项式（polynomial） 曲线**，我们会解释它为什么不常用。后面还会讲到贝塞尔曲线、B样条曲线等。这些曲线最终的表达能力可能是相同的，比如都可以表达圆形，但因为各自的优缺点不同，最终流行开来的是某一种，而不是另一种。曲线表达方式的优劣，和实际使用场景有非常大的关系。

多项式曲线形式简单，从数学角度非常容易理解，最开始受到人们的关注，但它存在明显的不足。

我们举一个实际应用的例子：空间中有一些点，想知道这些点定义了怎样一条曲线。这是真实世界中常见的应用场景：比如手绘了一条复杂曲线的草稿，描了一些散点， 想要恢复曲线的形状；又比如扫描真实建筑物的点云，希望从点云恢复背后连续的曲线。

这时我们可能会考虑， 它会不会是一条多项式曲线。 如果是多项式曲线， 点的数量多、 自由度高，就可以尝试用高次多项式曲线去拟合这些点。

我们可以列一个自由度和点数一致的高次多项式方程，把所有点的坐标代入方程，反向求解所有系数，就能得到一条满足所有点都在曲线上的曲线。

多项式曲线的优势在于：

1. 高次方程是可求解的；
2. 保证曲线经过所有给定的点；
3. 高次多项式曲线足够光滑连续，可以求多阶微分。

但它的不足也非常明显。把曲线全貌展示出来就会发现，在定义范围之外，以及边界区域，曲线的形状会严重偏离预期，和最初几个点定义的曲线有非常大的差异。

这体现了高次多项式表达的不足：当幂次较高时，边界经常会出现**过冲（Over shooting）**的情况；同时，反向求解高次方程的过程也比较复杂。而如果不用高次多项式，又很难保证曲线一定穿过所有点。这体现了控制能力和表达复杂性之间的冲突。

分段曲线的思路

更通用、更被广泛选择的参数化曲线，是分段化的曲线，也是我们后面主要学习的内容。比如三次样条曲线，就广泛应用在各个领域。我们课程最开始介绍的字体字形设计，就大量使用了贝塞尔曲线，其中也包含直线段。

答案很明显：如果使用全局的多项式参数化表达，参数数量多、难以控制。我们的选择是把它切断，做分段化处理。

分段化之后，每一小节只需要以足够的自由度，表达周围的几个点就够了。通过分段化，降低了每一段的任务难度，曲线就可以用较低的阶数光滑表达每一段的内容。

整体通过分段，依然能保持全局足够的表达能力，去表达各种各样的曲线，同时也更符合人的理解习惯。字体中的字形就是由分段的贝塞尔曲线和直线组合而成，最终得到非常好用的参数化表达。

分段是一件好事，但也带来了新的挑战：分段之后，首先需要保证线段之间是连接的，这是最基本的C0连续性；除此之外，还会有更高阶的连续性要求。

比如我们可以规定分段的端点必须重合；在此基础上，还希望左右两边的多项式参数化表达，在端点处的微分方向和大小都具有连续性，让整体看起来更加光滑。

接下来我们会着重讲解两类曲线：贝塞尔曲线和样条曲线，重点关注它们在连续性和可控性两方面的表现，以及如何平衡二者。

插值与基函数基础

讲解这些曲线之前，最基础的知识是**插值（interpolation）**。各类曲线大量使用插值能力，不断提高插值的阶数。我们先从插值的基础讲起。

*（注:）线性插值（英文常称 lerp，全称 Linear Interpolation），最简单的理解就是：**已知两个点的数值，在它们之间连一条直线，算出这条直线上任意一个位置的数值**。因为插值的轨迹是直线，所以叫“线性”插值。*

线性插值

插值我们之前已经讲过，线性插值（lerp）是所有插值里最简单的形式。

比如有两个值u_0和u_1，用一个线性核，在u_0和u_1之间进行线性插值。线性插值的好处是，插值结果一定在u_0和u_1之间，并且满足一定的连续性。

把插值扩展到二维的xy坐标，点与点之间分段进行线性插值，就能得到最基础的折线。

另外值得一提的是，插值天然符合参数化曲线的需求。插值过程中天然产生了参数t，调整插值权重的量，正好就是决定参数化表达的参数。

1. 最基础：一维数值的线性插值

比如我们有两个已知的数值 u_0 和 u_1，想求它们之间某个位置的值，就引入一个参数 t（取值范围是 $0 \le t \le 1$）：

- $t = 0$ 对应起点，结果就是 u_0
- $t = 1$ 对应终点，结果就是 u_1
- t 在 0 到 1 之间，就对应两个点之间的位置

计算公式是：

$$\text{插值结果} = (1 - t) \times u_0 + t \times u_1$$

举个具体例子：$u_0 = 10$，$u_1 = 20$

- $t = 0.5$（正中间）：结果 = $0.5 \times 10 + 0.5 \times 20 = 15$
- $t = 0.2$（离起点更近）：结果 = $0.8 \times 10 + 0.2 \times 20 = 12$

其中 $(1 - t)$ 和 t 就是两个值的**权重**：离哪个点越近，哪个点的权重就越大，权重随 t 匀速（线性）变化 —— 这就是之前说的“线性核”。

2. 扩展到二维：坐标的线性插值

把插值用到平面的 xy 坐标上，就是课程里说的“扩展到二维”：
已知两个点 (x_0, y_0) 和 (x_1, y_1)，对 x 坐标和 y 坐标**分别做线性插值**，就能得到中间任意 t 位置的点坐标：

$$\begin{cases} x = (1 - t)x_0 + tx_1 \\ y = (1 - t)y_0 + ty_1 \end{cases}$$

把一串控制点两两之间都做线性插值、连起来，就是最基础的**折线**。

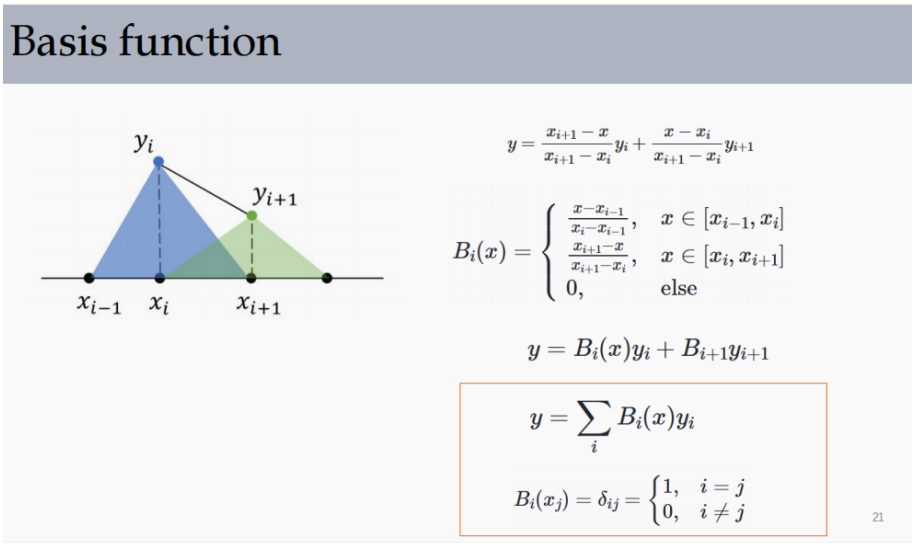
基函数的概念

我们对线性插值做一个扩展：插值的本质是在两个值之间选择不同的权重。对于每一对相邻的值y_i和y_{i+1}，它们都按照权重均匀分布在插值的x空间上。

我们可以可视化线性插值的权重分布：在对应点的位置权重最大，值为1；到下一个点的位置，权重衰减到0。从0到1、再从1到0的区间，是线性插值的基础设定。

我们可以把插值的核写成插值函数的形式，它本身往往是分段函数：在每一段内有对应的值，其他部分为0。它的形状是一个小三角形，中间上升、两边下降，两侧为0。

这个插值函数，我们称之为**基函数（basis function）**。基函数是用于插值的权重函数。



所有的插值过程，都可以写成已知量通过基函数最终累积成目标值的过程。已知量是离散的，基函数把它们延展到连续空间；最终曲线的连续性，很大程度上取决于使用的基函数。如果用线性基函数，最终曲线就是线性连续的情况。

理想的基函数需要满足一些特点：

- 在自身对应的采样点位置，权重正好为1，也就是该点的值就是它本身；
- 在其他基函数对应的采样点位置，尽可能不贡献值。

我们可以用线性基函数，但如果想要更好的连续性，就需要更光滑的基函数。比如高斯基函数，它的分布非常光滑，在自身点位置比重较高，在其他点对应的位置已经衰减到0。用高斯基函数做加权平均，得到的曲线整体更光滑，也更容易通过控制点进行控制。

大家也可以去Desmos上尝试感受各类参数化曲线的易编辑、易控制能力。

插值是图形学中非常重要的概念，因为我们需要用离散空间表达背后连续的对象，同时又希望尽可能高效地操作，插值就成了沟通离散和连续的有效手段。

后面讲到各类曲线时，我们通常都会反过来思考：这条参数化曲线的基函数是什么形式，这往往决定了曲线的很多属性。

(注:)

我们先拆解这句话：

> 在其他基函数对应的采样点位置，尽可能不贡献值。

每个采样点都有**专属的基函数**，比如采样点P_0对应基函数B_0，采样点P_1对应基函数B_1。

这句话的意思是：

基函数B_i（属于采样点P_i），在**别的采样点**P_j **($j \neq i$)** 的位置上，它的权重值最好是 0，也就是不对这个位置的结果产生贡献。

举线性插值的例子（满足这个理想条件）

两个采样点t_0、t_1：

- $B_0(t) = 1 - t$：在t_0处 = 1，在t_1处 = 0（在别人的采样点完全不贡献）
- $B_1(t) = t$：在t_1处 = 1，在t_0处 = 0（在别人的采样点完全不贡献）

插值公式：

$$P(t) = B_0(t)P_0 + B_1(t)P_1$$

当$t = t_1$（属于B_1的采样点）时，$B_0(t_1) = 0$，所以P_0不会对这个点产生任何影响，结果正好等于P_1。

这就能保证曲线**精确经过采样点**，也就是插值曲线。

为什么原文写 "尽可能" 而不是 "必须"?

有些基函数做不到严格等于 0。

比如贝塞尔曲线的伯恩斯坦基函数：它在其他控制点的位置**不等于 0**，所以贝塞尔曲线不会穿过中间控制点，只是被控制点 "拉着走"，属于**近似曲线**，不是插值曲线。

这就是原文说 "尽可能不贡献值" 的原因，这是理想基函数的目标，不是所有基函数都能实现。

贝塞尔曲线

【Freya｜熟肉】贝塞尔曲线之美_哔哩哔哩_bilibili

Animated Bézier Curves - Jason Davies

直观认识与构造过程

接下来我们来看贝塞尔曲线。二维贝塞尔曲线由曲线和上方的控制点组成。

大家可能对贝塞尔曲线并不陌生，比如系统自带的画图软件中画曲线，内部使用的就是贝塞尔曲线：先画起点、再画终点，然后设计控制点，就会自动形成曲线的形状。在PowerPoint等Office软件中编辑形状时，很多形状也都是以贝塞尔曲线的方式呈现的。

我们可以直观感受一下贝塞尔曲线的构造：通过拖动起点、终点和控制点，可以改变曲线的形状。

贝塞尔曲线通过控制点以插值的方式最终生成。

我们可以通过辅助线直观理解插值过程：比如三个点可以插值出一条线段，另外三个点插值出另一条线段；这两条线段再做一次插值，最终得到的就是贝塞尔曲线。

贝塞尔曲线的优势首先是好控制：可以控制起点和终点的位置，还可以控制端点处切线的方向。比如希望曲线终点处的边和y轴平行，就可以把控制点拖到和y轴一致的方向，曲线就会更贴合y轴；同理，如果希望起点处和x轴平行，也可以通过控制点实现一阶微分的控制。

贝塞尔曲线于1962年在实际工作中被提出，最初的应用场景是汽车设计。当时汽车是昂贵的奢侈品，人们希望它的造型足够经典，同时符合空气动力学；为了方便控制形状，同时保证足够光滑、不产生额外空气阻力，贝塞尔曲线应运而生。

阶数与递归构造

我们来看更数学化的内容。贝塞尔曲线是反复插值的过程，我们把它具体写下来。

先从只有一个控制点的情况开始，方便理解插值过程。曲线的形成过程可以按点的生成来理解：控制点和起点、端点连成两条直线，在起点和控制点之间做线性插值，在控制点和终点之间做线性插值，得到两个中间点。

接下来，对这两个中间点再做一次线性插值。我们可以用动图来感受这个过程：想象有一个小点从起点移动到控制点，同时另一个小点从控制点移动到终点；这两个点移动的过程中，它们之间拉着一根可以自由延伸的弹性绳，弹性绳上又有一个小点，从绳的起点走到绳的终点。这个中间点的移动轨迹，就是贝塞尔曲线的轨迹。

从公式上来说，中间移动的两个点通过线性插值获得，最终形成曲线轨迹的点，由这两个中间点再做一次线性插值获得。

最值得注意的是：每一次线性插值都会引入一个新的参数t，而贝塞尔曲线定义所有插值过程共用同一个参数t。t的取值范围在0到1之间，当t=0.6时，在起点和控制点之间插值到0.6的位置，在控制点和终点之间也插值到0.6的位置，最终曲线上的点也对应t=0.6的位置。

通过共用唯一的参数t，三步插值统一在一起，就完成了0到1区间内贝塞尔曲线的插值。

这里面包含递归的思想：

- 如果只有两个点，连接起来就是直线；
- 如果起点和终点中间有一个控制点，需要先完成第一层的两个线性插值，再完成第二层的插值，得到整条曲线；
- 如果增加更多的控制点，中间的线段每次都会增多，为了最终得到一条轨迹，就需要多一层插值过程。

每一层插值都基于线性插值，每增加一个控制点，就意味着多了一层线性插值，参数方程就增加一个阶数：

- 1个控制点 → 二阶贝塞尔曲线（二次）
- 2个控制点 → 三阶贝塞尔曲线（三次）

原因在于：两个点之间的直线是关于t的一次函数；控制点的轨迹是t的一次函数，两个一次函数再做线性插值，乘以(1-t)和t，幂次就升级到二次。因此一个控制点的曲线叫做二阶贝塞尔曲线。同理，每加一个控制点，就多一层线性插值，阶数从2次变到3次。这就是贝塞尔曲线作为多项式曲线，阶数增加的过程。

2. 常见低阶的具体形式

对应课程里的例子：

- **1 阶（直线）**：2 个控制点

  $$\boldsymbol{B}(t) = (1-t)P_0 + tP_1$$

  就是普通的线性插值，直线是特殊的贝塞尔曲线。

- **2 阶（二次）**：3 个控制点

  $$\boldsymbol{B}(t) = (1-t)^2P_0 + 2t(1-t)P_1 + t^2P_2$$

- **3 阶（三次）**：4 个控制点

  $$\boldsymbol{B}(t) = (1-t)^3P_0 + 3t(1-t)^2P_1 + 3t^2(1-t)P_2 + t^3P_3$$

  系数 1、3、3、1 就是杨辉三角的第三行，和课程描述完全对应。

代码实现思路

实验课lab1的任务，是写一个迭代算法来实现贝塞尔曲线。大家可以使用AI来辅助完成。

我们讨论一下如何用递归函数实现贝塞尔曲线。对于代码框架来说，比较有挑战的是函数的输入输出应该如何设计。

贝塞尔曲线的API调用已经基本固化：通常将所有控制点作为列表输入，再输入参数t的值，函数的任务就是返回这组控制点定义的贝塞尔曲线，在参数等于t时的屏幕坐标。

有了这样的定义，递归实现的思路就很清晰：先确定简单情况下的返回结果，再定义递归调用的规则。

这里额外说明一点：上节课我们讲直线扫描时做了非常多的优化，讲到曲线时，我们让大家实现递归的方法。大家可能会觉得递归方法效率不高，但事实上，真实场景中绘制曲线最常用的就是递归方法。

主要原因是：曲线自身的复杂度远超过直线，高次多项式曲线确实可以写出解析式，每个点都可以通过多项式计算。但对于离散浮点数运算来说，直接计算多项式会有很大的精度误差——尤其是控制点和t都在0到1之间，会出现很小的数和很大的数相乘的情况，导致浮点数误差，最终产生连续性上的问题。

为了保持较好的连续性，我们依赖插值的方式，而依赖插值就必须递归地完成曲线绘制。

数学形式与伯恩斯坦基函数

接下来我们从解析形式感受贝塞尔曲线的性质，理解它为什么流行、为什么切向容易控制。

二阶贝塞尔曲线的参数化形式展开后，我们可以把它写成基函数的形式，观察贝塞尔曲线的基函数分布。二阶贝塞尔曲线的基函数正好呈现杨辉三角的形状。增加阶数到三阶，基函数也会对应更高一层的杨辉三角形式。

这并不是巧合：因为基函数由一级一级的线性插值生成，每一次插值都在两个参数之间选择(1-t)或者t，取决于对应的控制点。最终每个控制点的权重，对应选了多少次(1-t)、多少次t，正好是组合数C(n,k)的排列。

因此贝塞尔曲线的基函数展开后，正好是杨辉三角的形式。随着控制点增多，阶数逐渐增高，每一项的幂次都会相应增加。由此我们可以把任意阶数的贝塞尔曲线，写成更通用的数学形式。

这类基函数对应的多项式，叫做**伯恩斯坦多项式（Bernstein polynomial）**，这个名字也被用来定义这类基函数。

一、定义

对于 n 次多项式，一共有 n+1 个伯恩斯坦基函数，第 k 个（k 从 0 到 n）的表达式为：

$$B_{n,k}(t) = \binom{n}{k} t^k (1-t)^{n-k}, \quad t \in [0,1]$$

其中 $\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$ 是组合数，也就是杨辉三角里对应的数值。

二、低阶实例（对应贝塞尔曲线）

我们常用的低阶贝塞尔曲线，权重就是对应阶数的伯恩斯坦多项式：

1. **1 次（线性贝塞尔 / 直线插值）**
   - $B_{1,0}(t) = 1 - t$
   - $B_{1,1}(t) = t$

     这就是线性插值 `lerp` 的两个权重。
2. **2 次（二次贝塞尔）**
   - $B_{2,0}(t) = (1-t)^2$
   - $B_{2,1}(t) = 2t(1-t)$
   - $B_{2,2}(t) = t^2$

     对应二次贝塞尔曲线的三项系数。

数值稳定性与矩阵形式

我们可以总结一下贝塞尔曲线的计算方式：可以用迭代的方法，也可以用解析的方法。

在基函数中，会频繁出现 $t\cdot(1-t)$ 的结构。因为t处于0到1之间，t和(1-t)一个接近0、一个接近1，二者相乘会带来浮点数精度的问题，误差容易被放大。

因此直接计算高次贝塞尔曲线的参数形式，浮点数乘法非常不稳定；相对而言，迭代算法往往更加高效、稳定。

除此之外，我们还可以把贝塞尔曲线写成向量与矩阵相乘的向量化计算结构：把参数写成向量，和控制点向量做点乘；或者进一步拆解基函数，提取参数的系数矩阵，写成“向量-矩阵-向量”的乘积形式。

写成矩阵形式的好处，主要是便于向更高阶、更多控制点的曲线扩展，也便于向二维、三维扩展，能帮助我们更好地理解曲线的可扩展性。

从基函数的角度感受：用线性基函数插值得到的是线性插值结果；用三阶贝塞尔对应的三阶基函数做插值，得到的曲线就更加连续。曲线的连续性本质上由基函数决定。

矩阵形式也更方便我们计算一阶微分、二阶微分，分析曲线的法向等性质。

端点与切线性质

贝塞尔曲线，尤其是三阶贝塞尔曲线，使用非常广泛。它能保证经过两个端点，这是插值本身的性质；同时，还可以用另外两个控制点控制两端的切线方向。我们从数学上解释这个性质。

通过矩阵形式对参数t求一阶微分，就可以得到端点处的微分表达式。以三阶贝塞尔曲线为例：

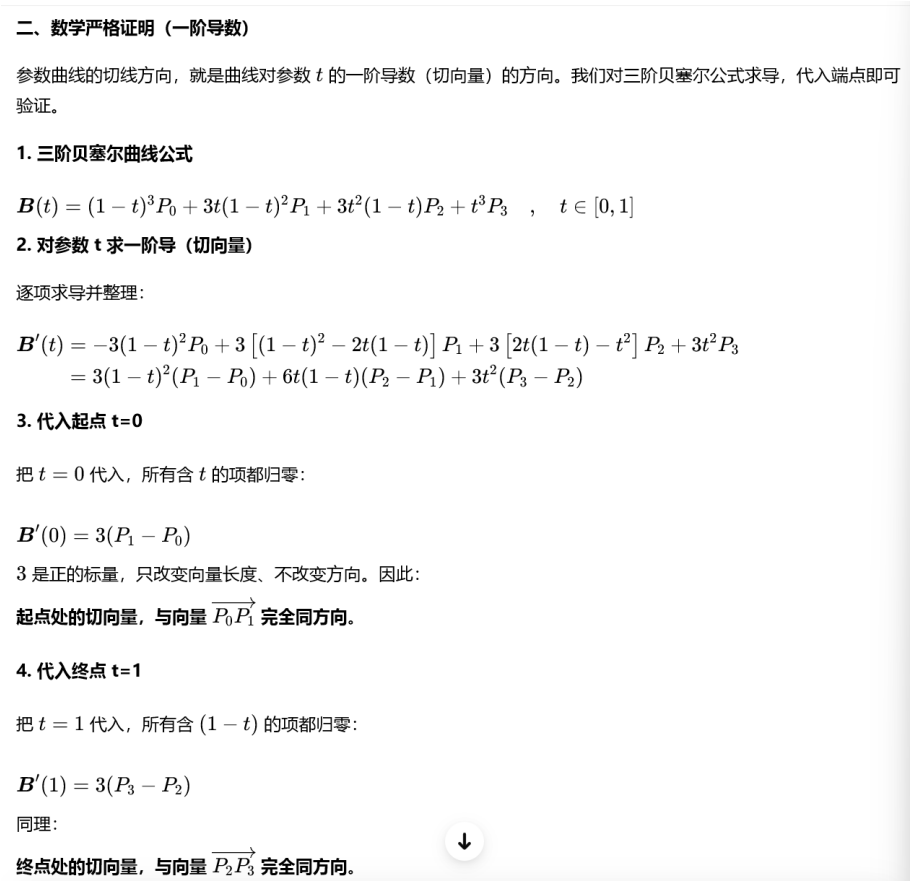
- 在起点t=0处，切线方向等于起点与第一个控制点连线的方向；
- 在终点t=1处，切线方向等于最后一个控制点与终点连线的方向。

更高阶的贝塞尔曲线，在端点处也都保持这样的性质。

这个性质带来的好处是：贝塞尔曲线非常适合作为分段函数中的一段。它能保证经过端点，保证C0连续；还可以通过调整控制点，实现C1连续，也就是切线的连续。

每一段三阶贝塞尔曲线有4个控制点：两个是起点和终点，保证曲线经过这两个点；另外两个是控制点，通过拖动它们可以控制曲线的切向，得到圆润的拐弯，也可以控制曲线的间距。配合直线段，它就是非常适合设计工作的曲线。

如果要实现C1连续，只需要让相邻两段在连接点处的两个控制点，沿同一方向且距离相等，就能保证该点处切线方向相同、大小相等，完成C1连续。



二、数学严格证明（一阶导数）

参数曲线的切线方向，就是曲线对参数 t 的一阶导数（切向量）的方向。我们对三阶贝塞尔公式求导，代入端点即可验证。

1. 三阶贝塞尔曲线公式

$$\boldsymbol{B}(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3 \quad , \quad t \in [0,1]$$

2. 对参数 t 求一阶导（切向量）

逐项求导并整理：

$$\begin{aligned}\boldsymbol{B}'(t) &= -3(1-t)^2 P_0 + 3\left[(1-t)^2 - 2t(1-t)\right] P_1 + 3\left[2t(1-t) - t^2\right] P_2 + 3t^2 P_3 \\ &= 3(1-t)^2(P_1 - P_0) + 6t(1-t)(P_2 - P_1) + 3t^2(P_3 - P_2)\end{aligned}$$

3. 代入起点 t=0

把 $t = 0$ 代入，所有含 t 的项都归零：

$$\boldsymbol{B}'(0) = 3(P_1 - P_0)$$

3 是正的标量，只改变向量长度、不改变方向。因此：

起点处的切向量，与向量 $\overrightarrow{P_0P_1}$ 完全同方向。

4. 代入终点 t=1

把 $t = 1$ 代入，所有含 $(1-t)$ 的项都归零：

$$\boldsymbol{B}'(1) = 3(P_3 - P_2)$$

同理：

终点处的切向量，与向量 $\overrightarrow{P_2P_3}$ 完全同方向。

连续性：C连续与G连续

对于分段曲线来说，连续性非常重要，在汽车设计等实际应用中都是核心需求。我们把连续性单独总结。

大家比较熟悉的是**C类连续性**，它完全来自数学定义：自变量是参数t，因变量是插值得到的曲线坐标。C类连续性描述的是坐标对t的各阶导数的连续性。

- C0连续：两个端点重合，位置连续；
- C1连续：一阶导数连续，也就是切线不仅平行，大小也相等；

- C2连续：二阶导数也连续匹配。

但需要注意的是，曲线的几何形状和参数本身没有必然联系。比如同一条曲线，可以用t做参数，也可以用2t做参数，只要取值范围对应调整，最终几何上的点完全一致。

因此在曲线领域，我们额外提出一组连续性定义，叫做**G连续（几何连续性）**。

G连续和C连续有很多相近的特性：

- G0连续和C0连续完全一致，是最基础的连续，即两个端点相接；
- G1连续只要求切线方向一致，不关心切线的大小（也就是参数变化的速率），比C1连续的要求更宽松；
- G2连续建立在G1的基础上，要求曲率连续。曲率可以理解为曲线某点处外接圆半径的倒数，描述切线随弧长的变化率，是和二阶微分相关的量。

需要说明的是：当参数是弧长时，曲率正好等于二阶微分的模长；如果参数不是弧长，曲率可以通过一阶、二阶微分计算得到。

两类连续性的包含关系：

- C类连续性中，C2连续必然满足C1连续，C1连续必然满足C0连续；
- G类连续性中，G2连续必然满足G1连续，G1连续必然满足G0连续；
- C2连续必然满足G2连续，但跨阶不存在包含关系：C1连续不代表满足G2连续，G2连续也不代表满足C1连续。

我们可以直观理解不同阶的G连续：

- G0：两条曲线在端点处相接即可；
- G1：两条曲线在端点处切线方向相同；
- G2：两条曲线在端点处曲率也连续匹配，切线变化的速率一致，整体更加平滑；
- G3：曲率本身也可微，连续性进一步升高。

工业界很多场景需要G2连续，因为仿真计算会要求几何表面的法线连续，甚至法线的微分也存在。设计领域的实际需求大概在G1到G2之间：纯视觉设计G1用得更多，涉及仿真、偏微分方程求解的场景对G2连续性需求更高。

贝塞尔曲线可以很容易做到G0、G1连续，甚至C1连续；但要做到G2、C2、G3连续，难度就会高很多。如果放开所有自由度，可以调整出满足G2的曲线；但如果入口和出口的控制点是固定的，不能天然保证一定存在满足条件的贝塞尔曲线。

注：对于单段n阶非退化贝塞尔曲线，一定满足$C^1, C^2, \ldots, C^n, G^1, G^2, \ldots, G^n$.上面的"条件"讨论的是多条曲线拼接。

贝塞尔曲面（三维扩展）

贝塞尔曲线非常容易扩展到三维。扩展到三维时，控制点从二维点变成三维空间点，同时引入两组参数u和v，就可以构成贝塞尔曲面。

比如4×4的三阶贝塞尔曲面，有4×4个控制点，和二维贝塞尔曲线非常类似：曲面一定会经过四个角的控制点，中间的控制点可以用来控制曲面沿u方向和v方向的切线，进而控制曲面的切面。

数学上也可以规整地写成u、v两组参数化形式，中间是控制点和参数矩阵，很容易应用到实际工作中做数学推导。

凸包性与局限性

贝塞尔曲线还有额外的好处：它具有**凸包性**。如果用控制点做一个包围盒，曲线一定落在这个包围盒内部。

这是因为曲线通过插值生成，即使控制点出现交叉的情况，插值结果也一定会落在控制点的包围盒内部。这个局域性特征非常方便计算碰撞检测等相关问题。

接下来我们讲贝塞尔曲线的不足。

贝塞尔曲线是高阶多项式，可以分段连接。但如果修改中间某一段曲线，并且要求保持连续性，就会影响相邻的分段。

比如要求C1连续性，意味着连接点两侧的控制点是相互耦合的关系。修改这一段曲线，一定会对相邻段产生影响；如果还要保证更多的连续性，就会连带影响更多分段。

因此贝塞尔曲线的控制虽然比全局多项式更局部，但分段之间依然存在耦合，修改一段会影响至少相邻两段。

这就形成了一组矛盾：增加曲线的阶数和连续性，会提升光滑性，但同时也会增加全局性，削弱局部控制能力。

总的来说，三阶贝塞尔曲线是最常用的，更高阶计算会更复杂。贝塞尔曲线被广泛的画图工具包所支持，比如我们课程实验使用的OpenGL，就原生支持贝塞尔曲线。

前沿拓展：犹他茶壶与G2连续问题

贝塞尔曲线从1960年代出现至今，已经是非常经典的内容。但有趣的是，近年的SIGGRAPH国际会议上，依然有相关的研究工作。

过去大家普遍认为，贝塞尔曲线能保证C1连续，保证不了G2连续是可以接受的。但最近的一篇SIGGRAPH论文专门展示了：只有C1连续性、没有G2连续性，确实会带来可见的问题。

这项工作和犹他茶壶（Utah Teapot）50周年相关。犹他茶壶是较早的图形学3D数字化模型，基于贝塞尔曲面设计。当时的设计者手工调整了一套贝塞尔曲面参数，来匹配真实茶壶的外形。为了保证连续性，最终得到的贝塞尔曲面茶壶是C1连续的，它被广泛应用在各类图形学渲染任务中。

渲染中的反射现象，会放法线连续性的问题。如果法线足够连续，反射出的窗户依然是横平竖直的；如果法线不够连续，窗户的投影就会被扭曲。

大家可以去论文的网站上查看动态效果，会有更直观的感受：茶壶曲面上有一条分界线，能明显看到窗户反射有时被“吃掉”、有时又出现，这就是法线变化速率不连续、C1连续性不足带来的视觉问题。而达到G2连续的曲面，反射效果会更加光滑。

如果想让曲线从C1变成G2，就不再是三阶贝塞尔曲线了。可以通过增加阶数的方式，让曲率达到更连续的状态；如果不想增加阶数，就必须分更多的段，来满足更高的连续性需求。

这项工作的核心贡献是：在提高连续性的同时，依然能让用户像控制贝塞尔曲线一样控制曲线。

其他参数化曲线

既然贝塞尔曲线在高阶连续性上存在局限，我们自然会问：能不能找到一种曲线，更容易满足G2甚至C2连续性，同时又方便用户控制，不需要反复调整左右两边？

接下来我们介绍其他几类曲线。

三次样条曲线

样条曲线（spline curve）是真实世界中天然存在的一类曲线。比如木匠制作木条时，敲几个控制点，把有韧性的木条或竹条放在中间，木条会自动平衡内部张力，稳定平衡后的形状，就是自然三次样条曲线的效果。同理，弹性绳固定几个点后，自然张成的曲线也是样条曲线。

我们先对曲线做一个分类：

- **插值曲线**：精确经过给定的控制点，对位置的控制更精准；
- **近似曲线**：不精确经过所有控制点，只在大致区域内，给曲线留出更多自由度来保证光滑性。

贝塞尔曲线就属于近似曲线，每一段内部都在控制点的包围盒内。插值曲线有可能超出包围盒，但至少已知几个点的固定位置，空间范围依然可计算。

我们来看三次插值样条曲线，这也是产业界用得比较多的曲线。它本身是分段的，每一段都是三次多项式，通过连接构成整体曲线。我们通常希望它至少满足C0连续，最好C1也连续。

接下来的问题是：如果已知一组控制点，要求曲线是三次插值曲线，同时满足C2连续性，这条曲线能不能求出来?

答案是可以的。每一段三次多项式有4个参数，两段就有8个参数。我们可以通过约束条件列方程：

- 端点经过控制点，得到位置约束方程；
- C1连续，得到一阶导数约束方程；
- C2连续，得到二阶导数约束方程。

把所有边界条件列出来，会得到一个方程组。对于两段三次曲线，未知数和约束条件的数量正好相等，因此可以求解出所有参数。

最终得到的就是自然三次样条曲线，它天然满足二阶连续的需求。

它之所以能和自然界的现象匹配，是因为二阶导连续的情况下，曲线能够最小化弹性势能，是物体满足约束条件下能量最小的状态。

但它也有不足：求解过程是全局解方程，改变其中一个控制点，就会改变所有约束，最终所有分段的参数都会发生变化。也就是说，修改一个点，整条曲线都会跟着变动，来保证整体的C2连续性。

这再次体现了连续性和局部控制之间的矛盾：连续性越高，控制的全局性就越强。三次样条非常光滑，但局部控制能力差。

Catmull-Rom曲线与厄米曲线

在设计领域，很多时候大家会选择放弃部分连续性，比如只要求C1连续，来换取更强的局部控制能力——修改某一段曲线时，其他分段不受影响。

这类曲线的代表就是**Catmull-Rom曲线**和**厄米（Hermite）曲线**，都属于C1连续的曲线，在图形学中应用广泛。

Catmull-Rom曲线

Catmull-Rom曲线的定义是：曲线本身连续，并且中间点的一阶微分，等于前后两个控制点连线的方向。

基于这两个约束，通过当前点、前一个点和后一个点的位置，就可以唯一解算出每一段三次多项式的具体形式。也就是说，每一段曲线只受相邻三个点的影响，实现了局部控制。

厄米曲线

厄米曲线则是把一阶微分的自由度直接交给用户：由用户决定每个端点的位置，以及每个控制点上的一阶微分。

点的位置和一阶微分构成四个约束条件，唯一控制一段曲线的多项式表达。每一段曲线都由自己的端点和微分控制，用户只要保证相邻端点的微分一致，就能满足C1连续。

厄米曲线和贝塞尔曲线之间存在一一对应的映射关系：任何一条厄米曲线，都有一条完全对应的贝塞尔曲线。

原因在于：厄米曲线给定了两个端点和两端的一阶微分，而三阶贝塞尔曲线由两个端点和两个控制点定义。已知端点和一阶微分，就可以计算出对应的控制点位置，得到完全相同的曲线。反过来，给定贝塞尔曲线的控制点，也可以算出端点处的一阶微分，得到对应的厄米曲线。因此二者的表达能力是一一对应的，厄米曲线可以看作是贝塞尔曲线中满足C1连续约束的一个子集。

B样条曲线简介

最后我们简单介绍B样条曲线，下节课会详细讲解。B样条的B是basis（基）的意思。

B样条曲线有两个核心特点：

1. 操作是局部影响的，修改一个控制点只会影响附近的分段；
2. 可以自动满足甚至C2的连续性。

B样条曲线属于近似曲线，它不经过任何控制点，但一定保证曲线在控制点的包围盒内部。通过减少对精确位置的限制，B样条曲线获得了更好的光滑性，具备非常多优秀的特性。

B样条曲线和贝塞尔曲线有很多相似性。可以这么理解：把两条贝塞尔曲线连接的公共端点拿掉，在相邻的控制点之间做插值，离哪个控制点近，哪条曲线的权重就高，最终过渡得到的就是B样条曲线。

下节课我们会详细拆解B样条曲线。

课堂问答

问题1：C1连续和G1连续的核心区别是什么？

答：核心区别在于是否关心参数变化的“速度”。
C1连续是对参数t的一阶导数连续，既要求切线方向相同，也要求导数的大小相等，对应参数t变化的速率一致。如果把曲线想象成小球运动的轨迹，C1连续的情况下小球会以均匀的速度经过连接点。
G1连续是几何连续性，只要求切线方向一致，不关心导数的大小，也就是不关心参数变化的快慢。哪怕两边参数变化的速度不同，只要切线方向相同，就满足G1连续。
比如两条曲线在连接点处切线方向一致，但一边参数变化快、一边变化慢，几何上看起来是相切的，这就是G1连续，但不是C1连续。

问题2：厄米曲线和贝塞尔曲线的对应关系是什么？

答：任何一条厄米曲线，都可以找到一条完全对应的贝塞尔曲线，二者表达能力是一一对应的。
厄米曲线由两个端点和两端的一阶微分定义，而三阶贝塞尔曲线由两个端点和两个控制点定义。已知端点和一阶微分，就可以计算出对应的控制点位置，得到完全相同的曲线。反过来，给定贝塞尔曲线的控制点，也可以算出端点处的一阶微分，得到对应的厄米曲线。
厄米曲线可以看作是贝塞尔曲线中，满足C1连续约束的一个子集。

问题3：伯恩斯坦多项式可以表示任意多项式曲线吗？

答：是的。伯恩斯坦多项式本身构成了多项式空间的一组基，因此它可以表示出任意同阶的多项式曲线。这也是为什么各类三次曲线之间可以相互转换的原因。

问题4：考试中曲线部分的考察重点是什么？

答：考试中曲线部分的考察不会特别难，核心是希望大家建立显式、隐式和参数化曲线的概念与直观感受。习题中会涉及相关实现，考试不会考过于细节的区别。

LESSON 5.1（2026-09-28 第1-2节）

Bellaard.com: B-Splines

第1节 B样条曲线的引入（对应PPT第56页）

1.1 贝塞尔曲线的控制特性回顾

贝塞尔曲线的参数表达式为：

$$p(t) = b(t)^T (\boldsymbol{P})$$

其中$(\boldsymbol{P}) = (\boldsymbol{P})_0, (\boldsymbol{P})_1, ..., (\boldsymbol{P})_m$为控制点集合。贝塞尔曲线的控制特性存在鲜明的两极分化：

- **单段内部：全局影响**：在一段贝塞尔曲线中，任意一个控制点的修改都会牵动整段曲线的形状，所有控制点对整段曲线都有贡献；
- **段与段之间：无影响**：分段拼接的多段贝塞尔曲线，各段的控制点相互独立，修改某一段的控制点不会影响其他段落。

这种“全局控制”的特性，使得贝塞尔曲线在构建长曲线、复杂曲线时调整不够灵活，由此催生了具备局部控制能力的B样条曲线。

1.2 B样条曲线的定义式

B样条（B-Spline）全称为“基样条（Basis Spline）”，是对贝塞尔曲线的改进，其核心表达式为：

$$b(t) = \sum_{i=0}^{m} N_{i,n}(t) \cdot (\boldsymbol{P})_i$$

其中：

- $(\boldsymbol{P})_i$ 为第i个控制点；
- $N_{i,n}(t)$ 称为**第i个n次B样条基函数**，它是一个n次多项式，决定了对应控制点$(\boldsymbol{P})_i$对曲线的权重；
- 本质上，B样条曲线上的每一点，都是对应范围内控制点的加权求和。

1.3 核心基础概念：节点（Knot）

节点是参数轴上的一系列分点，是B样条“局部控制”特性的核心来源。所有B样条基函数的定义域都落在节点区间 $[u_0, u_m]$ 上；为了计算方便，通常将参数区间归一化到 $([0,1])$，即令 $u_0 = 0, u_m = 1$。

第2节 B样条基函数的递归定义（对应第57-58页）

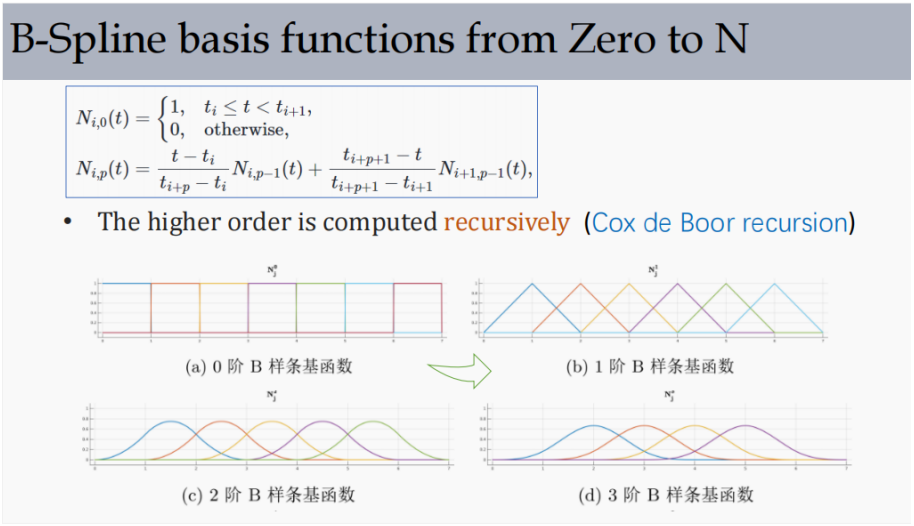
B样条基函数采用递归方式定义，称为**Cox-de Boor递归公式**，这是所有B样条基函数的统一数学形式。

2.1 递归起点：0阶B样条基函数

最低阶的0次B样条基函数是分段常数函数，定义为：

$$N_{i,0}(t) = \begin{cases} 1, & t_i \le t < t_{i+1} \\ 0, & \text{otherwise} \end{cases}$$

- 物理意义：第i个0阶基函数，只在第i个节点区间 $[t_i, t_{i+1})$ 内取值为1，区间外恒为0；
- 形态：矩形脉冲函数，本身在节点处存在跳跃间断；
- 地位：是整个B样条基函数体系的底层基础，所有高阶基函数都由它递归生成。



2.2 Cox-de Boor递推公式

对于p阶（$p \ge 1$）的B样条基函数，通过两个相邻的$p-1$阶基函数线性加权组合得到，递推公式为：

$$N_{i,p}(t) = \frac{t - t_i}{t_{i+p} - t_i} N_{i,p-1}(t) + \frac{t_{i+(p+1)} - t}{t_{i+(p+1)} - t_{i+1}} N_{i+1,p-1}(t)$$

公式解释：

- 第一项权重 $\frac{t - t_i}{t_{i+p} - t_i}$：随参数$t$从左到右从0线性增加到1；
- 第二项权重 $\frac{t_{i+(p+1)} - t}{t_{i+(p+1)} - t_{i+1}}$：随参数$t$从左到右从1线性减少到0；
- 本质：两个相邻的低阶基函数，通过线性渐变过渡拼接成一个更平滑、范围更宽的高阶基函数。

2.3 低阶基函数的形态

通过递推公式可以得到各阶基函数的典型形态：

- **0阶**：矩形脉冲，函数本身不连续；
- **1阶**：三角形（折线），函数连续，但节点处一阶导数不连续（存在尖点）；
- **2阶**：二次多项式构成的光滑钟形，一阶导数连续；
- **3阶（三次）**：三次多项式构成的对称钟形，二阶导数连续——这是工程与图形学中最常用的形式。

2.4 基函数的核心性质：单位分解

对任意参数$t\in([0,1])$，所有在该位置有非零值的基函数，函数值之和恒等于1：

$$\sum_i N_{i,p}(t)=1$$

该性质保证了B样条曲线上的每一点都是控制点的**凸组合**，是凸包性质的数学基础。

第3节 B样条基函数的通用规律（对应第59-60页）

所有B样条基函数的特性都可以从递归定义推导而来，是B样条曲线所有工程特性的根源。

3.1 局部支撑规律

p阶B样条基函数$N_{i,p}(t)$的**支撑区间**（函数值非零的参数范围）严格限定在 $[t_i, t_{i+(p+1)}]$ 内，即**横跨$(p+1)$个节点区间**，超出该范围函数值恒为0。

- 例：3阶（三次）B样条基函数，支撑区间横跨4个节点区间；
- 意义：这是B样条“局部控制”特性的数学根源——修改一个控制点，只会影响它周围$(p+1)$段曲线，不会牵动全局。

3.2 光滑性规律

在简单节点（节点不重复）处，p阶B样条基函数具有 $p-1$ 阶连续导数：

- 1阶（一次）：C^0连续（函数本身连续，导数不连续）；
- 2阶（二次）：C^1连续（一阶导数连续）；
- 3阶（三次）：C^2连续（二阶导数连续）；
- 一般规律：阶数每升高1阶，可导性提高1阶。

3.3 非负性与凸包性质

在支撑区间内，B样条基函数的函数值始终≥ 0，不会出现负权重。
结合单位分解性质（和为1），可以直接推导出：**B样条曲线完全落在控制点构成的凸包（凸多边形）内部**，不会超出控制点的范围。

3.4 局部控制特性的本质

对参数轴上任意一个位置t，只有相邻的$(p+1)$个基函数有非零值。对应到曲线上：

- 曲线上任意一点，只受距离它最近的$(p+1)$个控制点影响；
- 反过来，修改任意一个控制点，只会改变它周围$(p+1)$个节点区间内的曲线形状。

这是B样条对比贝塞尔曲线最核心的优势：贝塞尔是“单段全局控制”，B样条是“分段局部控制”，调整细节不会破坏整体形状。

第4节 三次均匀B样条的显式表达（对应第61-62页）

对于均匀节点的特殊情况，B样条基函数可以展开为固定的显式多项式，其中三次（3阶）均匀B样条是工程应用最广泛的形式。

4.1 参数归一化

计算单段三次B样条时，将当前节点区间的参数t归一化到 $u\in([0,1])$：

$$u=\frac{t-t_i}{t_{i+1}-t_i}$$

其中t_i和t_{i+1}是当前区间的左右节点。归一化后，每一段曲线都可以在统一的$([0,1])$区间内计算。

4.2 三次均匀B样条的显式基函数

对于第i段曲线，它由相邻的4个控制点 $p_{i-2},p_{i-1},p_i,p_{i+1}$ 共同决定，对应的4个三次基函数的显式表达式为：

$$\boldsymbol{b}(u)=\frac{1}{6}\begin{bmatrix}\ (1-u)^3 & 4-6u^2+3u^3 & 1+3u+3u^2-3u^3 & u^3\ \end{bmatrix}$$

该组基函数满足：

1. 定义域内函数值非负；
2. 4个基函数之和恒等于1（单位分解）；
3. 均为三次多项式，天然保证分段处的C^2连续性。

4.3 矩阵形式（便于编程计算）

三次均匀B样条可以写成矩阵乘法形式，极大地方便编程实现：

$$p(u)=\boldsymbol{u}^T\boldsymbol{M}_S\boldsymbol{p}=\boldsymbol{b}(u)^T\boldsymbol{p}$$

其中：

- 幂向量：$\boldsymbol{u}=\begin{bmatrix}\ 1 & u & u^2 & u^3\ \end{bmatrix}^T$
- 控制点向量：$\boldsymbol{p}=[\ p_{i-2} \quad p_{i-1} \quad p_i \quad p_{i+1}\ ]^T$
- 三次均匀B样条基矩阵（常数矩阵）：

$$\boldsymbol{M}_S=\frac{1}{6}[\ 1 \quad 4 \quad 1 \quad 0 \quad -3 \quad 0 \quad 3 \quad 0 \quad 3 \quad -6 \quad 3 \quad 0 \quad -1 \quad 3 \quad -3 \quad 1\ ]$$

4.4 凸包性质验证

由于基函数非负且和为1，曲线上任意一点都是4个相邻控制点的凸组合，因此单段曲线必然落在这4个控制点构成的凸包内，整条曲线则落在所有控制点的凸包内。

第5节 三次B样条的分段逻辑与核心特性（对应第63-64页）

5.1 滑动窗口的分段机制

整条三次B样条曲线采用“滑动窗口”的方式分段生成：

- 第1段曲线：取前4个控制点代入基矩阵计算；
- 第2段曲线：控制窗口向后滑动1位，取第2-5个控制点，复用完全相同的基矩阵计算；
- 以此类推，每新增1个控制点，就多生成1段曲线。

规律：**控制点数量 = 分段数 + 阶数**。例如16段三次B样条，需要$(16+3=19)$个控制点。

5.2 天然高阶连续性

三次B样条在分段与分段的连接处，**天然满足C^2连续**——函数值、一阶导数、二阶导数全部连续。

- 该特性由基函数的数学性质自动保证，不需要像拼接贝塞尔曲线那样手动调整控制点共线；
- 这是B样条在工业建模中被广泛应用的核心原因之一：自动保证光滑性，大幅降低造型成本。

5.3 与三次贝塞尔曲线的等价转换

每一段三次B样条，都可以等价转换为一段三次贝塞尔曲线，两者几何形状完全一致，仅控制点表示不同。
设贝塞尔基矩阵为$\boldsymbol{M}_B$，贝塞尔控制点向量为$(\boldsymbol{P})_B$；B样条基矩阵为$\boldsymbol{M}_S$，B样条控制点向量为$(\boldsymbol{P})_S$，则有：

$$p(u)=\boldsymbol{u}^T\boldsymbol{M}_B(\boldsymbol{P})_B=\boldsymbol{u}^T\boldsymbol{M}_S(\boldsymbol{P})_S$$

因此控制点转换公式为：

$$(\boldsymbol{P})_B=(\boldsymbol{M}_B)^{-1}\boldsymbol{M}_S(\boldsymbol{P})_S$$

该性质是建模软件中B样条与贝塞尔互相切换编辑的数学基础。

第6节 三次B样条曲面片（对应第66页）

B样条曲面是B样条曲线的张量积推广，继承了B样条曲线的全部特性。

6.1 张量积构造思想

将两个方向的B样条基函数做张量积，即可得到B样条曲面的基函数；对应的控制点从一维序列扩展为二维网格。

6.2 曲面的数学表达式

三次B样条曲面片的表达式为两个方向三次B样条基函数的张量积：

$$p(u,v)=\sum_{i=0}^{3}\sum_{j=0}^{3}b_i(u),b_j(v),p_{\mathrm{ij}}$$

写成矩阵形式为：

$$p(u,v)=\boldsymbol{u}^T\boldsymbol{M}_S(\boldsymbol{P})\,\boldsymbol{M}_S^T\boldsymbol{v}$$

其中：

- $\boldsymbol{M}_S$ 为三次B样条基矩阵，u、v两个方向复用；
- $(\boldsymbol{P})$ 是4×4的控制点网格，对应两个参数方向各4个控制点。

6.3 曲面的核心特性

1. **局部控制**：修改任意一个控制点，只会影响它周围一小片曲面区域，不会牵动整张曲面；

2. **天然光滑拼接**：均匀三次B样条曲面在片与片的拼接处，天然满足C^2连续（位置、一阶偏导、二阶偏导都连续）；
3. **凸包性质**：曲面片完全落在对应控制点网格的凸包内部。

第7节 样条的推广：节点向量的一般理论（对应第67-71页）

7.1 节点与节点向量的严格定义

设U是一组包含$(m+1)$个非递减实数的序列：

$$U = u_0, u_1, u_2, ..., u_m$$

- 每个数值u_i称为一个**节点（Knot）**；
- 整个序列U称为**节点向量（Knot Vector）**；
- 半开区间 $[u_i, u_{i+1})$ 称为第i个**节点区间**。

> 注意：节点序列只要求“非递减”，不要求“严格递增”，因此允许相邻节点值相等。

7.2 重节点的定义

如果一个节点值u_i连续出现k次，即满足：

$$u_i = u_{i+1} = ... = u_{i+k-1}$$

且$(k>1)$，就称u_i是一个**重数为k的多重节点**，记作$u_i(k)$；只出现一次的节点称为**简单节点**。

7.3 均匀节点与非均匀节点

- **均匀节点序列**：所有相邻节点的间距都相等，即对任意$0 \le i \le m-1$，$u_{i+1} - u_i$为常数。
  例：$0, 1, 2, 3, 4, 5$ 是均匀节点，间距恒为1。
- **非均匀节点序列**：相邻节点间距不全相等，称为非均匀节点序列。
  例：$0, 0.2, 1, 2.5, 3$ 是非均匀节点。

7.4 非均匀节点的核心作用

（1）调整局部控制精度

节点间距决定了基函数的分布密度：

- 节点越密集（间距越小）：参数变化相同时，曲线的几何变化越小，控制点的控制精度越高；
- 节点越稀疏（间距越大）：控制精度越低，但曲线过渡越平缓。

工程中可以在细节丰富的区域加密节点，在平缓区域放宽节点，兼顾造型精度与计算效率。

（2）重节点控制曲线连续性

核心规则：**节点每重复1次，曲线在该位置的连续性就降低1阶**。

7.5 重节点降低连续性的原理推导

该结论可以从B样条基函数的递归起源严格推导：

1. **底层间断性**：0阶基函数是分段常数，在每个节点处都存在函数值跳跃，即每个节点天然自带1阶间断属性。
2. **递推＝平滑**：每进行一次递推升阶（从$p-1$阶到p阶），相当于对低阶基函数做一次线性加权过渡，等效于一次平滑，光滑性提升1阶。
   因此p阶基函数在简单节点处，可以抹平$p-1$阶间断，最终保留$p-1$阶连续导数，即C^{p-1}连续。
3. **重节点＝间断叠加**：节点重复k次，相当于同一个参数位置上叠加了k个节点的间断属性，累积了k阶间断。
   因此最终的连续阶数为：

   $$\text{连续阶数}=p-k$$

   即p阶B样条在k重节点处，连续性为C^{p-k}。

实例验证（三次B样条，$(p=3)$）

- 简单节点（$(k=1)$）：连续阶数$=3-1=2\ \rightarrow C^2$连续；
- 2重节点（$(k=2)$）：连续阶数$=3-2=1\ \rightarrow C^1$连续，二阶导数跳变；
- 3重节点（$(k=3)$）：连续阶数$=3-3=0\ \rightarrow C^0$连续，仅位置连续，出现尖点；
- 4重节点（$(k=4)$）：连续阶数$=3-4=-1\ \rightarrow$ 函数本身不连续，曲线断开。

第8节 节点向量的类型与端点插值（对应第72-73页）

8.1 节点间距对基函数权重的影响

节点间距直接改变基函数的形态与权重分布：

- 均匀节点：基函数形状对称，权重均匀分布，控制点的影响范围对称；
- 非均匀节点：节点间距宽的区域，基函数被拉宽，峰值降低，控制点影响更分散；节点间距窄的区域，基函数被压缩，峰值升高，控制点影响更集中。

8.2 开节点向量与闭节点向量

- **开节点向量（Open）**：首尾节点都只出现1次，没有重复。
  此时首尾位置的基函数还处于渐变阶段，端点处权重达不到1，因此曲线不会精确经过首尾控制点。
- **闭节点向量（Closed）**：将节点向量的第一个和最后一个节点值，各重复 $(k+1)$ 次（k为B样条的阶数）。
  例：二次B样条（$(k=2)$）闭节点：$0,0,0,1,1,1$；
  三次B样条（$(k=3)$）闭节点：$0,0,0,0,1,1,1,1$。

8.3 闭节点实现端点插值的原理

当首节点重复$(k+1)$次时，在参数起点$t=u_0$处，只有第一个基函数的值为1，其余基函数的值全为0，因此曲线在起点处精确等于第一个控制点$(\boldsymbol{P})_0$；终点同理精确等于最后一个控制点$(\boldsymbol{P})_n$。

闭节点向量是工业建模的标准设置：既保留了中间段的局部控制和高阶光滑性，又能固定端点位置，方便与其他几何体拼接。

第9节 NURBS曲线（对应第74-75页）

NURBS全称为**Non- Uniform Rational B- Spline（非均匀有理B样条）**，是B样条的高阶扩展，也是当前工业CAD、几何建模领域的标准曲线表示方法。

9.1 核心扩展：控制点权重

NURBS在普通非均匀B样条的基础上，给每个控制点$(\boldsymbol{P})_i$额外增加了一个**权重参数$\boldsymbol{w_i}$**，用于精细控制该控制点对曲线的“牵引强度”。

9.2 有理基函数与曲线表达式

NURBS的基函数为有理分式形式，第i个控制点的有理基函数为：

$$R_{i,k}(t)=\frac{w_i\cdot N_{i,k}(t)}{\sum_{j=0}^{n} w_j\cdot N_{j,k}(t)}$$

其中$N_{i,k}(t)$为k次非均匀B样条基函数，分母为所有控制点的加权基函数之和，起到归一化作用。

NURBS曲线的完整表达式为：

$$(\boldsymbol{P})(t)=\sum_{i=0}^{n} R_{i,k}(t)\cdot(\boldsymbol{P})_i$$

9.3 权重的几何意义

权重w_i可以理解为控制点的“吸引力”：

- w_i越大，曲线越向对应的控制点$(\boldsymbol{P})_i$靠近；
- w_i越小，曲线越远离对应的控制点$(\boldsymbol{P})_i$；
- 单独调整单个权重，即可微调曲线局部形状，造型灵活性远高于普通B样条。

9.4 与普通B样条的包含关系

当所有控制点的权重都等于1（$w_i\equiv1$）时，分母退化为$\sum_{j=0}^{n} N_{j,k}(t)=1$（B样条基函数的单位分解性质），此时有理基函数等于普通B样条基函数，NURBS曲线退化为普通非均匀B样条曲线。

即：**普通非均匀B样条是NURBS在所有权重为1时的特例**。

9.5 齐次坐标视角下的NURBS推导

NURBS可以通过“四维齐次空间的普通B样条做透视投影”推导而来，这是工程中NURBS算法的核心实现思路。

步骤1：控制点升维到齐次空间

对于三维空间的控制点$(\boldsymbol{P})\,i=(x_i,\,y_i,\,z_i)$，乘以权重$w_i$，得到对应的**齐次控制点**：

$$(\boldsymbol{P})\,\mathrm{iH}=(x_i\cdot w_i,\,y_i\cdot w_i,\,z_i\cdot w_i,\,w_i)$$

即将三维点升维到四维齐次空间，第四个分量为权重w_i。

步骤2：齐次空间中做普通B样条插值

在四维齐次空间中，对齐次控制点做k次非均匀B样条插值，得到齐次空间中的B样条曲线：

$$(\boldsymbol{P})\,H(t)=\sum\, i=0^{n}\,N_{i,k}(t)\cdot(\boldsymbol{P})\,\mathrm{iH}$$

展开为四个分量：

$$\left\{\; x_H(t)=\sum i=0^{n}\,w_i\,x_i\cdot N_{i,k}(t)\;y_H(t)=\sum_{i=0}^{n}\,w_i\,y_i\cdot N_{i,k}(t)\;z_H(t)=\sum_{i=0}^{n}\,w_i\,z_i\cdot N_{i,k}(t)\;\omega(t)=\sum_{i=0}^{n}\,w_i\cdot N_{i,k}(t)\right.$$

步骤3：透视投影得到三维NURBS

将齐次坐标做透视投影（所有分量除以第四个分量$\omega(t)$），即可得到三维空间中的NURBS曲线：

$$x(t)=\frac{x_H(t)}{\omega(t)},\quad y(t)=\frac{y_H(t)}{\omega(t)},\quad z(t)=\frac{z_H(t)}{\omega(t)}$$

该视角的工程意义：NURBS的所有计算，都可以先转化为齐次空间中的普通B样条计算，最后再做投影，极大简化了算法实现。

LESSON 5.2 反走样

第1章 走样现象与本质

1.1 什么是走样

走样（Aliasing，也常称锯齿、混淆）是数字图像与计算机图形学中最基础的失真现象。我们日常看到的图片边缘“锯齿”、远处纹理的闪烁杂纹、直线显示的阶梯感，本质都是走样。

从现象上看：连续平滑的几何图形（直线、曲线、物体轮廓），显示在由离散像素组成的屏幕上时，边缘会呈现出阶梯状的锯齿；精细的纹理在缩小显示时，会出现错乱的花纹甚至闪烁。

1.2 走样的直观成因

走样的根源可以用一句话总结：**离散的采样点，无法完整捕捉连续信号的快速变化，本质是采样密度不足**。

第2章 频域分析基础：傅里叶变换

要从本质上理解走样与抗锯齿，仅靠空间域的直观认知远远不够，我们需要引入**频域分析**工具——傅里叶变换。它能把我们熟悉的“空间/时间域”信号，转换到“频率域”去观察，从更深层的规律解释走样的成因。

2.1 一维傅里叶变换：从时域到频域

核心思想

任何一个复杂的连续信号，都可以拆解成**无数个不同频率的正弦/余弦信号的加权叠加**。每个正弦分量的幅度，对应原信号中该频率成分的强度；相位对应该成分的位置偏移。

傅里叶变换就是实现“时域→频域”转换的数学工具，对应的逆变换可以从频域无失真地还原回时域信号。

数学定义

设连续时间信号为 $g(t)$)，其傅里叶正变换（时域→频域）为:

$$G(f)=\int_{-\infty}^{\infty} g(t)\ e^{-2\pi itf}\ \mathrm{dt},\quad \forall f\in\mathbb{R}$$

其中 f 是频率， 单位为Hz； $e^{-2\pi itf}$ 是复指数基函数， 根据欧拉公式可展开为 $\cos(2\pi ft)-\mathrm{isin}(2\pi ft)$，也就是正弦+余弦的组合。

对应的傅里叶逆变换（频域→时域）为:

$$g(t)=\int_{-\infty}^{\infty} G(f)\ e^{2\pi itf}\ \mathrm{df}$$

它的物理意义是：把所有频率的正弦分量按幅度加权叠加，还原出原始信号。

2.2 空间频率：从一维信号到二维图像

对于图像这类二维空间信号，我们只需要把“时间”替换成“空间坐标”，原理完全一致。

- 时间频率：单位时间内振动的周期数;
- 空间频率：单位长度内，亮度/颜色变化的周期数。

举个直观例子：一条横向的灰度条纹，从左到右完成一次“黑→白→黑”的明暗循环，就是1个空间周期；单位长度内循环越多，条纹越密，空间频率就越高。

推广到二维图像，频率是一对数值 (f_x, f_y):

- f_x：水平方向的空间频率，数值越大，水平方向明暗变化越细密;
- f_y：竖直方向的空间频率，数值越大，竖直方向明暗变化越细密。

2.3 二维傅里叶分解：图像的基底展开

二维傅里叶变换的核心逻辑是：**任意一张复杂图像，都可以拆解成无数个二维正弦基底的加权叠加**。这些基底就是傅里叶变换的“基函数”，和线性代数中的基向量作用一致——用一组最简单的标准单元，组合出任意复杂的结果。

典型基底与视觉形态

每一组频率 (f_x, f_y) 都对应一个最简单的明暗图案，也就是空间基底:

1. **频率 (0, 0)：直流分量**
   x、y方向都没有明暗变化，整个画面亮度均匀，对应图像的整体平均亮度。在频谱图上对应正中心的点，能量通常最强。
2. **频率 (1, 0)：竖直条纹**
   x方向有1个完整的明暗周期，y方向完全没有变化，呈现为竖直的明暗条纹。在频谱图上对应水平轴、中心左右对称的两个点。
3. **频率 (0, 1)：水平条纹**
   y方向有1个完整的明暗周期，x方向完全没有变化，呈现为水平的明暗条纹。在频谱图上对应竖直轴、中心上下对称的两个点。
4. **频率 (1, 1)：2×2棋盘格**
   x、y方向各有1个明暗周期，两个方向的变化叠加，形成2×2的棋盘格图案。在频谱图上对应四个对角方向、关于中心对称的四个点。

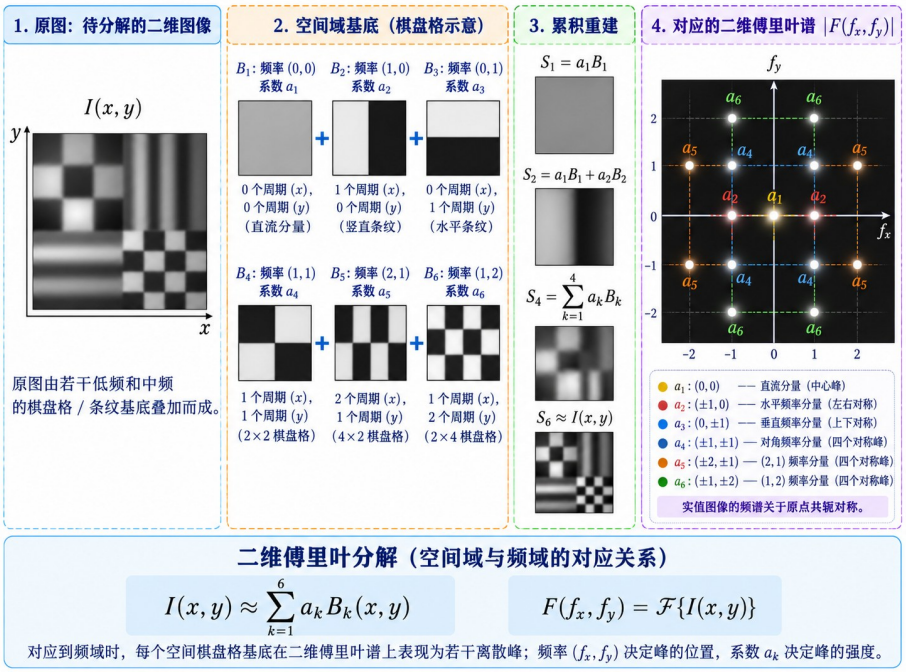
更高阶的频率（如(2,1)、(1,2)）对应更细密的棋盘格，在频谱图上位置更靠外（离中心更远）。

图像的重建过程

从直流分量开始，逐次叠加更高频的基底，就能逐步还原出原图：

$$S_1 = a_1 B_1, \quad S_2 = a_1 B_1 + a_2 B_2, \quad S_4 = \sum_{k=1}^{4} a_k B_k, \quad \ldots \quad S_n \approx I(x,y)$$

其中 B_k 是第k个频率基底，a_k 是对应的权重系数。自然图像的能量大多集中在低频和中频，仅需少量高频分量补充细节，就能较好地还原原图。



2.4 二维频谱的性质

1. **一一对应性**：频谱图上的每一个点，对应一个空间基底；点的位置对应频率高低，点的亮度对应该频率分量的强度（权重系数）。
2. **共轭对称性**：实值图像的频谱关于原点中心对称，因此频谱图永远是左右、上下对称的。
3. **周期重复性**：离散图像的频谱会沿 f_x 和 f_y 两个方向，以采样频率 f_s 为周期无限平铺重复。我们通常只显示中心的一个基本周期（奈奎斯特区间）。

第3章 采样定理与走样的数学本质

3.1 离散采样的数学表达

离散采样不是简单的“跳过信息”，它有严格的数学定义：**用采样冲激串（梳状函数）与原连续信号相乘，只保留冲激位置的信号值。**

设原连续信号为 $x_a(t)$，采样周期为 T_s（相邻两个采样点的时间间隔），则采样频率 $f_s = \frac{1}{T_s}$。

采样冲激串 $p(t)$) 是一系列间隔为 T_s 的冲激函数：

$$p(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s)$$

其中 $\delta(t)$ 是单位冲激函数，只在t=0处非零，积分面积为1。

采样后的离散信号 $x_s(t)$ 就是原信号与冲激串的乘积：

$$x_s(t) = x_a(t) \cdot p(t)$$

相乘后，只有冲激所在的采样时刻保留了原信号的数值，其余位置都为0，这就是离散采样的数学本质。

3.2 采样的频域效应：频谱周期延拓（完整推导）

傅里叶变换有一条核心对偶性质：**时域的乘法，等价于频域的卷积**。
即：若 $x_s(t) = x_a(t) \cdot p(t)$ ），则它们的频谱满足

$$X_s(f) = X_a(f) \ P(f)$$

其中 表示卷积运算，$X_a(f)$ 是原信号的频谱，$P(f)$ ）是采样冲激串的频谱。

第一步：采样冲激串的频谱

这是傅里叶变换的经典结论：**时域中间隔为 T_s 的冲激串，其傅里叶变换仍然是冲激串，频域间隔为 $f_s = \frac{1}{T_s}$**。

$$P(f) = \frac{1}{T_s} \sum_{k=-\infty}^{\infty} \delta(f - k f_s)$$

简单来说：时域梳状函数，变换到频域还是梳状函数，时间间隔和频率间隔互为倒数。

第二步：卷积的物理意义

卷积有一个非常直观的性质：**任何函数与冲激函数卷积，等价于把这个函数平移到冲激所在的位置**。

将原信号频谱 $X_a(f)$ 与冲激串频谱 $P(f)$ ）做卷积， 就相当于把原频谱分别平移到 $0, \pm f_s, \pm 2f_s, \pm 3f_s ...$ 的位置，然后全部叠加起来：

$$X_s(f) = \frac{1}{T_s} \sum_{k=-\infty}^{\infty} X_a(f - k f_s)$$

核心结论

对连续信号以采样频率 f_s 做离散采样，**等价于在频域中把原信号的频谱以 f_s 为间隔，向正负方向无限复制，产生无数个完全相同的频谱副本**。

3.3 奈奎斯特-香农采样定理

临界条件推导

混叠（走样）的本质，就是**相邻的频谱副本发生了重叠**。要让采样后信号不失真，就必须保证所有频谱副本互不重叠。

假设原信号是带限信号，最高频率为 $f_{\max}$，即原频谱只在 $[-f_{\max}, f_{\max}]$ 内非零。

- 原始频谱的最右端（最高正频率）是 $f_{\max}$；
- 右侧第一个频谱副本的最左端是 $f_s - f_{\max}$（副本整体向右平移了 f_s）。

要让两者不重叠，必须满足：右侧副本的左端 $\geq$ 原频谱的右端，即

$$f_s - f_{\max} \geq f_{\max}$$

整理得：

$$f_s \geq 2f_{\max}$$

定理正式表述

奈奎斯特-香农采样定理：要想从离散采样信号中无失真地还原原始连续信号，采样频率必须大于等于信号最高频率的2倍。

其中 $\frac{f_s}{2}$ 被称为**奈奎斯特频率**，它是采样系统能够准确还原的最高信号频率，是频谱副本刚好"肩并肩"不重叠的临界值。

Aliasing... Or How Sampling Distorts Signals - YouTube

https://chat.deepseek.com/a/chat/s/576a977f-4772-4689-888f-e5a9ac576675

3.4 混叠的频率折叠效应

当信号频率超过奈奎斯特频率（$f > f_s/2$）时，频谱副本发生重叠，高频分量会被"折叠"到低频段，呈现为一个虚假的低频信号，称为**混叠频率**。

举个典型例子：若真实信号频率 $f = 1.25f_s$，远高于奈奎斯特频率 $0.5f_s$，采样后等效的混叠频率为 $f_{\text{alias}} = 0.25f_s$。

也就是说，一个频率为1.25倍采样率的连续信号，离散采样后得到的采样点，和一个0.25倍采样率的低频信号完全一致（仅相位可能有差异）。人眼会将其识别为低频信号，这就是走样的直观表现——高频细节被错误地呈现为低频杂纹。如果有多个不同的高频分量都折叠到同一个低频位置，就会互相干扰，再也无法区分原始的真实频率。

3.5 二维图像的采样与走样

以上结论可以直接推广到二维图像：

- 单方向的1像素条纹，对应频率 $(f_s/2, 0)$，刚好达到x方向的奈奎斯特极限；
- 全像素棋盘格，对应频率 $(f_s/2, f_s/2)$，两个方向都达到奈奎斯特极限。

如果计算频谱到中心的距离（频率半径），棋盘格的等效频率更高：

$$\text{角点半径} = \sqrt{\left(\frac{f_s}{2}\right)^2 + \left(\frac{f_s}{2}\right)^2} = \frac{f_s}{\sqrt{2}} = \sqrt{\left(\frac{f_s}{2}\right)^2 + \left(\frac{f_s}{2}\right)^2} = \frac{f_s}{\sqrt{2}}$$

这也是斜向细密纹理更容易出现锯齿、走样更严重的原因——它的等效空间频率更高，更容易突破奈奎斯特极限。

第4章 抗锯齿的核心机制：低通滤波

4.1 抗锯齿的两种等价思路

既然走样的根源是信号中存在高于奈奎斯特频率的分量，采样后频谱重叠失真，那么抗锯齿的核心思想就非常明确：**采样前，把信号中高于奈奎斯特频率的分量滤除掉**，也就是低通滤波。

工程上有两种等价的实现思路，数学效果完全一致：

1. **先滤波，再采样**：先对原始连续信号做低通模糊，滤除高频分量，再用目标分辨率采样。
2. **先超采样，再滤波降采样**：先用远高于目标的分辨率采样（超采样），再通过低通滤波平均，降回目标分辨率。

4.2 面积采样：光栅化中的低通实现

在绘制直线、三角形这类几何图形的光栅化场景中，最直观的低通实现就是**面积采样**。

普通的点采样是“二值判断”：只看像素中心是否在图形内部，结果要么全亮、要么全暗，因此边缘是锐利的锯齿。

面积采样是“连续取值”：计算**图形与像素方块的相交面积**，用面积占比来决定像素的明暗程度，本质就是用均匀的盒式滤波做低通。

面积采样分为两种：

- **等权面积采样**：像素的明暗强度只由相交面积决定，面积相等则强度相等，对应盒式滤波。
- **加权面积采样**：像素的明暗 = 相交面积 × 权重函数 $f(d)$ ，其中 d 是像素中心到图形边缘的距离。权重函数就是低通滤波核，通常离中心越近权重越高，边缘过渡更自然。

4.3 常见的低通滤波核

设计一个合格的低通滤波核，需要满足三个基本条件：

1. 有限支撑：只在有限区域内非零；
2. 中心衰减：数值从中心向边缘逐渐降低，距离越远权重越小；
3. 归一化：整个滤波核的积分（总面积）为1，保证滤波后图像整体亮度不变。

常见的滤波核类型：

- **Box（盒式滤波）**：窗口内权重均匀相等，最简单，对应等权面积采样；
- **Tent/Bartlet（三角滤波）**：权重从中心向边缘线性衰减，效果比盒式滤波更平滑；
- **Gaussian（高斯滤波）**：权重服从高斯分布，衰减平滑，是最常用的高质量滤波核；
- **Cone（锥形滤波）**：二维三角滤波，呈圆锥状分布。

4.4 基于距离函数的滤波实现

除了计算相交面积，我们还可以直接用**像素中心到图形边缘的距离**代入滤波核函数，直接计算像素的明暗值，不需要先超采样再降采样。

比如绘制直线时，只要算出像素中心到直线的垂直距离，代入高斯滤波函数，就能得到该像素的灰度值。这种“先滤波后采样”的实现，最终效果和“先超采样再滤波降采样”几乎完全一致，数学上等价。

第5章 纹理重采样与Mipmap技术

5.1 图像映射中的走样问题

走样不只出现在绘制几何图形时，在对图像做缩放、变形、纹理映射的过程中，同样会出现走样，称为图像重采样走样。

当我们把高分辨率的源纹理映射到3D模型表面时，目标像素和源像素不是一一对应的：

- 距离相机越远，屏幕上一个像素覆盖的纹理范围越大，相当于用很低的采样率去采样源纹理的高频细节；
- 采样频率不满足奈奎斯特条件，就会发生欠采样走样，表现为远处纹理闪烁、出现错乱杂纹。

5.2 Mipmap：预滤波图像金字塔

Mipmap（也叫多级渐远纹理，MIP映射）是纹理抗锯齿的标准解决方案，核心思想是**预滤波 + 图像金字塔**。

核心原理

提前为原始纹理生成一套**图像金字塔**：

- 最底层是原始最高分辨率纹理；
- 往上每一级，都先对下一级做低通滤波，滤除该分辨率下会引发走样的高频分量，再将宽高都缩小为原来的1/2（面积缩小为1/4）；
- 依次生成1/2、1/4、1/8.....分辨率的各级子纹理，直到最小的1×1像素。

因为每一级都提前做好了低通滤波，所以该分辨率下的信号都满足奈奎斯特条件，采样时不会产生走样。

使用方式

渲染时，根据屏幕上该纹理区域的实际分辨率（一个像素覆盖多少纹理区域），自动选择金字塔中最匹配的层级来采样。实际工程中通常会选取相邻的两个层级，再在两级之间做线性插值，让过渡更平滑。

5.3 Mipmap的存储开销计算

Mipmap的额外存储开销非常小，我们可以通过等比数列求和精确计算：

假设原始纹理的面积为1，那么各级子纹理的面积分别是 $1, \frac{1}{4}, \frac{1}{16}, \frac{1}{64}, ...$，这是公比为 $\frac{1}{4}$ 的无穷等比数列。

根据等比数列求和公式 $S = \frac{a_1}{1-q}$，总存储面积为：

$$S = 1 + \frac{1}{4} + \frac{1}{16} + ... = \frac{1}{1-\frac{1}{4}} = \frac{4}{3}$$

也就是说，整套Mipmap的总存储量是原始纹理的 $\frac{4}{3}$ 倍，**仅额外增加了1/3的存储开销**，性价比极高。

5.4 透视场景下的层级计算

在透视投影中，屏幕上均匀的像素步长，对应到3D模型表面并不是均匀的：距离相机越远，一个屏幕像素覆盖的纹理范围越大。因此渲染时需要实时计算当前像素对应的纹理采样率，匹配到最合适的Mipmap层级，才能刚好避免走样。

第6章 几何光栅化抗锯齿技术

针对三角形、直线等几何图形光栅化时的边缘锯齿，有两类主流技术：超采样抗锯齿（SSAA）和多重采样抗锯齿（MSAA）。

6.1 超采样抗锯齿（SSAA）

SSAA是最直接、画质最高的抗锯齿技术，本质就是"先超采样、再滤波降采样"思路的完整实现。

实现原理

用高于屏幕原生分辨率的规格渲染画面，比如2倍SSAA就是用2倍宽高的分辨率渲染，屏幕上1个像素对应高分辨率图像里2×2共4个采样点。**每个采样点都会完整执行一遍着色计算**（颜色、光照、纹理等），最后对每个像素对应的所有子采样点做颜色平均，得到最终屏幕像素的颜色。

采样点布局

除了最基础的N×N规则网格，还有很多优化的采样点布局，可以用更少的采样量达到更好的效果：

- Quincunx（五点梅花形）：中心1个+四角4个，共5个采样点，效果接近2×2网格；
- 棋盘格布局、车形布局等，通过优化空间分布提升抗锯齿效率。

SSAA的优点是画质最高，能同时解决边缘锯齿和内部着色走样；缺点是计算量极大，着色成本随采样数线性增长。

6.2 多重采样抗锯齿（MSAA）

MSAA是实际游戏和3D渲染中最常用的几何抗锯齿技术，是SSAA的低成本优化方案。

核心思想

MSAA的核心优化是：**只对「几何覆盖情况」做多重采样，颜色只计算一次**。

每个像素内布置多个子采样点（通常是2×2共4个），但流程和SSAA完全不同：

1. **覆盖检测**：统计有多少个子采样点落在三角形内部，得到三角形对该像素的覆盖率；
2. **单次着色**：整个像素只计算一次三角形的颜色（通常取像素中心或覆盖区域中心）；
3. **颜色混合**：最终像素颜色 = 覆盖率 × 三角形颜色 + (1-覆盖率) × 背景颜色。

举个例子：4个子采样点有3个被三角形覆盖，最终颜色 = 3/4×三角形颜色 + 1/4×背景色。

6.3 SSAA与MSAA的对比

维度	SSAA	MSAA
采样方式	每个子采样点都完整着色	仅覆盖检测多重采样，颜色只算一次
画质	最高，边缘与内部细节都能平滑	边缘效果接近SSAA，内部细节无优化
性能开销	极高，着色成本随采样数线性增长	低，仅边界像素增加覆盖检测开销
解决的走样类型	几何走样 + 着色走样	仅几何走样

6.4 几何走样与着色走样的区分

理解MSAA的能力边界，需要区分两类不同的走样：

1. **几何走样**：物体轮廓、三角形边界的锯齿，由几何边缘与像素网格不匹配导致。MSAA通过覆盖率计算可以很好地平滑这类走样。
2. **着色走样**：三角形内部的高频细节，比如镜面反射、精细纹理、高频光照变化导致的走样。这类走样源于颜色信号本身的频率超过奈奎斯特频率，MSAA只计算一次颜色，无法平滑这类问题，需要靠纹理滤波、着色超采样来解决。

全书知识框架总结

1. **现象层**：走样就是锯齿、杂纹、闪烁，本质是离散采样密度不足。
2. **原理层**： 采样等价于频域周期复制； 超过奈奎斯特频率就会混叠； 奈奎斯特采样定理是 $f_s \geq 2f_{\max}$。
3. **方法层**：所有抗锯齿本质都是低通滤波，分为先滤波后采样、先超采样后降采样两类。
4. **应用层**：纹理抗锯齿用Mipmap（预滤波金字塔）；几何边缘抗锯齿常用MSAA（低成本覆盖采样），高质量用SSAA（全采样）。

LESSON 6 2026-09-29 第5-6节

一、课程导入与安排

从本次课程开始，我们进入3D几何部分的学习，核心内容是几何。几何是图形学的基础环节，后续课程会基于几何进一步讲解动画与渲染相关内容。
之前的课程以基础内容为主，建议大家利用十一假期完成课程的Lab练习。在此提醒大家：十一假期结束后的周二第一节课，将进行第一次区间测试，时长为一节课（50分钟，不含课间），第二节课正常授课。

二、几何概述

2.1 几何的词源与研究对象

几何的英文单词源自希腊语，本义是土地测量。在图形学领域，几何关注物体的空间形态，具体研究长度、角度等空间量化特征。
图形学中几何表达的整体逻辑，通常从离散表达入手，再研究如何通过离散形式体现连续的几何内容，包括离散点之间的插值方法。同时，图形学与微分学关联紧密，我们也需要研究在离散表达上如何完成微分等相关运算。几何表达是图形学非常重要的组成部分。

2.2 经典几何模型：犹他茶壶

犹他茶壶是图形学的经典模型， 由Martin Newell创作。 在贝塞尔曲线、 曲面的定义成熟之后，Newell以家中的茶壶为参照，通过拍摄照片、人工标定点位，创建了首个由人工制作的贝塞尔曲面茶壶模型。
最初的茶壶模型满足C1连续性，而真实物体的表面具备更高的连续性。围绕该模型的几何表达，图形学领域产生了大量相关研究，是图形学发展中值得了解的经典工作。
人工标定几何表达的方式难以规模化，也难以标准化地生成3D模型数据。当前行业内3D数据的稀缺性远高于图片、视频等2D数据，但图形学领域很早就开始使用采集工具构造各类几何数据。

2.3 图形学经典数据集：斯坦福四大模型

图形学领域有四个经典的测试模型，被称为“四大神兽”，最初都诞生于斯坦福实验室。研究人员利用激光扫描技术获取物体的点云数据，再通过表面重建构建三角网格，形成了图形学领域最早的一批基础模型。
四个模型分别是： 斯坦福兔子（Bunny）、 中国龙（Dragon）、 犰狳（Armadillo）、 快乐佛陀（Happy Buddha）。 受限于当时的激光扫描技术， 这些模型的体积都比较小， 但扫描精度很高，其中犰狳模型达到数万面片，佛陀模型的面片数也非常高。
这些模型均开源在斯坦福实验室的官方网站上， 网站上有其诞生背景的详细介绍， 也频繁出现在各类图形学论文的实验结果中。

不同模型的适用场景各有不同：

- 物理仿真领域常用斯坦福兔子做各类测试，比如切割、碾压等形变实验，被戏称为“兔子的100种死法”。
- 渲染领域更常用龙和快乐佛陀模型。行业内的一种说法是，龙和佛陀带有一定的宗教、文化象征意义，不适合用于切割、大规模形变等测试，因此更多被用于渲染实验。
- 兔子和犰狳则更多出现在各类极端测试中。通常以兔子为测试对象的算法成熟度相对较低，因为兔子的顶点数远少于犰狳；如果算法能在犰狳模型上稳定运行，说明算法的效率已经达到较高水平。

三、几何的表达分类

几何的表达方式和曲线类似，分为显式表达与隐式表达两类。本课程主要围绕显式表达展开，仅在曲面重建等部分应用中涉及隐式表达。两类表达各有优劣，显式表达更便捷、效率更高，相关技术发展也更成熟。

3.1 显式表达

显式表达是图形学中应用最广泛的几何表达方式，核心类型包括：

- **多边形网格（Polygon Mesh）**：最常用的是三角网格；在很多追求平滑性的领域中，四边形网格的应用也十分广泛，这也是本课程重点讲解的内容。
- **点云**：主要通过激光扫描等方式获取。

显式表达本质是用离散的形式表达连续的几何体，为了提升表达精度，衍生出了细分曲面等一系列技术。显式表达多用于几何制造、物理仿真等场景，这类场景对制造精度、仿真效率的要求较高，更适配显式表达的特性。

3.2 隐式表达

隐式表达通常以方程的形式呈现，需要通过解方程才能得到表面上点的位置，以此描述几何体的表面。
隐式表达非常适合描述细节丰富、形变频繁的对象，比如流体表面。很难用精确的网格表达流体的动态表面，但可以用水平集函数，即形如$F(x,y,z)=0$的方程来表示，方程等于0的等值面就构成了水的表面。这类表达在流体等动态场景中应用较多。
本课程对隐式表达的讲解相对较少，因为其处理逻辑更为复杂。

四、点云与激光扫描技术

点云是最基础的几何表达形式之一，前文提到的斯坦福四大模型，最初都是通过激光扫描获得的点云数据。
激光扫描的基本原理：扫描设备向空间中可控地发射激光，激光遇到物体表面后发生反射，设备接收反射光束，通过发射与接收的时间差计算物体与设备之间的距离，进而生成空间中对应位置的点，完成表面点云的采集。
早期的激光扫描设备可以设计为封闭的扫描舱，从四周向中心发射激光，中间放置旋转的物体，即可完成物体整个表面的点云扫描，这是最早实现高精度点云采集的技术之一。

激光扫描技术至今仍有很高的实用价值，在自动驾驶领域发展迅速。车载激光雷达通过多个传感器全方位采集周围环境的点云，硬件层面不断提升传感器的精度与抗干扰能力，软硬件协同发展，也为3D几何数据的生产提供了便利。

五、多边形网格基础

多边形网格是图形学的主流几何表达，其中最主流的是三角网格和四边形网格，由顶点、边、面三种基本元素构成。
多边形网格之所以流行，是因为其配套了便捷高效的数据结构，能够很好地支持模型的编辑、控制与各类计算。比如修改模型表面的控制点后，需要同步更新对应的面、法向量等信息，这就需要高效的数据结构来支持快速检索，比如查询一个顶点周围的所有面、一条边对应的对角顶点等。这也是本课程后续要讲解的核心内容之一。

两类网格的适用场景：

- 表面细节丰富的模型，更多使用三角网格。
- 对平滑度要求高的模型，更多使用四边形网格，或是四边形与三角网格混合的形式。

图形学中需要表达的几何体十分抽象复杂，三角网格甚至可以表达非流形的几何体，比如物体表面突出的片状结构，或是克莱因瓶这类数学概念上的几何体，能够适配各类应用的不同需求。

除了基础的三角、四边形网格，网格相关的技术还包括细分网格、网格参数化等，后续会逐一讲解，帮助大家理解网格如何实现离散存储，以及如何在离散格式上满足连续性相关的操作需求。

- **细分网格**：作用是让模型表面更加精细。
- **网格参数化（Mesh Parameterization）**：将三维的表面网格映射到二维空间，也就是UV映射，是纹理展开等应用的基础，核心工作是建立3D空间与2D空间的映射关系。

六、三角网格的存储方式

6.1 OBJ格式

OBJ是一种常见的三角网格存储格式，属于纯文本格式，可读性强，可以直接用文本编辑器打开。
OBJ的存储逻辑：

1. 先存储模型所有顶点的坐标，形成顶点列表。
2. 再存储所有的面，每个面通过顶点列表的下标来指定组成该面的三个顶点。

通过“顶点坐标+面的索引”的结构，即可完整描述一个三角网格。
OBJ是存储层面非常通用的格式，但它并不是最高效的运算结构，实际应用中通常需要基于OBJ构建更高效的数据结构，来支持各类检索操作。OBJ是构建其他高级数据结构的基础，具有很高的实用价值。

6.2 Triangle Soup（三角汤）

最原始、最简单的三角网格存储方案，直接存储每个三角形的三个顶点坐标。
由于封闭几何体有内外之分，三角形的顶点需要按照固定顺序存储，通过右手定则可以判断面的法向，区分几何体的内外。
这种存储方式的缺点非常明显：数据高度冗余，编辑操作效率极低。修改一个顶点的坐标，需要遍历所有包含该顶点的三角形，逐一匹配、替换，操作十分繁琐，因此并不是理想的存储方式。

6.3 两种存储方式的对比

- Triangle Soup：结构简单，但存储冗余度极高，计算与编辑效率低。
- OBJ格式：存储冗余度低，是最精简的存储方式之一；修改顶点坐标时只需修改一次，支持基于索引的检索；但邻接查询等操作仍需要遍历搜索，计算效率有限，直观性也相对较弱。

6.4 邻接关系的预计算

如果需要频繁查询邻接面，可以基于OBJ格式做预计算，提前构建整个网格的邻接关系。
具体方法：

1. 展开所有三角形的边，为每条边标记所属的三角形下标，生成完整的边列表。
2. 对边列表按字典序排序，相同的边会排列在一起，由此可以识别出公共边。
3. 整理公共边对应的三角形，即可得到邻接三角形关系。

同理，如果要查询一个顶点的所有一阶邻接顶点，也可以遍历边列表，筛选出包含该顶点的所有边，进而得到邻接顶点。
这类操作的时间复杂度都与网格的规模（边数、顶点数、面片数）正相关，难以满足实时编辑与调整的需求。

七、半边数据结构（Half-Edge Data Structure）

半边数据结构是三角网格领域应用最广泛的数据结构之一，在几何处理领域已经沿用了数十年。
它的核心思想是：以边为核心连接所有拓扑信息。一条完整的边可以拆分为一对方向相反的半边，每条半边只属于一个面，同时存储其对应的孪生半边（另一个方向、属于相邻面的半边），以此支持高效的拓扑检索。

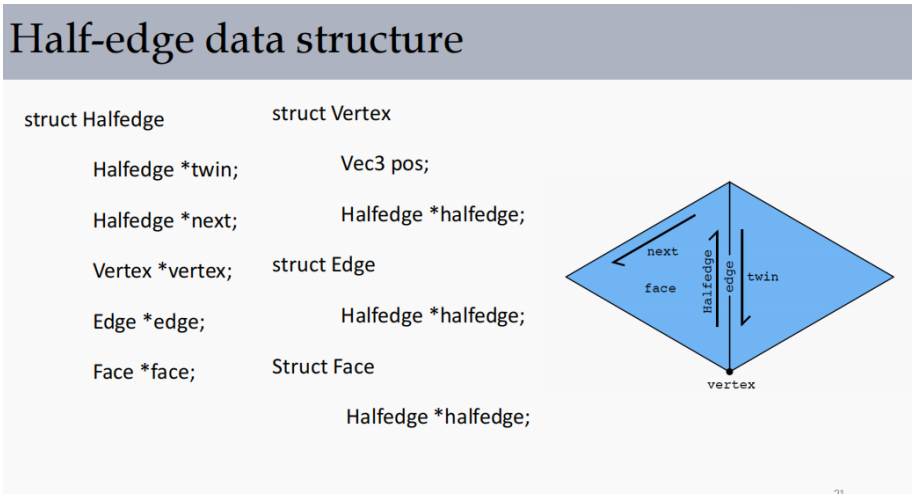
7.1 数据结构定义

半边数据结构包含三类核心元素：顶点、半边、面，各自的存储内容如下：

- **顶点（Vertex）**：存储自身的三维空间坐标(x, y, z)，以及从该顶点出发的任意一条半边。
- **半边（Half-Edge）**：存储四个核心信息：
    1. 起始顶点（origin）：这条半边从哪个顶点发出。
    2. 孪生半边（twin）：方向相反、共享同一条边的另一条半边。
    3. 下一条半边（next）：同一个面内，按逆时针方向排列的下一条半边。
    4. 所属面（face）：这条半边属于哪个面。

所有信息均以指针形式存储，其中`twin`和`next`是支持递归遍历的核心字段。如果存储空间充足，还可以额外存储前一条半边（previous），让后续操作更加灵活对称。

- **面（Face）**：存储属于该面的任意一条半边。
- **边（Edge）**：可选存储，一般存储对应边的任意一条半边即可。



7.2 拓扑遍历操作

半边数据结构的优势在于可以高效完成各类拓扑遍历，以下是常见的操作场景：

1. **遍历单个面的所有顶点**
   从该面的任意一条半边出发，递归访问`next`指针，即可按逆时针顺序遍历完该面的所有顶点；也可以反向顺时针遍历。
2. **遍历顶点的所有一阶邻接顶点**
   从该顶点存储的任意一条半边出发，通过“previous的twin”或是“twin的next”递归遍历，即可快速找到该顶点的所有一阶邻接顶点，完成邻域遍历。这类操作常用于网格平滑等算法，比如将顶点移动到其所有邻接点的平均位置，实现平滑效果。
3. **边交换（Edge Swap）**
   当网格中出现角度过小的锐角三角形时，会影响计算精度，容易产生数值误差。此时可以通过边交换操作优化网格拓扑：删除原有边，替换为另一条对角线，重构两个三角形的连接关系。
   这类操作常用于网格简化、网格优化等场景。

7.3 应用特点

半边数据结构存在大量数据冗余，因此实际存储中仍普遍使用OBJ等格式。标准的应用流程是：读取OBJ文件，根据文件中的三角面片信息构建半边数据结构，再基于半边结构完成后续的检索、编辑等几何操作。
半边数据结构不仅可以表达完整的流形表面网格，也可以表达任意连接关系的三角形或多边形，通用性很强。

八、面向渲染的专用网格结构

几何数据广泛应用于图形学的各类下游场景，不同的下游应用对数据结构的需求不同，统一的半边结构无法满足所有场景的高效使用需求。因此面向特定应用，衍生出了专用的高效数据结构。
以面向渲染的OpenGL为例，它定义了多种适配渲染管线的网格存储结构：

1. **独立三角形**：本质就是Triangle Soup结构，每个三角形独立存储三个顶点。
2. **三角带（Triangle Strip）**：按顺序存储一系列顶点，默认每相邻三个顶点构成一个三角形。通过共享顶点大幅压缩存储量，核心目标是节省显存，同时支持并行三角形绘制加速。
3. **三角扇（Triangle Fan）**：以第一个顶点为中心，其余顶点按顺序排列，默认中心顶点与每两个相邻顶点构成一个三角形，形成扇形的三角网格。

OpenGL允许组合使用这些结构来高效存储几何体，这类结构是面向渲染任务的专用高效数据结构，语义更抽象，牺牲了部分几何编辑的灵活性。
在几何处理领域，通常需要为下游应用定制对应的数据结构。而半边数据结构属于通用型的高效数据表达，是三角网格操作的基础结构。

九、四边形网格（Quad Mesh）

四边形网格也是非常常见的网格形式，它与数学上的参数曲面有很好的对应关系。比如球面可以用经纬度的球坐标表示，按照坐标分辨率离散化后，自然就形成了四边形网格。
四边形网格的网格角度通常接近90度，表面更平滑，因此在很多领域都有广泛应用。比如人体、人脸这类对表面平滑性要求高的模型，更多使用四边形网格。

四边形网格最核心的应用场景是纹理展开，也就是后续要讲的网格参数化。四边形网格的平滑性更有利于将3D表面展开为2D平面，建立一一对应的映射关系。

需要注意的是，四边形本身不是唯一确定的几何形状：它可以拆分为两个三角形进行渲染，也可以通过双线性插值得到连续的曲面。最终的呈现方式由渲染软件等下游工具决定，比如OpenGL通常将四边形拆分为两个三角形分别渲染。因此四边形网格的空间表达形式，需要人为指定。

十、多边形网格的优缺点

- **优势**：作为一种表面表达方式，结构简单，完全支撑了图形学渲染及下游各领域的应用，是图形学的基础几何表达。
- **劣势**：作为离散表达，如果网格分辨率不足，表面会不够平滑，编辑难度较高；如果三角形质量差（出现尖锐的锐角），后续优化计算会存在误差。其核心局限在于表达能力有限。

十一、网格细分（Mesh Subdivision）

多边形网格的核心局限是表达能力有限，而网格细分是提升网格表达能力的重要手段。通过插值增加顶点数量，提升几何的自由度，从而更精确地表达连续的几何体。

细分的思想可以类比曲线的细化：曲线由离散的控制顶点定义，通过不断在顶点间插值，可以逐步恢复出连续的曲线。几何细分也是同理，通过不断二分加点、细化网格，逐步逼近真实的连续几何体。

细分算法有很多种，本节介绍两类最经典的算法：Catmull-Clark细分和Loop细分。选择细分算法时，通常希望算法能保持网格的拓扑特性：四边形网格细分后仍为四边形网格，三角网格细分后仍为三角网格。

11.1 Catmull-Clark细分

Catmull-Clark细分由Catmull和Clark于1978年共同提出，在图形学领域应用极其广泛。模型可以只存储低精度的粗糙版本，在需要近景展示时，通过多层Catmull-Clark细分提升精度。该技术从学术界普及到游戏、动画电影等领域，两位发明者也因此获得2006年奥斯卡技术奖。

算法步骤：

1. **生成新顶点**
   - 面心点：为每个面计算中心点，四边形的面心即四个顶点坐标的平均值。
   - 边中点：每条边的新顶点并非单纯的两端顶点平均，而是同时考虑这条边共享的两个面的面心，做加权平均得到。（保证得到更加光滑的曲面，而不是只增加面片数量）
2. **更新旧顶点**
   对原有的每个顶点，通过加权平均计算新的位置，权重规则为：
   - 原顶点自身权重为 $n-3$（n 为该顶点相邻的面的数量）。
   - 相邻所有面的面心的平均值，权重为1。
   - 相邻所有原始边的边中点的平均值，权重为2。
     最终将三者加权后除以 n，得到更新后的顶点坐标。
3. **构建新拓扑**
   将每个面心点与对应四条边的边中点相连，再将更新后的旧顶点与相邻的边中点相连，每个原始四边形会被均匀拆分为4个小四边形。

递归执行以上步骤，即可完成多层细分。

算法特点：

- 支持四边形网格、三角网格，或是二者混合的网格。
- 无论原始网格是三角还是四边形式，经过一次Catmull-Clark细分后，都会得到纯四边形网格，且后续细分永远保持四边形拓扑。
- 细分结果表面平滑，能够很好地保留原始几何体的曲率特征。

11.2 Loop细分

Loop细分适用于三角网格，细分后永远保持三角网格拓扑，逻辑相对更简洁。

算法步骤：

1. **基本拆分规则**
   每条边取中点，将一个三角形拆分为4个小三角形，这是Loop细分的递归基础。
2. **新顶点计算**
   边上插入的新顶点并非简单的边中点，而是采用加权平均计算：
   - 边的两个端点各占3/8权重。
   - 该边所属两个三角形的对角顶点各占1/8权重。
     对于边界上的边，由于没有相邻面，直接取两端顶点的中点，保证边界形状不发生偏移。
3. **旧顶点更新**
   原有顶点也需要根据邻接信息加权更新，综合考虑自身历史位置与周围邻居的位置，通过特定权重计算得到新坐标。
4. **构建新拓扑**
   连接各边的新顶点，形成4个小三角形，递归执行即可。

算法特点：

- 每次细分后面片数变为原来的4倍，几何自由度与表达能力大幅提升。
- 能够很好地表达表面曲率，细分结果趋于平滑，非常适合三角网格。

网格细分的各类操作，都需要基于半边数据结构来实现，需要频繁查询顶点的邻接边、边的邻接面等拓扑信息。

十二、网格参数化（Mesh Parameterization）

12.1 概念与应用

网格参数化的核心是建立三维表面与二维UV空间的映射关系，让三维表面上的任意一个顶点，都能唯一映射到二维UV坐标上。

这项技术在图形学中应用广泛，最核心的应用是纹理映射：

- 比如给动物模型绘制表面纹理（眼睛、毛色等），可以先在2D平面上绘制纹理图，再通过UV映射将纹理贴到3D模型表面。
- 优势在于可以复用2D图像处理的所有技术，批量修改模型的外观。比如同一款骆驼模型，可以通过修改纹理图快速得到不同毛色、不同亮度的效果。

除了颜色纹理，纹理映射还可以存储更多几何信息：

- 法向图（Normal Map）：在UV空间存储表面法向量。即使模型本身的网格很粗糙，也可以通过法向图插值得到平滑的法向，在渲染时实现更真实的光照效果，提升视觉质感。

这类存储几何信息的纹理也被称为几何纹理（Geometry Image），现代显卡的缓冲区也专门支持这类数据，服务于渲染等下游应用。

12.2 常见的参数化方法

不同领域有各自适配的参数化方法，以下是几类经典的投影方式：

1. **墨卡托投影（Mercator Projection）**
   沿经纬线展开球面，是地球仪等球面模型最常用的参数化方式。
   - 优势：数学映射逻辑简单，保角性好，方向不会发生扭曲。
   - 劣势：不保面积，高纬度地区的面积会被大幅放大，比如地图上的南极洲、俄罗斯看起来面积远大于实际值。
2. **立体投影**
   选取一个虚拟视点，从视点向几何体发射射线，射线与平面的交点就是三维点投影到二维UV空间的位置。
   - 特点：保角性较好，但单个投影只能覆盖几何体的半面，完整表达需要至少两个投影。
3. **兰伯特投影（Lambert Projection）**
   属于非线性投影，核心目标是尽可能保证面积一致。
   - 特点：保面积效果好，通常单张投影可以较好地展开半球面，完整球面需要两张投影图。

参数化的核心需求通常包括保角（共形）、保面积、保长度，但三者很难同时满足。从三维曲面展开到二维平面，无法同时做到完美保角又完美保面积，只能根据应用需求权衡。
在CAD等工业领域，最优参数化是核心问题：比如汽车表面、机翼的设计，需要严格的保角保面积展开，且有统一的行业标准展开方式，后续的温度扩散、空气阻力等计算，都要基于展开后的平面进行。
图形学领域对参数化的要求相对宽松，主要服务于纹理映射，只需要尽量减少纹理的扭曲与偏差即可。

不同领域也衍生了专属的参数化方案：

- 接近球体的几何体，适合用球面参数化，保角效果较好。
- 人脸模型有专门的参数化方案：先人工标定眼角、嘴角等关键特征点，再通过优化得到整体的UV映射，是行业内的成熟方案。
- 工业元器件常用基于包围盒的参数化：先构建模型的包围盒，再将包围盒展开为二维平面，实现表面到UV空间的映射。

12.3 参数化的分类

根据模型是否有自然边界，平面参数化可以分为两类：

- 有边界模型：比如人脸，以模型的自然轮廓为边界，在边界约束下展开内部表面。
- 无边界模型：比如完整的球体，需要先选择一条剖分曲线将模型切开，以切开的曲线为边界进行展开。剖分曲线通常选择在隐蔽、平整的位置，减少对纹理效果的影响。

12.4 基于弹簧能量的参数化方法

这是一种通用的参数化方法，核心思想基于弹性势能优化。
可以直观理解为：将一块布蒙在三维几何体上，布上的每个点对应几何体的一个顶点；将布从几何体上取下并平铺，布会在弹性作用下自然收缩成平面，此时布上各点的位置就是对应的UV坐标。

数学上的实现逻辑：

- 假设所有相邻顶点之间都连接着一个理想弹簧，整个系统的总能量是所有弹簧的弹性势能之和。
- 弹簧的劲度系数可以人工设定，简化情况下可以统一设为1。
- 优化目标是最小化系统的总弹性势能。（也就是找展开布面后的二维的稳定状态）

需要注意的是，如果没有边界约束，理想弹簧系统的最优解是所有点坍塌成一个点，能量为0。因此必须添加边界条件：固定边界顶点的UV坐标，只优化内部顶点的位置，使得总弹性势能最小。

这是一个二阶能量最小化问题，可以通过求导找到最优解：能量的一阶导数为0时，系统达到能量最小的稳定状态。该问题不需要梯度下降等迭代优化，可以直接求解得到解析结果：每个内部顶点的最优位置，是其所有一阶邻接顶点的加权平均，权重由弹簧的劲度系数决定。如果所有弹簧劲度系数都为1，那么顶点坐标就等于邻接顶点坐标的平均值。

具体求解时，可以将问题转化为线性方程组 $A\,x = B$ 的形式：

- 矩阵 A 由顶点的连接关系与权重系数构成。
- 向量 B 由固定的边界顶点坐标构成。

可以通过高斯-赛德尔迭代等方法求解方程组，得到所有内部顶点的UV坐标。

基础的均匀权重方案只和邻接关系相关，和模型本身的边长、角度无关，展开效果有限。后续衍生出了很多改进的权重方案：

- 针对三角网格，为了提升保角性，可以用角度的余切值作为权重。
- 还有一些方法同时考虑边长因素，进一步优化展开效果。

大家可以先实现基础的均匀权重版本，再尝试改进的权重方案，对比不同方案在保角、保长度方面的效果。

十三、Lab2 介绍

本次课程发布了Lab2，建议大家在十一假期了解题目，假期后开始完成即可。几何部分的Lab难度比第一次Lab更高，需要大量运用半边数据结构完成操作。
Lab2已经提供了半边数据结构的基础实现（DCEL结构）以及对应的API，大家需要通过调用半边的`next`、`previous`、`twin`等接口，完成代码填空，实现对应的几何算法。

Lab2包含以下核心内容：

1. **Loop细分**
   实现Loop细分算法，对立方体等模型进行细分。0次细分时是原始立方体，随着细分次数增加，模型逐渐趋近于球体，但仍保留一定的棱角特征。程序支持控制相机视角查看结果。
2. **网格参数化**
   实验提供人脸模型，固定人脸边界的顶点坐标，需要大家实现基于弹簧能量的参数化优化，通过高斯-赛德尔迭代求解线性方程组，最小化系统能量。
   优化前，棋盘格纹理贴在人脸上会有明显的扭曲；随着迭代次数增加，弹簧能量逐渐降低，纹理会越来越平整，最终均匀地贴合在人脸表面。

后续课程还会讲解更多几何处理算法，包括网格平滑（Mesh Smoothing）、网格简化、基于Marching Cubes的表面重建等，这些内容会更多涉及离散微分几何的核心思想。
本次讲解的细分、参数化等内容，可以作为半边数据结构的练习。后续的网格简化、平滑等操作，还会涉及边删除等更复杂的半边操作，对半边数据结构的掌握要求更高。

本次课程提前结束，谢谢大家。